自然语言处理

深度学习基础

杨沐昀

哈尔滨工业大学计算学部
语言技术研究中心

yangmuyun@hit.edu.cn

主要内容

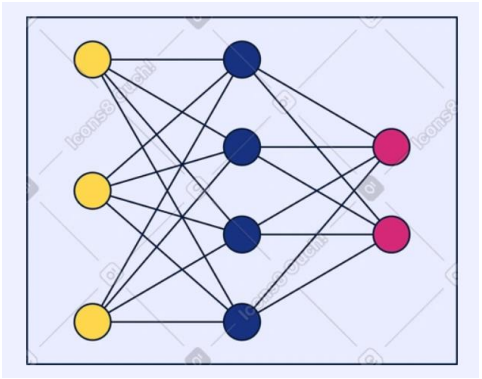
- 神经网络基础
- 前馈神经网络
- 卷积神经网络
- 循环神经网络
- Transformer

神经网络概述



人类智能行为的来源：神经系统

模仿

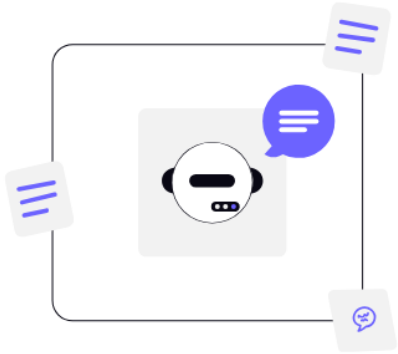


人工神经网络（Artificial Neural Network，ANN）



人类的行为（如对话等）

模仿



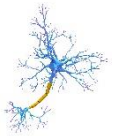
人工智能行为

神经网络概述

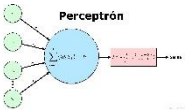
- 人工神经网络（Artificial Neural Network，ANN）
  - 一种模仿大脑神经系统的数学模型；
  - 由大量模拟神经元功能的计算单元构成；
  - 人工神经元通过“连接权重”相互关联；
  - 训练过程中调整这些权重来学习。

神经网络概述

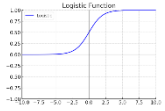
三个核心概念：



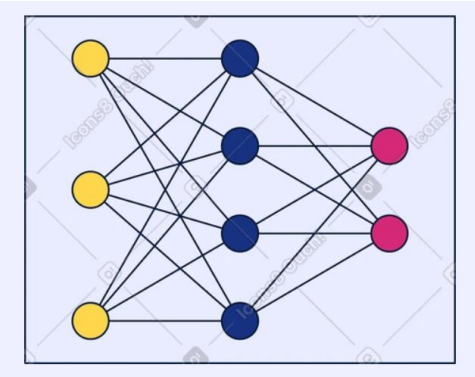
神经元



感知机



激活函数

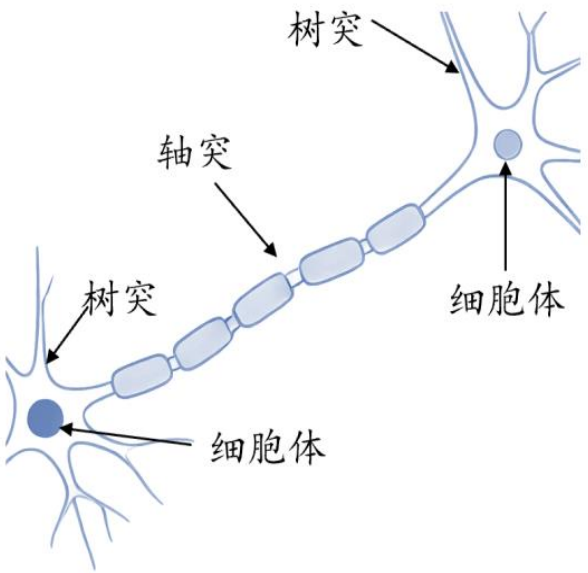


人工神经网络

- 本章介绍的网络架构：
  - 前馈神经网络；
  - 卷积神经网络；
  - 循环神经网络；
  - Transformer；

人工神经网络的生物学基础

- 神经元：
  - 生物神经系统的基本组成单元；
  - 基于权重的自适应学习



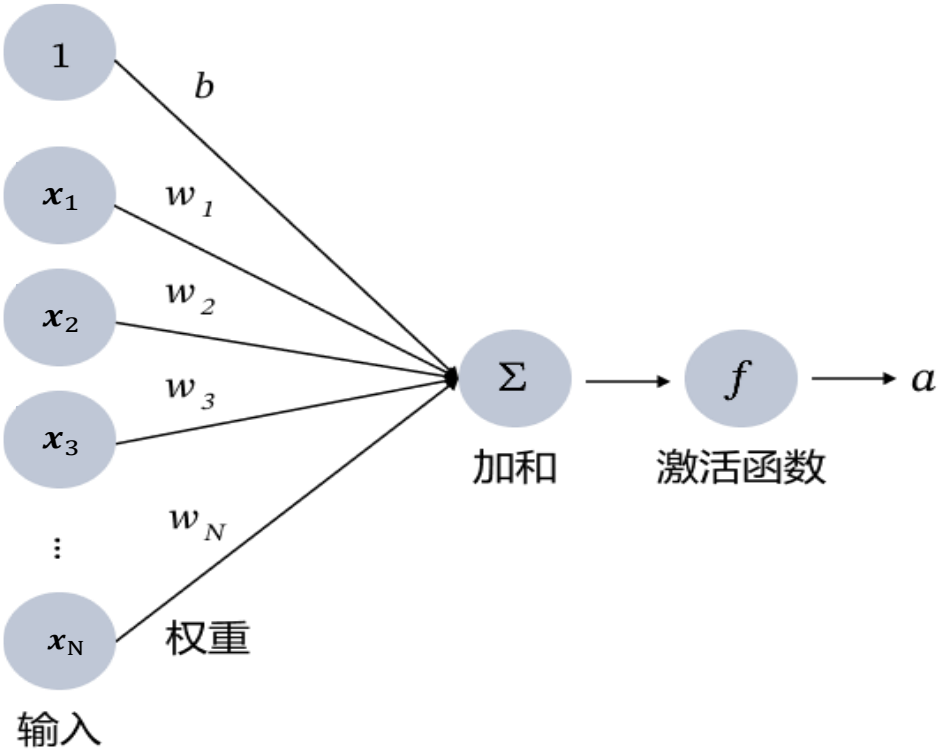
自适应学习的数学解释：

$$\Delta w_{ij} = \eta \cdot x_i \cdot y_j$$

x_i 和 y_j 分别是神经元 i 和 j 的信号输出， η 为学习速率。

人工神经元

- 机器感知（Machine Perception，MP）模型（1943，Warren McCulloch，Walter Pitts）

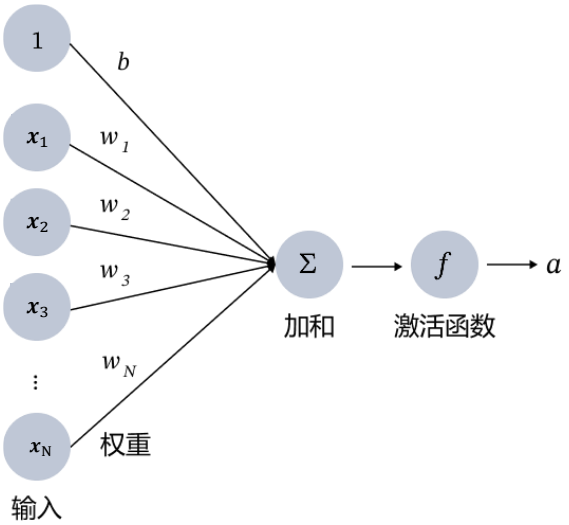


人工神经元

- N个输入$x_1, x_2, x_3, \ldots, x_N$——生物神经元的多个树突接收到的信息；
- 达到一定阈值后，神经元将被激活；
- 具体来讲，对于输入x= $[x_1, x_2, x_3, \ldots, x_N]$，神经元的计算方式为：

$$z = \sum_{i=1}^{N} w_i x_i + b$$

$$a = f(z) = \begin{cases} z, & z \geq \theta \\ 0, & z < \theta \end{cases}$$



感知机

- 感知机（Perceptron）
  - 也称感知器，是人工神经网络的最早模型之一
  - 由心理学家弗兰克·罗森布拉特（Frank Rosenblatt）于1957年提出。
  - 感知机的结构与MP神经元的结构一样，其区别在于：**感知机的权重不需要预先设定，可以在模型训练过程中拟合数据自动学习。**

感知机

- 感知机学习算法：

对于每个输入样本 $(\mathbf{x}, y)$，若感知机的预测输出 $\hat{y}$ 与实际标签 y 不一致，即 $\hat{y} \neq y$，则更新权重和偏置：

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (y - \hat{y}) \cdot \mathbf{x}$$

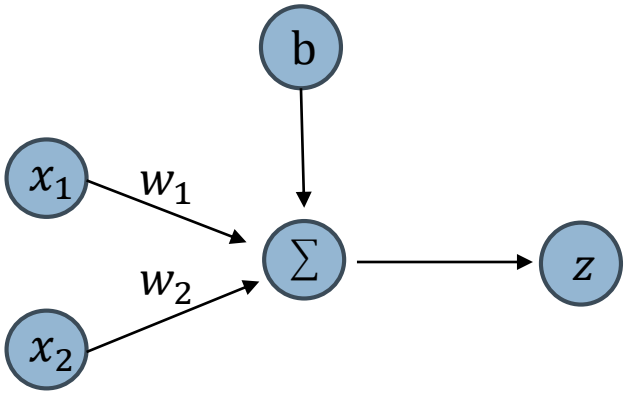
$$b \leftarrow b + \eta \cdot (y - \hat{y})$$

其中，η 为学习率，用于控制更新步长，$y - \hat{y}$ 为误差。

感知机

- 一个简单的二分类示例：
  - 一个感知机用于判断天气条件是否适合户外活动。
  - 输入：温度（x_1）和湿度（x_2），权重 $w_1 = 0.6$ 和 $w_2 = -0.8$，偏置 $b = 0.2$。
  - 假设设定的条件是：若安静活动性值 $z \geq 0$，则适合户外活动；否则不适合。





感知机

- 一个简单的二分类示例：

情况1：当在凉爽的雨天，温度为20℃，湿度为80时，即 $(x_1, x_2) = (20,80)$ 时:

$$z = 0.6 \cdot 20 + (-0.8) \cdot 80 + 0.2 = -51.8 < 0 \Rightarrow$$ 不适合



情况2：当在温暖的晴天，温度为25℃，湿度为10时，输入为 $(x_1, x_2) = (25,10)$ 时:

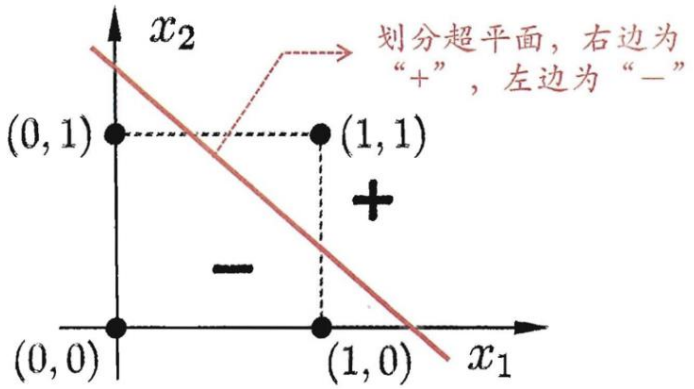
$$z = 0.6 \cdot 25 + (-0.8) \cdot 10 + 0.2 = 7.2 > 0 \Rightarrow$$ 适合



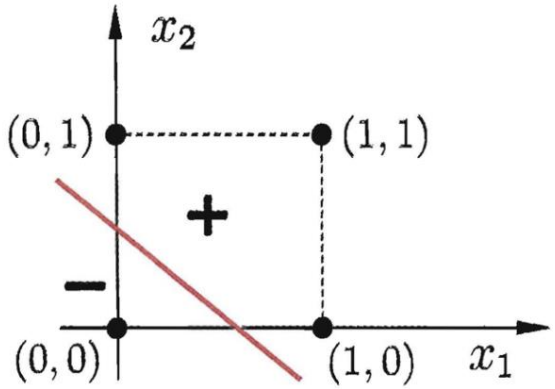
感知机

- 感知机的缺陷：只能处理线性可分的问题。
- 例如，感知机无法解决“异或（XOR）问题”，因为异或问题中的数据是非线性可分的，无法通过一条直线将两类数据分开。
- 这个局限性导致了感知机在上世纪60年代末陷入发展瓶颈。

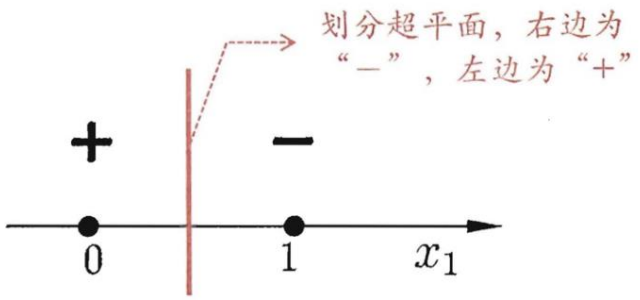
感知机无法处理的异或问题



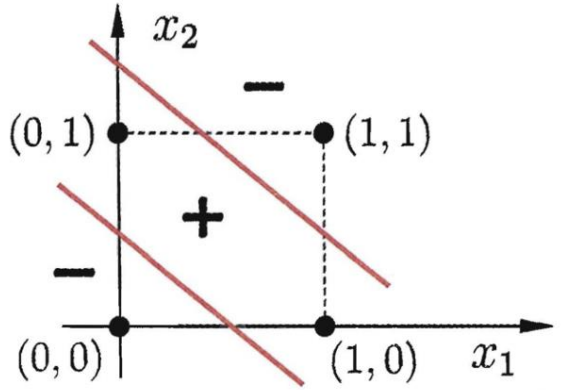
(a) “与”问题 $(x_1 \wedge x_2)$



(b) “或”问题 $(x_1 \vee x_2)$



(c) “非”问题 $(\neg x_1)$



(d) “异或”问题 $(x_1 \oplus x_2)$

激活函数

激活函数（Activation Function）是神经网络中每个神经元的核心组件之一，决定了输入信号通过神经元后的输出形式。

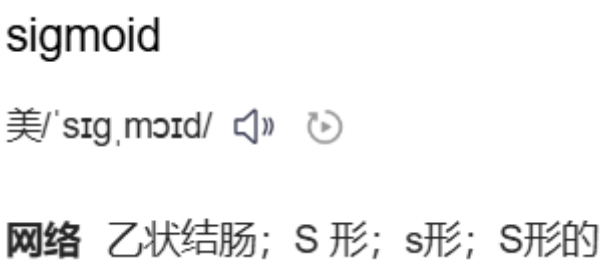
- 单层感知机——无法处理异或问题；
- 多层感知机——在引入激活函数前，可从线代知识得知多层等价于单层；
- 多层感知机——在引入激活函数后，网络能够形成更复杂的表示。

激活函数

- 激活函数需要具备的性质：
  - **连续并可导（允许少数点上不可导）的非线性函数**；
  - 激活函数及其导函数要**尽可能的简单**，有利于提高网络计算效率；
  - 激活函数的**导函数的值域要在一个合适的区间内**，不能过大也不能过小，否则会影响训练的效率和稳定性。

激活函数

（1）sigmoid函数：



- Sigmoid函数是指一类S型曲线函数；
- 在神经网络中常用的Sigmoid函数有两个：逻辑斯谛（Logistic）函数和双曲正切（hyperbolic tangent, 简写为Tanh）函数。

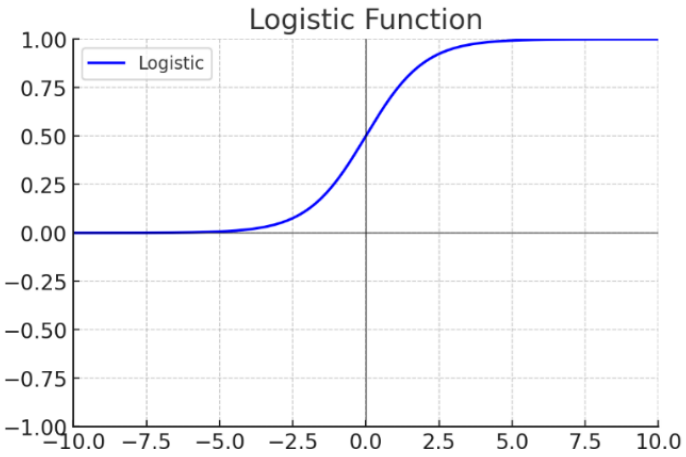
激活函数

（1a）Logistic函数

Logistic函数的定义为：

$$\sigma(x) = \frac{1}{1+\exp(-x)}$$

函数曲线如图所示：



激活函数

- Logistic函数的优点：
  - 值域为（0,1）既可用于二分类任务，又可将输出值视为概率；
  - 可以用作一种门控机制，控制输入信息的输出量。

- Logistic函数的缺点：
  - 曲线两端的斜率逐步接近于0，容易导致**梯度消失问题**。
  - Logistic函数值恒大于0，在多层神经网络中非零中心化的输出会使其后一层的神经元的输入发生偏置偏移（bias shift），并进一步使得梯度下降的**收敛速度变慢**。

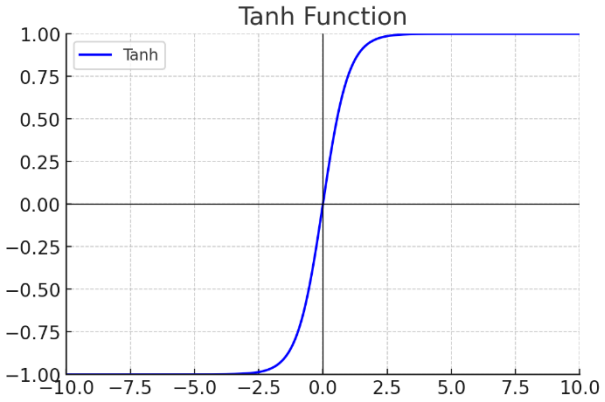
激活函数

（1b）Tanh函数

Tanh 函数的定义为：

$$\tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)} = 2\sigma(2x) - 1$$

函数曲线如图所示：



激活函数

Tanh实际上是sigmoid函数的缩放和平移版本；

为什么要进行这样的变换：

- 输出以0为中心，加速收敛、缓解偏置偏移问题；

缺点：

- 与Logistic函数一样，依然存在饱和区，容易导致梯度消失问题。

思考：你会如何设计激活函数来避免这样的问题呢？注意保持激活函数需要具备的基本性质。

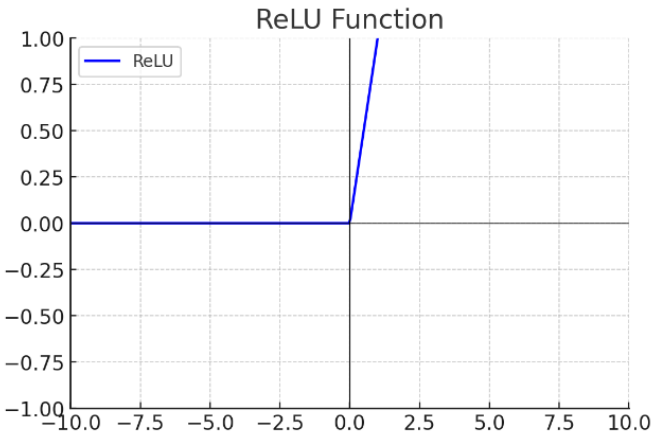
激活函数

（2）ReLU函数

ReLU（Rectified Linear Unit，修正线性单元），也叫Rectifier函数，是目前深度神经网络中经常使用的激活函数，ReLU 函数的定义为：

$$\mathrm{ReLU}(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

其曲线如图所示：



激活函数

- ReLU函数的优点：
  - 缓解梯度消失问题；
  - 计算简单（不涉及幂运算），提高训练效率；

- ReLU函数的缺点：
  - 非零中心化，在训练过程中会一定程度地影响模型的收敛效率；
  - 可能导致特定神经元不再被更新的Dead ReLU现象。

激活函数

为了解决Dead ReLU问题，研究者提出了一些改进版本：

- Leaky ReLU在负值区域引入一个小的非零斜率 α ：

$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \end{cases}$$

通常设定 α 为一个小的常数（如0.01）。
这样始终存在梯度，在理论上避免了神经元的死亡。

- 但是， Leaky ReLU需要手动设置或学习超参数 α；效果不一定总是比原始ReLU好；在负区间引入的线性关系可能并非最优。

激活函数

激活函数	优点	缺点
Sigmoid	输出概率（0-1），光滑梯度	非零中心、计算量大
Tanh	零中心输出（-1~1）	梯度消失、计算量稍大
ReLU	计算高效、缓解梯度消失	死亡神经元、输出非零中心
Leaky ReLU	缓解死亡神经元	死亡神经元，输出非零中心

激活函数

总结：

- 激活函数是神经网络学习非线性关系的关键，决定了神经网络的表达能力。
- 不同的激活函数对网络的收敛速度、梯度消失或爆炸以及计算效率都有显著影响。
- 在实际应用中，通过实验选择合适的激活函数，能够提升模型的表现，避免不必要的训练瓶颈。

主要内容

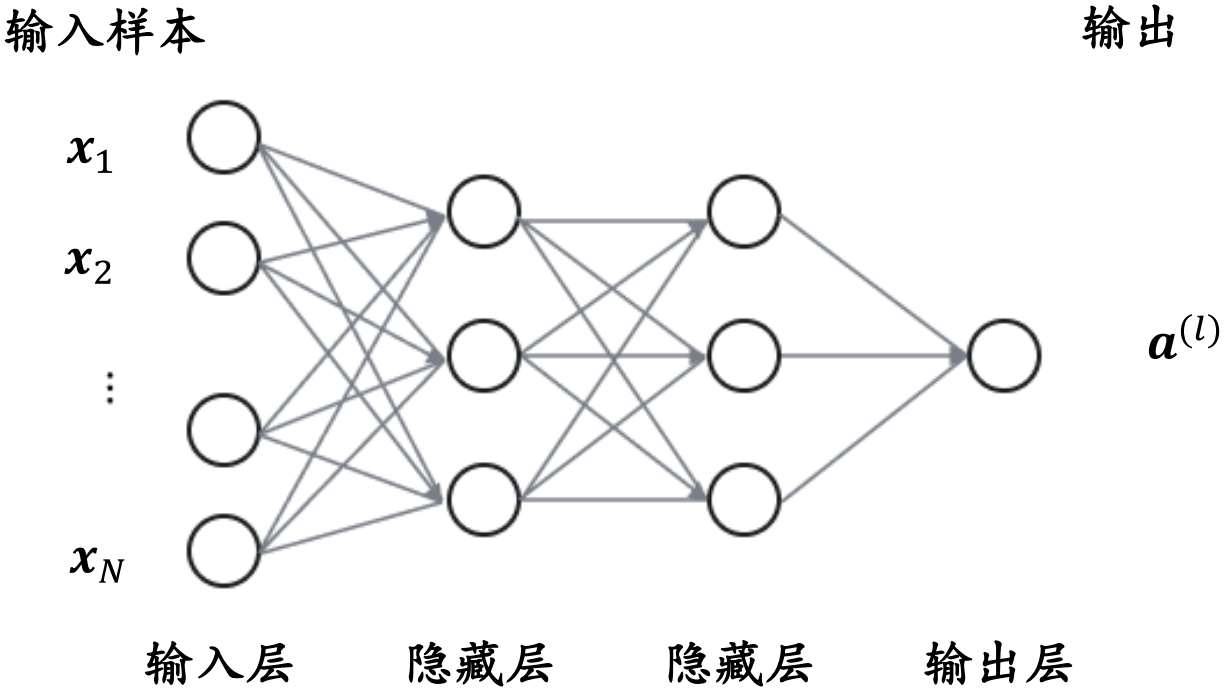
- 神经网络基础
- 前馈神经网络
- 卷积神经网络
- 循环神经网络
- Transformer

前馈神经网络

前馈神经网络（Feedforward Neural Network, FNN）是一种最基本的人工神经网络结构，又称为**多层感知器**（MultiLayer Perceptron, MLP）。

- 名称来源：数据在网络中只向前传播，没有反馈连接。
- FNN通过将输入数据进行多层非线性变换，每一层将前一层的输出作为输入，应用权重、偏置和激活函数，产生新的输出（称为激活值）。

前馈神经网络



多层前馈神经网络

前馈神经网络

FNN包括以下关键组件：

- **输入层：**接收原始数据（例如，图像的像素值、文本的词向量、数值特征）。每个输入节点对应数据的一个特征。
- **隐藏层：**这是网络进行特征提取、转换和组合的地方。隐藏层中的每个节点（神经元）接收来自前一层所有节点的输入，计算加权和，加上偏置，然后通过一个激活函数。
- **输出层：**产生网络的最终预测结果。输出层的节点数和激活函数取决于任务类型。

FNN的训练

三个训练步骤：

前向传播（Forward Propagation）、损失函数（Loss Function）和反向传播（Back propagation）

- 前向传播将输入数据逐层传递，计算生成输出预测值；
- 损失函数是通过计算预测输出与真实标签之间的偏差，定义训练的最小化目标；
- 反向传播根据损失的大小和方向逐层更新网络参数。

FNN的训练

我们先给出将要使用的FNN计算符号：

符号	含义
L	神经网络的层数
N	输入层神经元的个数
N_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\boldsymbol{W}^{(l)} \in \mathbb{R}^{N_l \times N_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\boldsymbol{b}^{(l)} \in \mathbb{R}^{N_l}$	第 $l-1$ 层到第 l 层的偏置向量
$\boldsymbol{z}^{(l)} \in \mathbb{R}^{N_l}$	第 l 层中的神经元的输出
$\boldsymbol{a}^{(l)} \in \mathbb{R}^{N_l}$	第 l 层神经元的激活值

FNN的训练

前向传播：

- 前向传播的过程可以理解为“信息流”从输入到输出的逐层传递；
- 假设输入为向量 $\boldsymbol{x} = [x_1, x_2, \ldots, x_N]$，令 $\boldsymbol{a}^{(0)} = \boldsymbol{x}$，网络的第 l 层中的神经元的输出可以表示为：

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

- 然后，激活值通过激活函数 $f_l(\cdot)$ 进行计算，得到：

$$\mathbf{a}^{(l)} = f_l\left(\mathbf{z}^{(l)}\right)$$

FNN的训练

损失函数：

- 在前馈神经网络中，损失函数（Loss Function）用于衡量模型的预测输出与真实标签之间的差异。
- 损失函数定义了网络的学习目标，通过最小化损失函数的值，模型能够不断调整参数以优化预测效果。

假设神经网络对输入样本的最终预测结果为$\hat{y}$。对于一个样本，其真实标签为 y 。损失函数 $L(\hat{y}, y)$ 衡量预测值 $\hat{y}$ 与真实值 y 之间的差异。

常见的损失函数

（1）均方误差损失函数

计算每个样本的预测值和真实值之间的误差平方；

假设有 N 个样本，第i个样本的预测值为 $\hat{y}_i$ (即网络的输出)，真实值为 y_i ，则均方误差定义为:

$$L_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

其中， $y_i - \hat{y}_i$ 表示第 i 个样本的预测误差，将它平方可以确保误差的正负不会相互抵消。

常见的损失函数

（2）交叉熵损失函数

衡量模型预测的类别概率分布与真实类别分布之间的差异。

假设一个多分类任务有 C 个类别，模型的输出通过 Softmax 激活函数得到每个类别的预测概率分布。设模型输出的原始得分向量为 $z = [z_1, z_2, \ldots, z_C]$，则经过 Softmax 激活后得到每个类别的预测概率 $\hat{y} = [\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_C]$：

$$\hat{y}_i = \text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

常见的损失函数

设真实标签用独热编码表示为 $y = [y_1, y_2, \ldots, y_C]$，其中 $y_i \in \{0,1\}$ 表示第i类是否为真实类别。

对于样本的多分类交叉熵损失定义为：

$$L(y, \hat{y}) = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

对于真实标签所在的类别 i，如果预测概率$\hat{y}_i$越接近1（即模型对该类别越有信心），损失就越低。

常见的损失函数

- 以二分类为例说明交叉熵损失：
  - 假设我们要判断一段文本的情感极性是“正面”还是“负面”。类别数$C = 2$。
  - 如果模型给出一段话情感类别 1（即“正面”）的概率为 $\hat{y}$，那么对应地，情感类别 0（即“负面”）的概率为$1 - \hat{y}$。
  - 根据交叉熵计算公式，可以得到二分类的交叉熵损失如下：

$$L(y, \hat{y}) = -(y\log(\hat{y}) + (1 - y)\log(1 - \hat{y}))$$

(接下页)

常见的损失函数

假设输入文本“自然语言处理非常有趣”属于“正面情感”（即 $y=1$），而模型预测它是“正面情感”的概率为 $\hat{y}=0.9$。代入交叉熵损失函数：

$$L(y,\hat{y})=-(y\log(\hat{y})+(1-y)\log(1-\hat{y}))$$

代入 $y=1$ 和 $\hat{y}=0.9$ 得到：

$$L(1,0.9)=-\log(0.9)\approx 0.105$$

可以看到模型预测的概率越接近真实的概率 1，损失越小。

常见的损失函数

- 为什么分类问题常采用交叉熵损失函数而不是平方损失函数?
  - 交叉熵损失能提供更大、更有效的梯度，使得模型学习得更快。
  - 从概率角度看，最小化交叉熵损失等价于最小化预测概率分布与真实概率分布之间的差异，这在数学上是极其自然和合理的。

简单来说，**交叉熵是为衡量概率分布差异而“生”的，而MSE是为衡量数值差异而“生”的。**

FNN的训练

- 反向传播算法（Back Propagation，BP）：
  - 根据损失函数的值调整权重和偏置；
  - 基于链式法则，将损失的梯度从输出层逐层传播回输入层。

反向传播算法改变模型权重，使得网络逐步逼近最优的预测效果，是训练的关键步骤。

下面，我们介绍反向传播计算的基本过程。

FNN的训练

给定一个样本 $(\boldsymbol{x}, \boldsymbol{y})$, 将其输入到神经网络模型中, 得到网络输出为 $\hat{\boldsymbol{y}}$。假设损失函数为 $\mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})$。

假设第$l+1$层未经过激活函数的网络输出为:

$$\boldsymbol{z}^{(l+1)} = \boldsymbol{W}^{(l+1)}\boldsymbol{a}^{(l)} + \boldsymbol{b}^{(l+1)}$$

从链式法则可得:

$$\frac{\partial \boldsymbol{z}^{(l+1)}}{\partial \boldsymbol{a}^{(l)}} = \left(\boldsymbol{W}^{(l+1)}\right)^{\mathrm{T}} \in \mathbb{R}^{N_l \times N_{l+1}}$$

(接下页)

FNN的训练

损失函数 $\mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})$ 关于参数矩阵$\boldsymbol{W}^{(l)}$中每个元素的偏导数为：

$$\frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \boldsymbol{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l)}}$$

其中$\frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l)}}$ 即是损失函数关于第 l 层的神经元 $\boldsymbol{z}^{(l)}$的偏导数,一般称为误差项，用 $\delta^{(l)}$ 来表示。从链式法则可得：

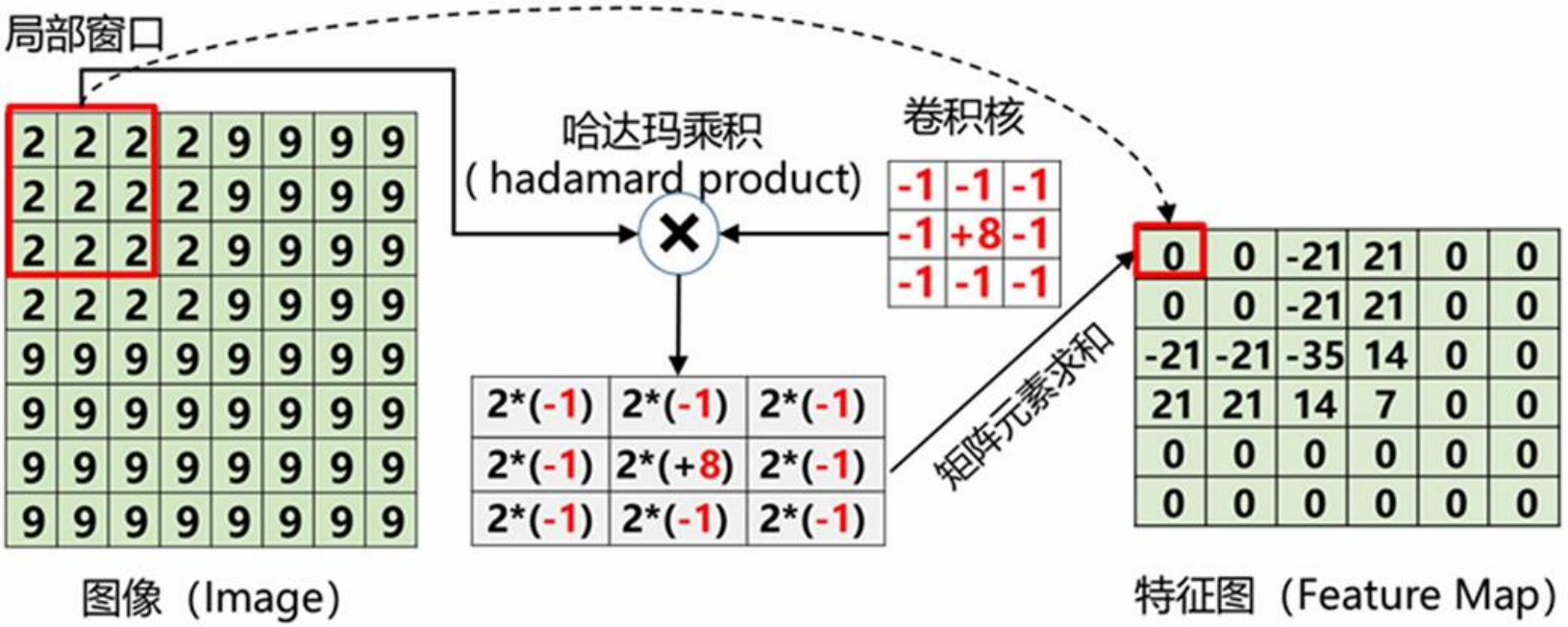
$$\delta^{(l)} \triangleq \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l)}} = \frac{\partial \boldsymbol{a}^{(l)}}{\partial \boldsymbol{z}^{(l)}} \cdot \frac{\partial \boldsymbol{z}^{(l+1)}}{\partial \boldsymbol{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}})}{\partial \boldsymbol{z}^{(l+1)}} = \frac{\partial \boldsymbol{a}^{(l)}}{\partial \boldsymbol{z}^{(l)}} \cdot \left(\boldsymbol{W}^{(l+1)}\right)^{\mathsf{T}} \cdot \delta^{(l+1)}$$

主要内容

- 神经网络基础
- 前馈神经网络
- 卷积神经网络
- 循环神经网络
- Transformer

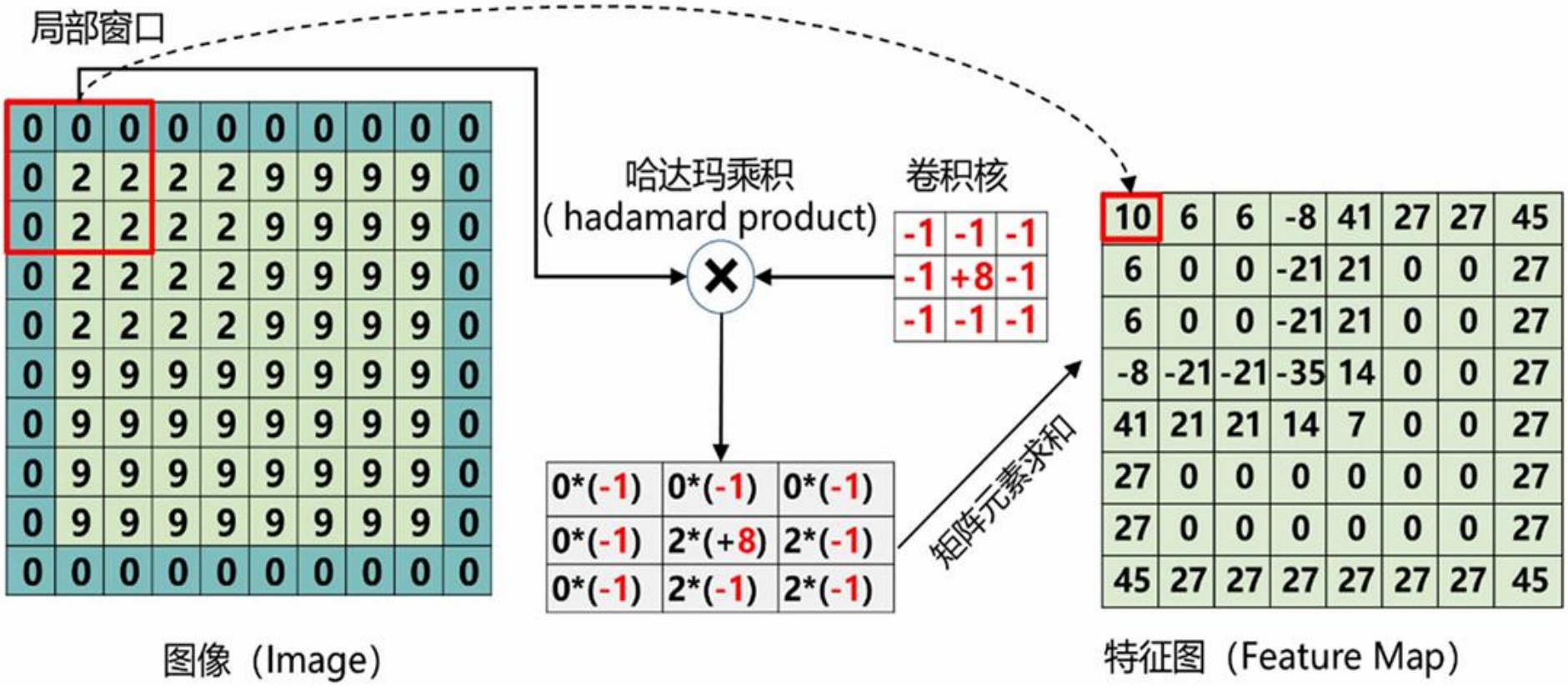
卷积

什么是卷积?


局部窗口
哈达玛乘积
(hadamard product)
卷积核
矩阵元素求和
图像（Image）
特征图（Feature Map）

卷积

宽卷积：填充0（padding）的卷积

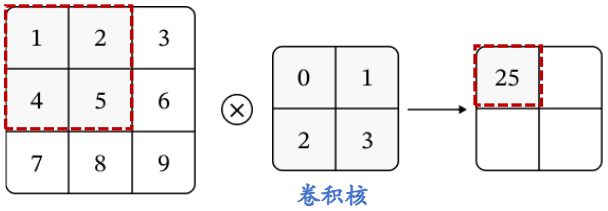


卷积

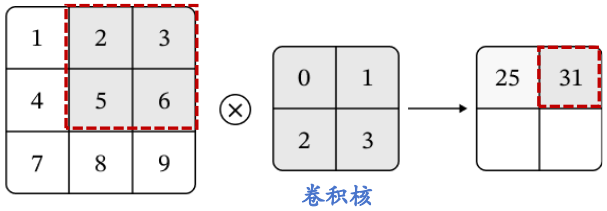
假设输入一幅图像I，卷积核 W 尺寸设置为 M × N，滑动步长设置为1，则卷积可表示为：

$$F(i,j) = (I * W)(i,j) = \sum_{m=0}^{\mathbb{M}-1} \sum_{n=0}^{\mathbb{N}-1} I(i+m, j+n) \cdot W(m,n)$$

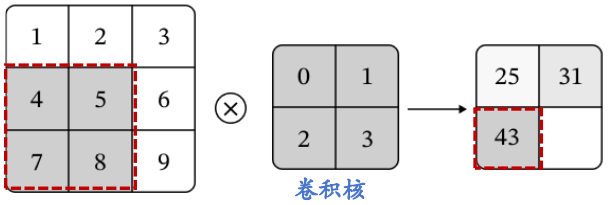
下面是一个简单的卷积过程：



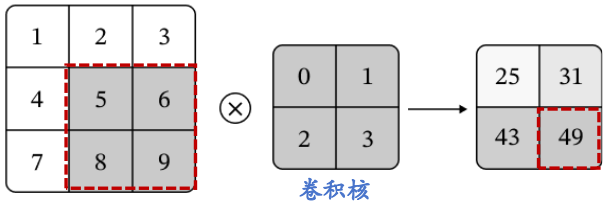
(a) $1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 = 25$



(b) $2 \times 0 + 3 \times 1 + 5 \times 2 + 6 \times 3 = 31$



(c) $4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 = 43$



(d) $5 \times 0 + 6 \times 1 + 8 \times 2 + 9 \times 3 = 49$

卷积

卷积处理图像的一个例子：



输入图像

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & +8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

卷积核



得到的特征图

卷积

使用不同的卷积核可以达到不同的图像处理效果：



输入图像

尖锐化处理的卷积核

0	-1	0
-1	5	-1
0	-1	0



边缘检测卷积核（一）

0	1	0
1	-4	1
0	1	0



模糊化处理的卷积核

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9



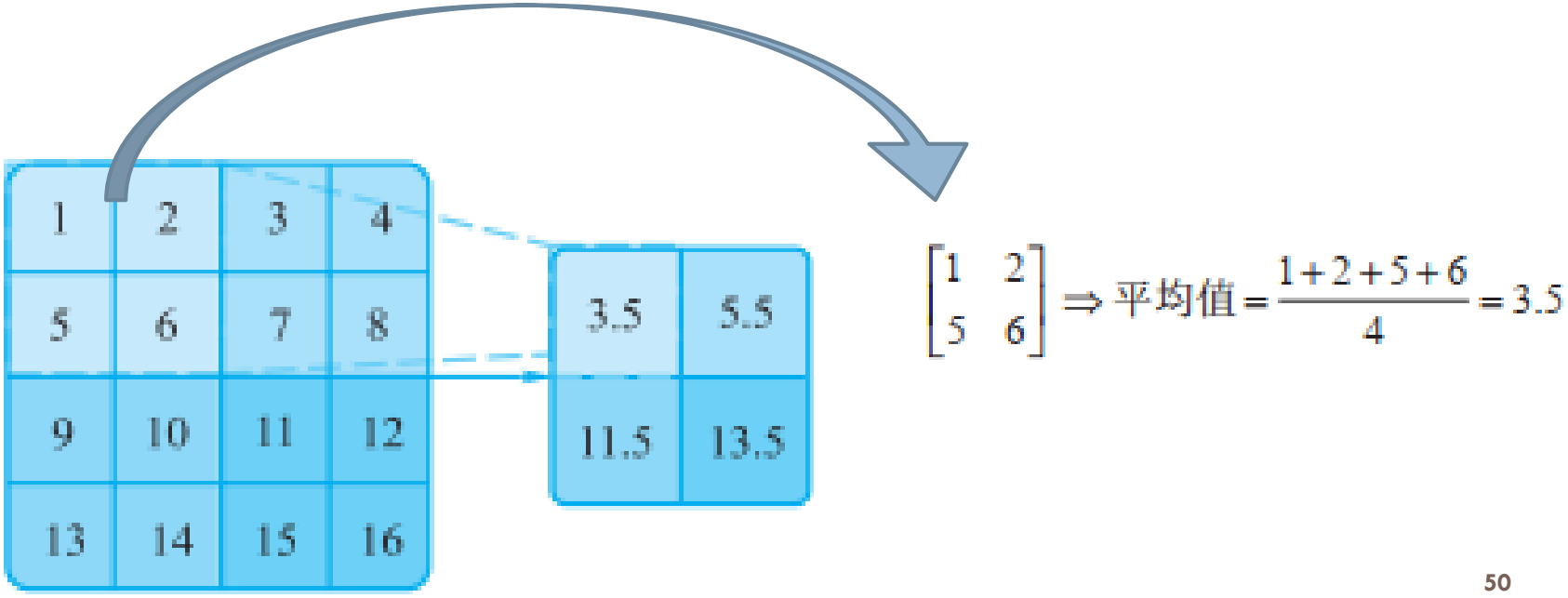
边缘检测卷积核（二）

1	0	-1
0	0	0
-1	0	1



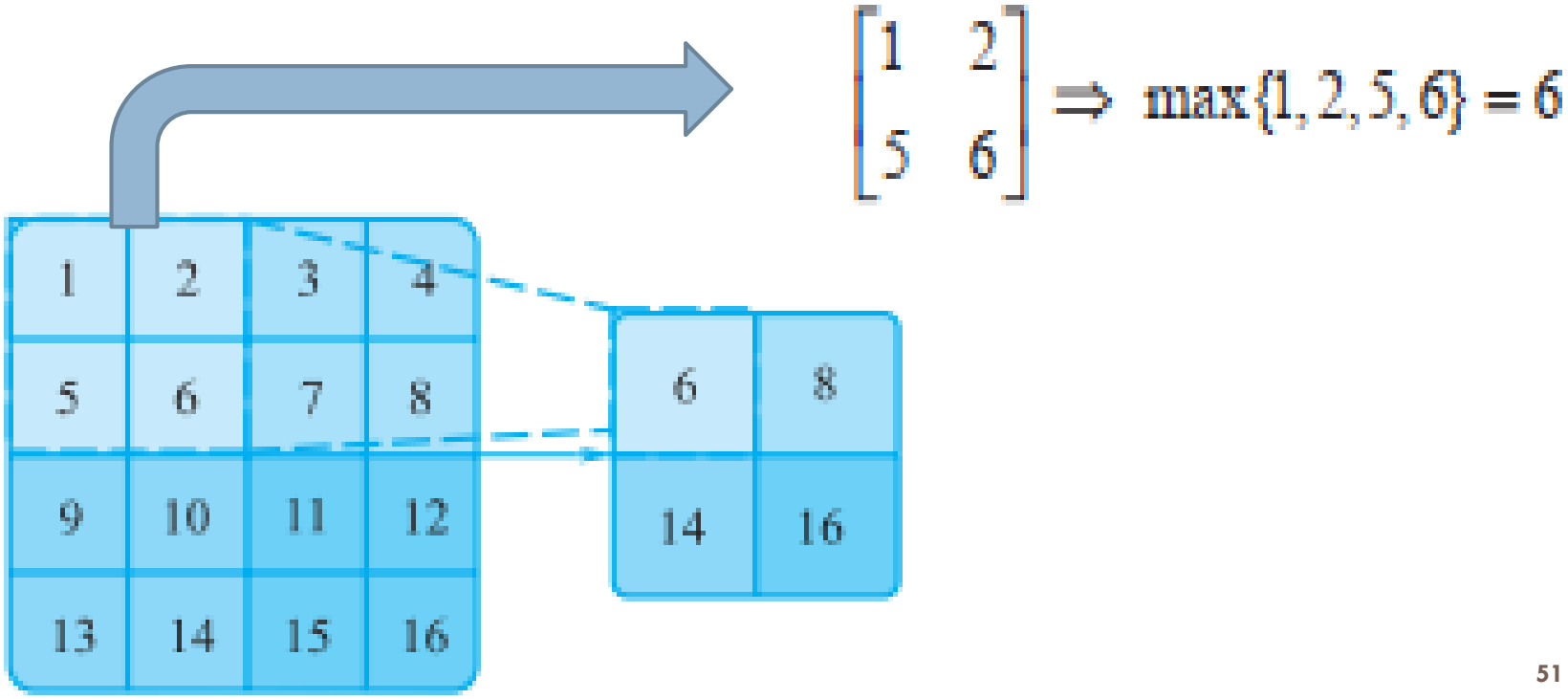
卷积vs池化

- 池化： （池化）窗口，滑动-- -类似卷积过程；
  平均池化，最大池化---计算策略不同
- 平均池化：对特定窗口进行均值计算



卷积vs池化

- 最大池化：对特定窗口取最大值作为输出



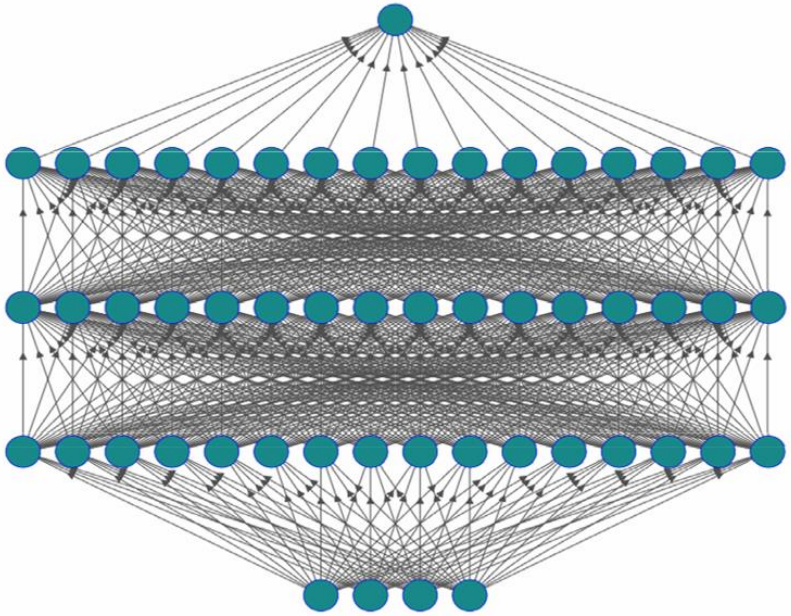
卷积神经网络概述

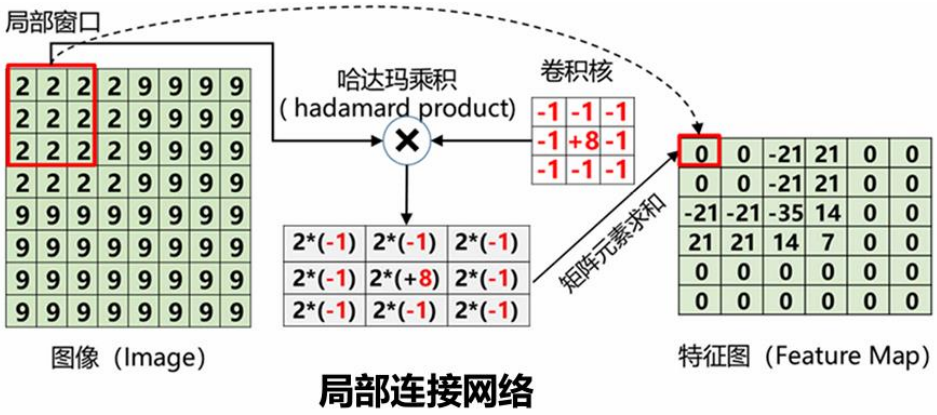
- 卷积神经网络（Convolutional Neural Networks, 简称CNN）是一 类包含卷积计算且具有深度结构的前馈神经网络，是深度学习的代表算法之一。
- 对卷积神经网络的研究始于二十世纪80至90年代
- 典型方法：LeNet-5、AlexNet、 VGGNet、 GooLeNet、ResNet等
- 卷积神经网络最先在图像分类上获得了成功，后来又在自然语言处理等领域中获得了较好的应用。

卷积神经网络概述

卷积神经网络的特点：

（1）局部连接和局部卷积



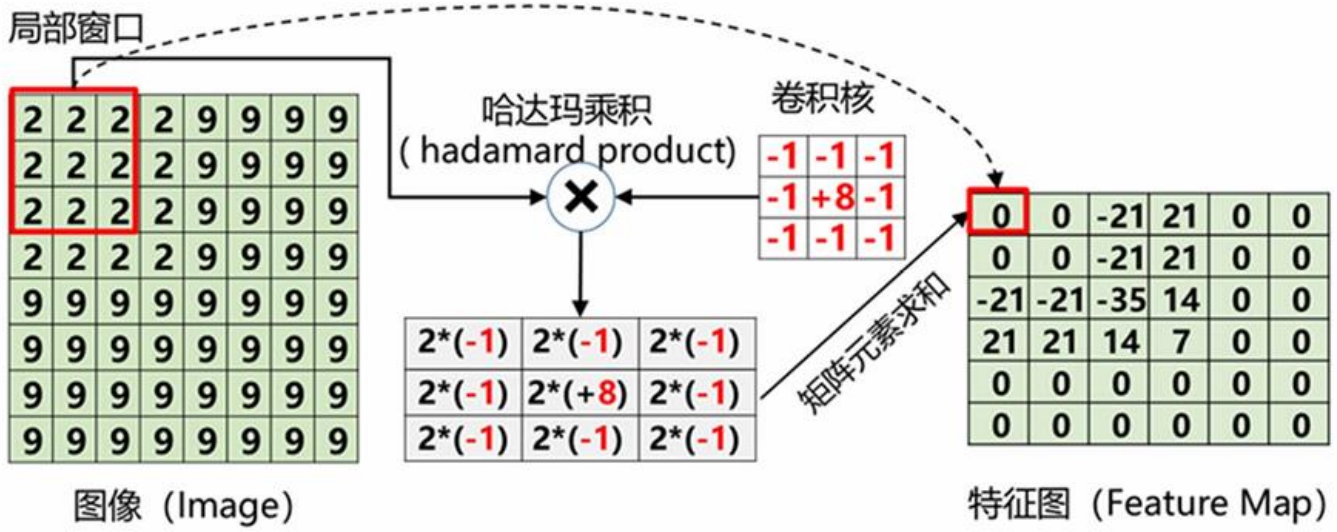


卷积神经网络概述

- 如前图，相对于一般的多层全连接网络，在卷积神经网络中采取的是**局部感知**的方式。
- 除去效率因素外，这点也是符合直觉的。对于图像而言，一般是局部的像素联系较为紧密，而距离较远的像素相关性则相对较弱。

卷积神经网络概述

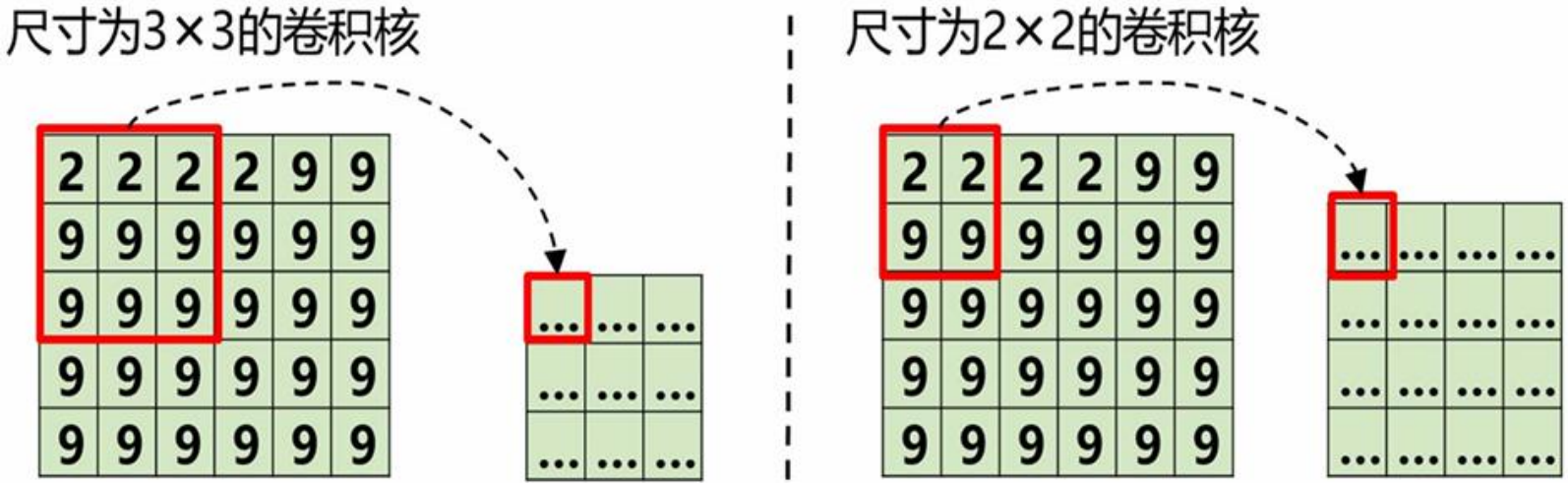
（2）参数共享



- 在上图中，无论计算图像的哪个位置，卷积核的参数都是相同的。
- 共享参数这一机制大大减少了参数的数量，从而有助于提高卷积计算的效率。

卷积神经网络概述

（3）多卷积核



- 为了充分提取特征，卷积神经网络一般使用多个卷积核。
- 这些卷积核的尺寸或者参数取值不同。 每个卷积核都会对输入图像进行卷积处理，生成另一幅图像。

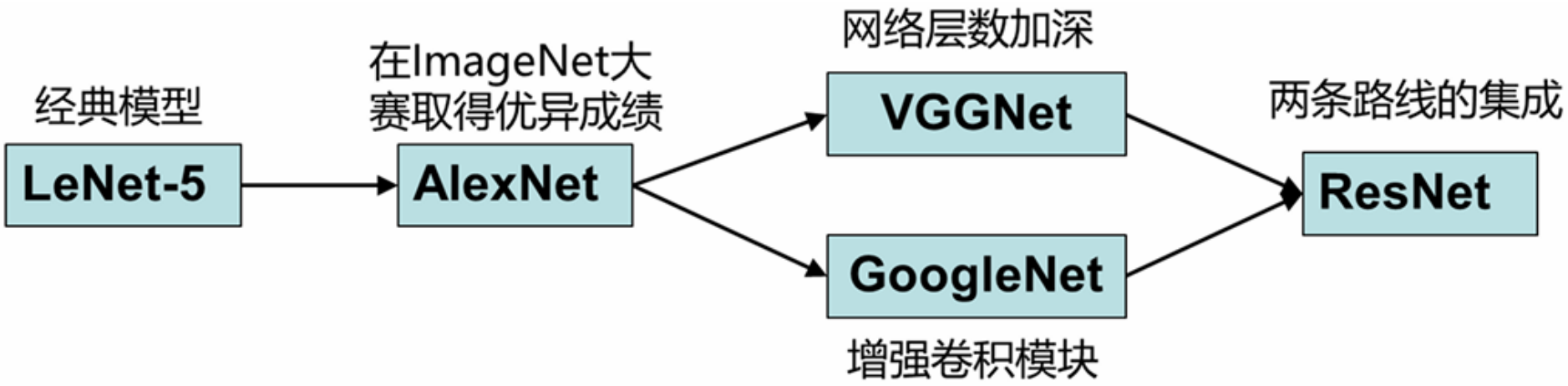
卷积神经网络概述

（4）池化处理

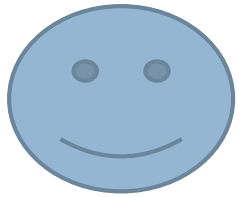
池化处理也叫作降采样处理，相应的层则叫池化层或者降采样层。池化处理对不同位置的特征进行聚合统计。池化主要有如下作用：

- 首先是**降维**。经过池化处理，特征数目得以压缩，计算效率也会提高；
- 其次是**降低模型的复杂度**。能够在一定程度上控制过拟合，提高分类器的泛化能力；
- 在图像识别领域，池化还能**提供平移和旋转不变性**。

经典卷积神经网络

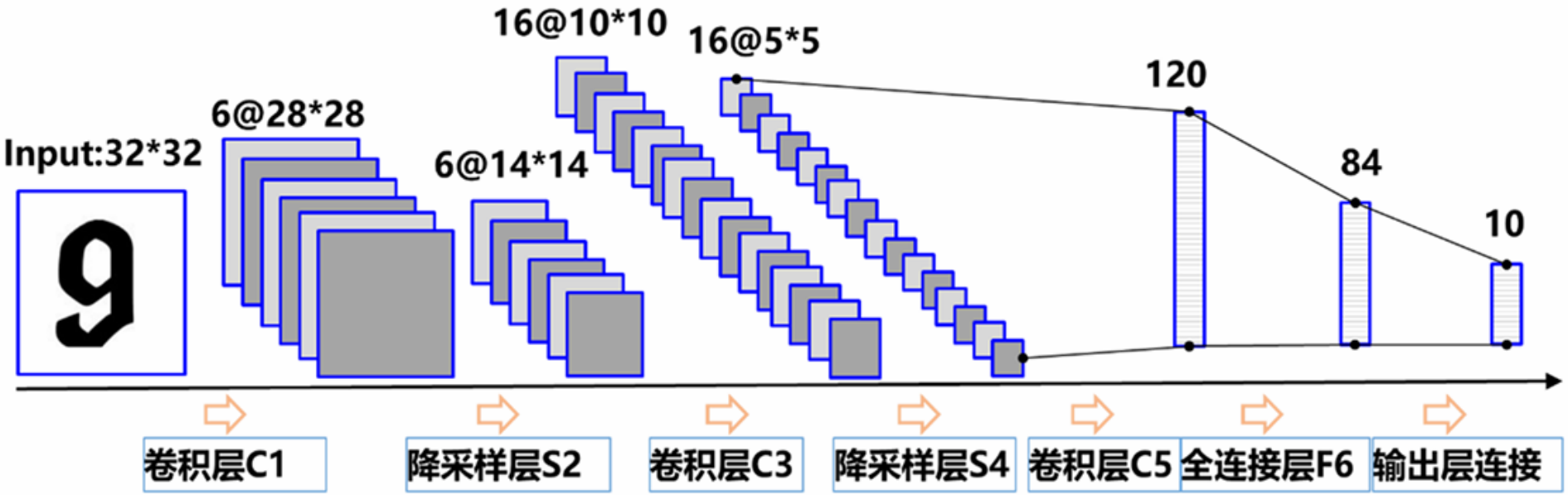

经典模型
LeNet-5
在ImageNet大
赛取得优异成绩
AlexNet
网络层数加深
VGGNet
GoogleNet
增强卷积模块
两条路线的集成
ResNet

LeNet-5

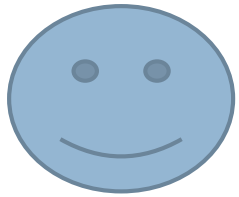


（1）LeNet-5

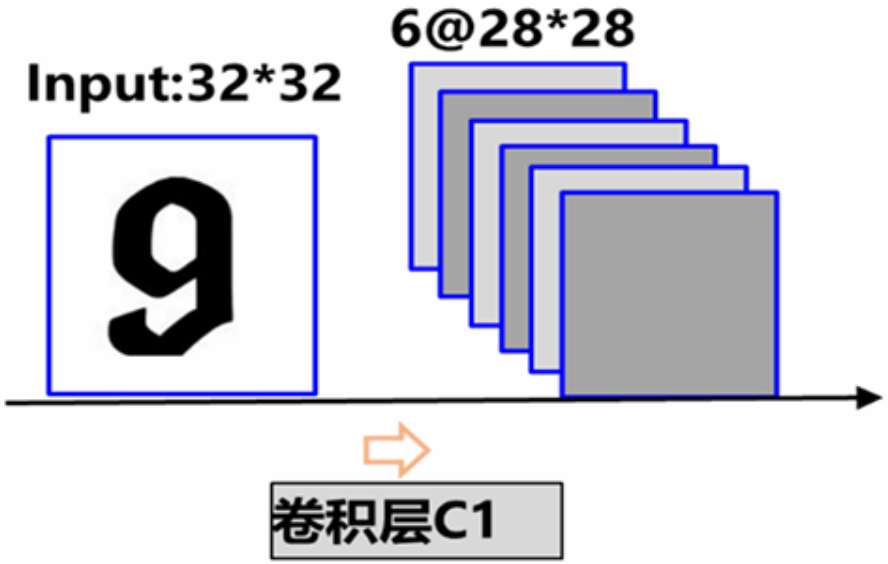
以数字手写体图像的识别问题为例：输入的图像尺寸为32x32像素，经过中间3个卷积层及2个降采样层和全连接层处理，可以输出该图像是10个数字之一概率有多大。



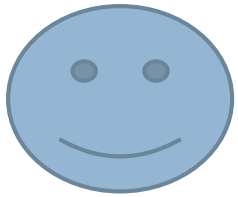
LeNet-5



- 卷积层C1的卷积核大小是多少?
- 该卷积层的总参数是多少?
- 卷积层C1中总的连接数目是多少?

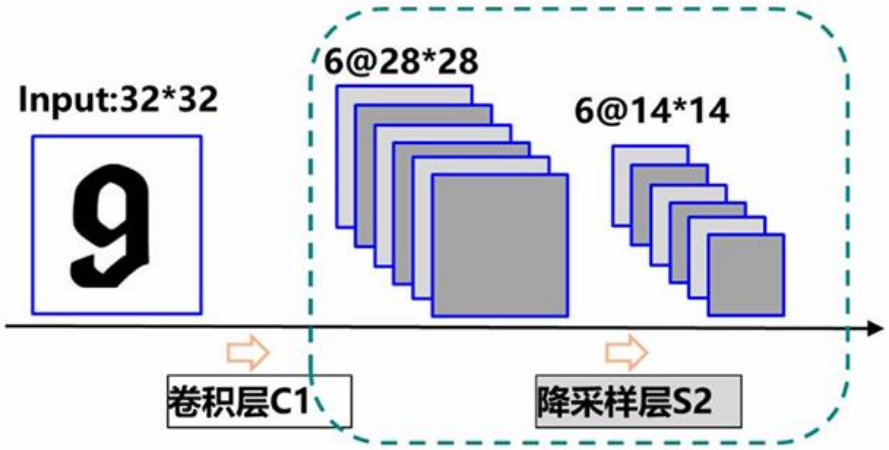


LeNet-5

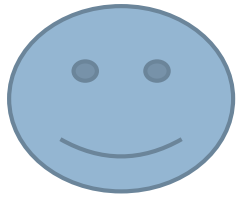


LeNet-5的降采样层（池化层）：

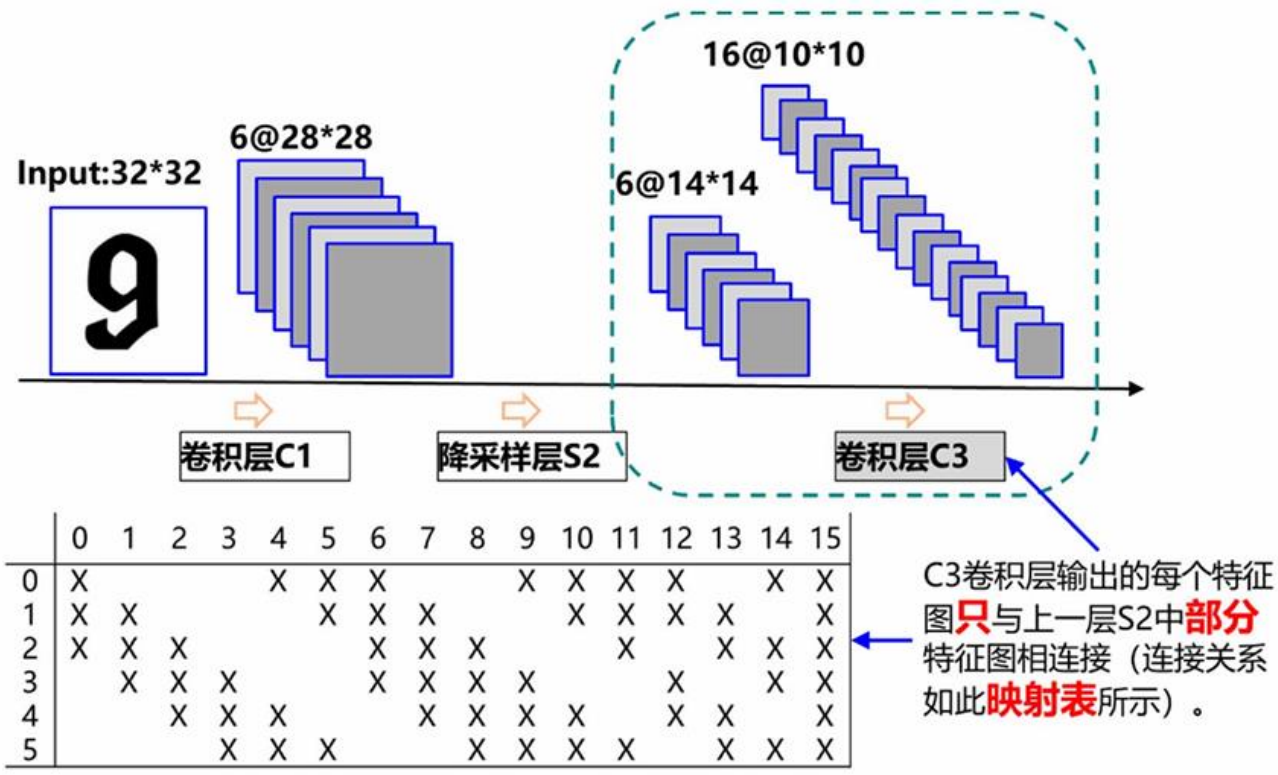
- 降采样层S2的输出仍然是6个特征图，然而其尺寸却有了降低。S2层有6个14x14像素的特征图。S2层的滑动窗口大小？



LeNet-5



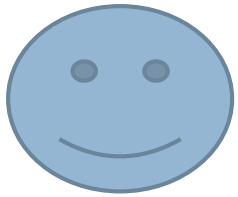
LeNet-5的卷积层C3（虚线框内）：



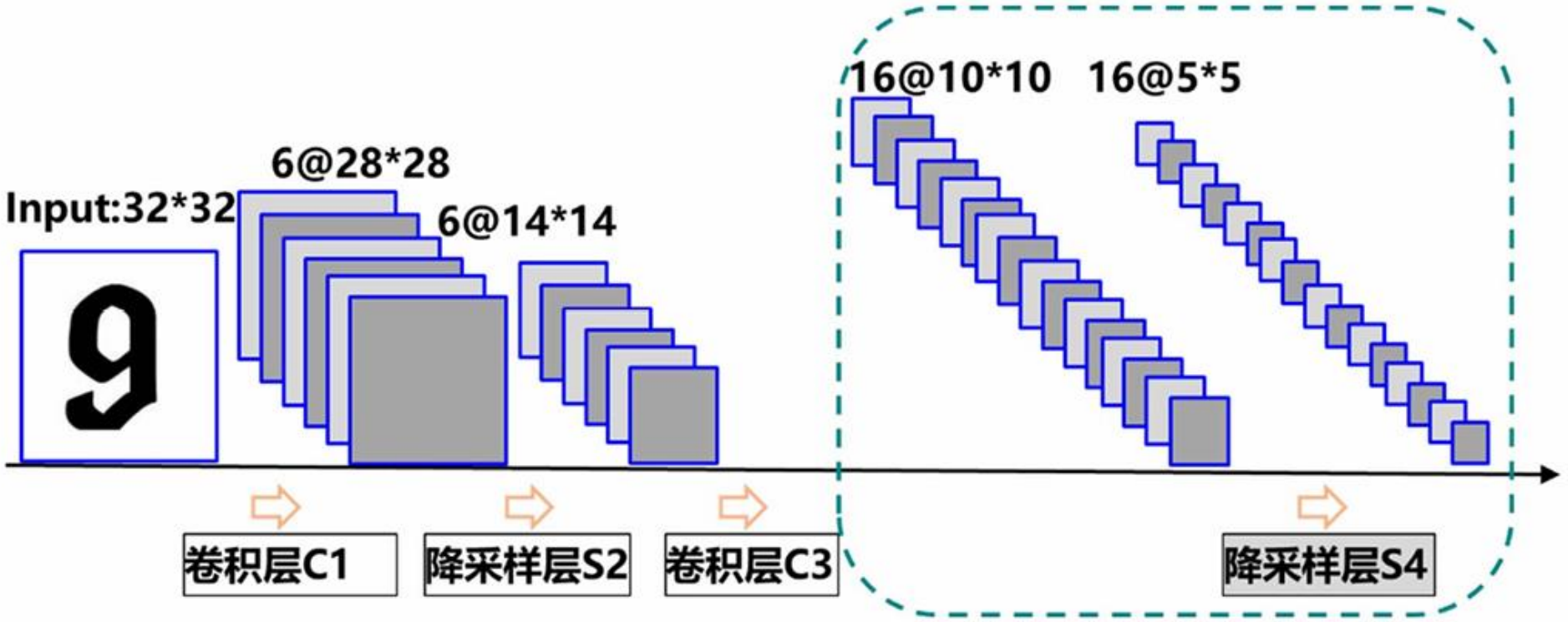
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X			X	X	X	X		X	X		X
5				X	X	X			X	X	X	X		X	X	X

C3卷积层输出的每个特征图**只**与上一层S2中**部分**特征图相连接（连接关系如此**映射表**所示）。

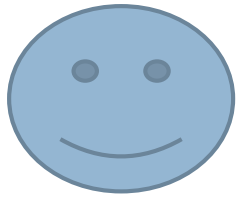
LeNet-5



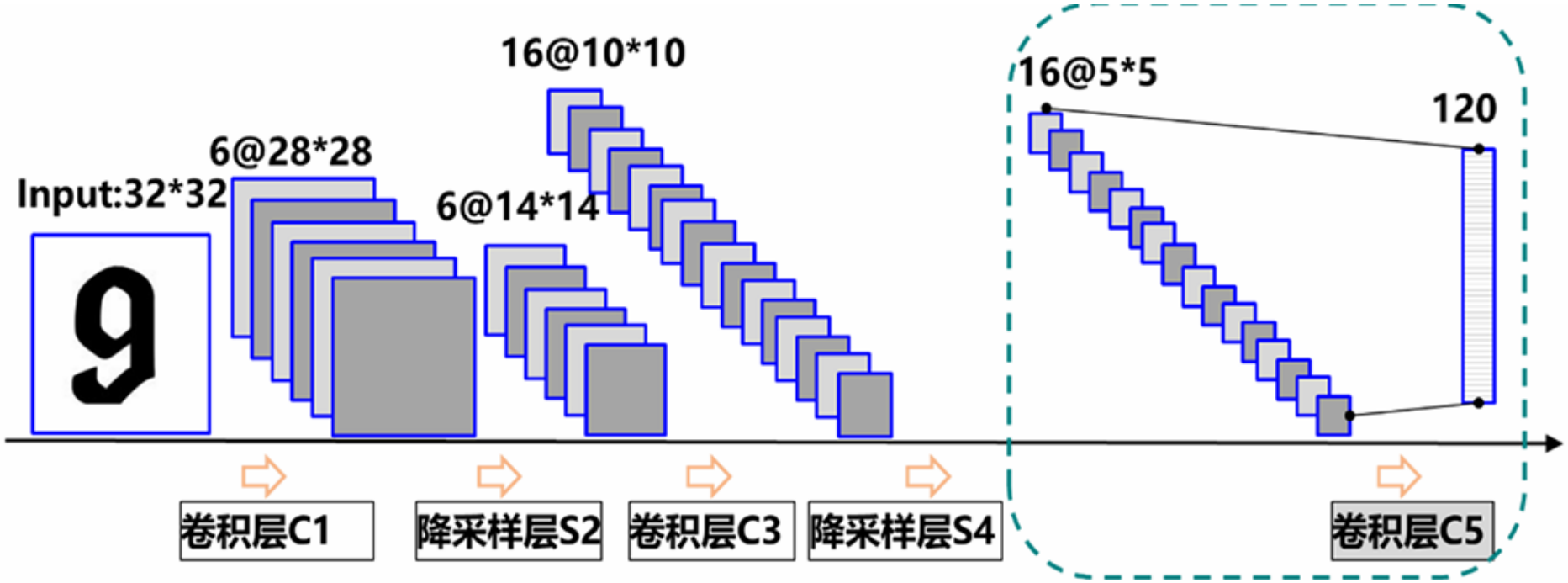
LeNet-5的降采样层S4（虚线框内）：



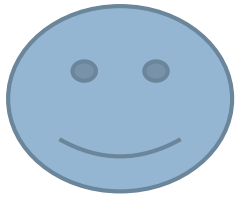
LeNet-5



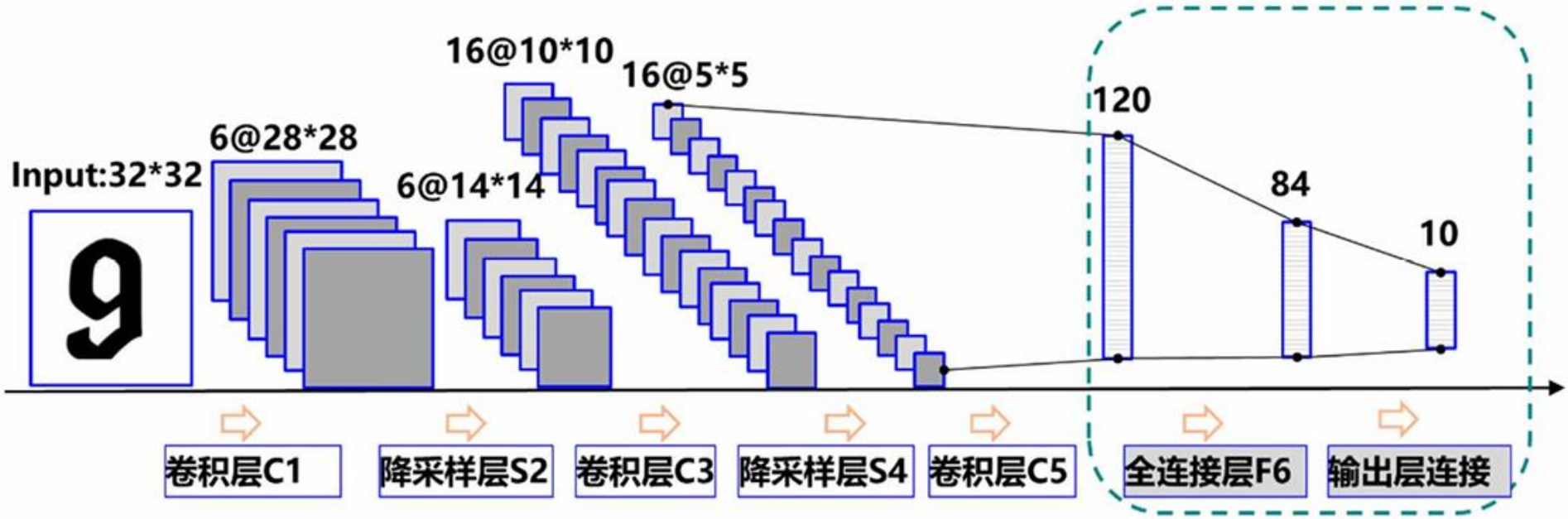
LeNet-5的卷积层C5（虚线框内）：



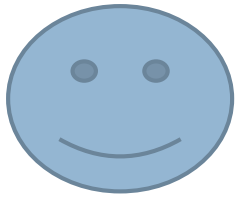
LeNet-5



LeNet-5的全连接层F6和输出层（虚线框内）：

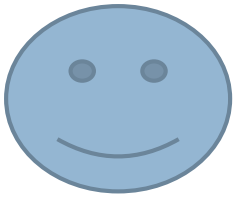


AlexNET

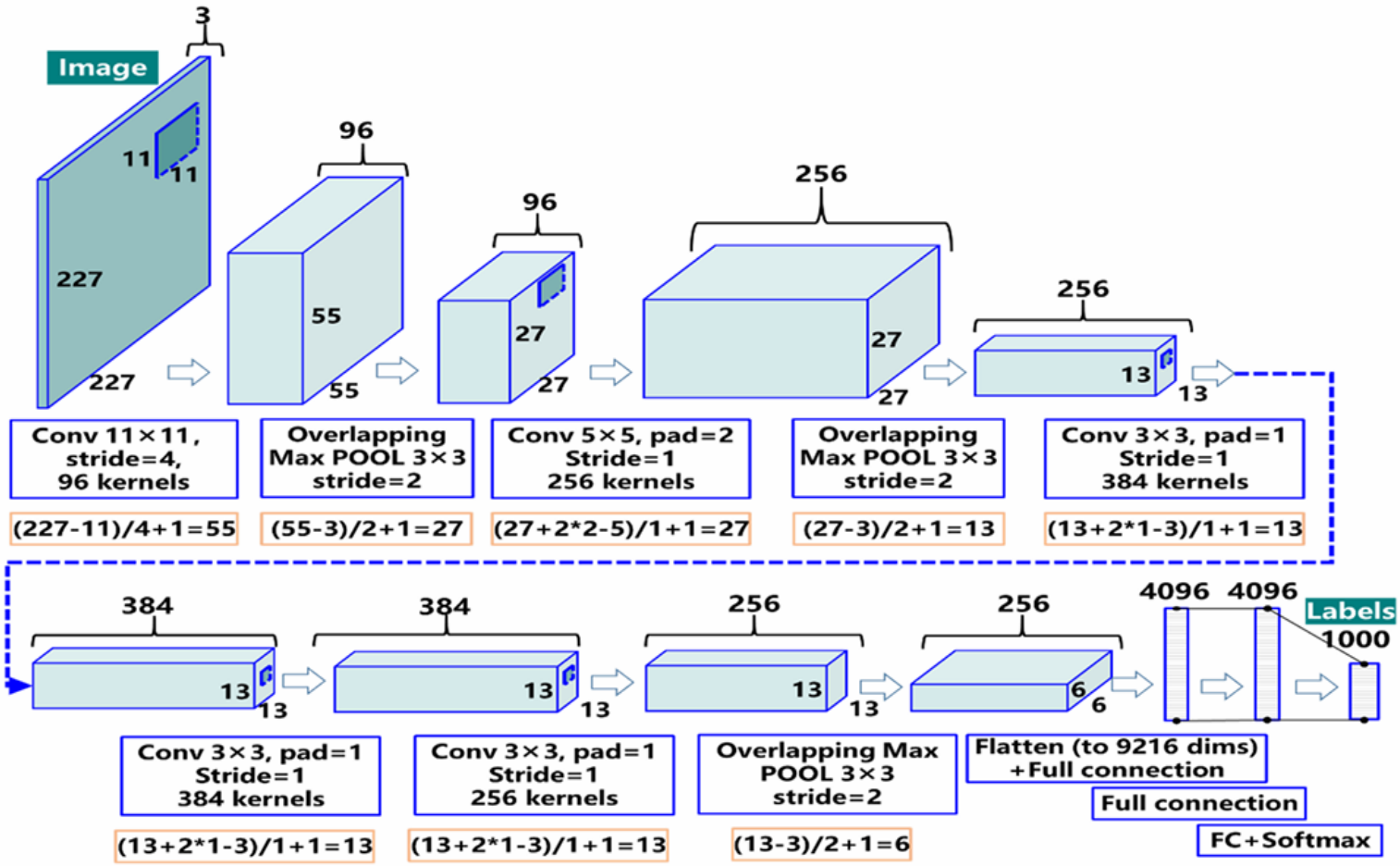


- AlexNet通过加深网络层数，在2012年举行的ImageNet大规模视觉识别挑战赛上以远超第二名的成绩获得冠军。该网络使用了ReLU激活函数和Dropout技术，缓解了训练深层网络时的过拟合问题。
- ImageNet是一个大规模的图像数据集，其规模自从2009年问世以来仍然在不断的扩充。

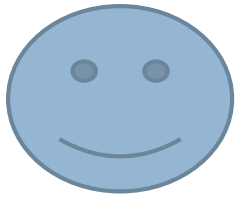
AlexNet



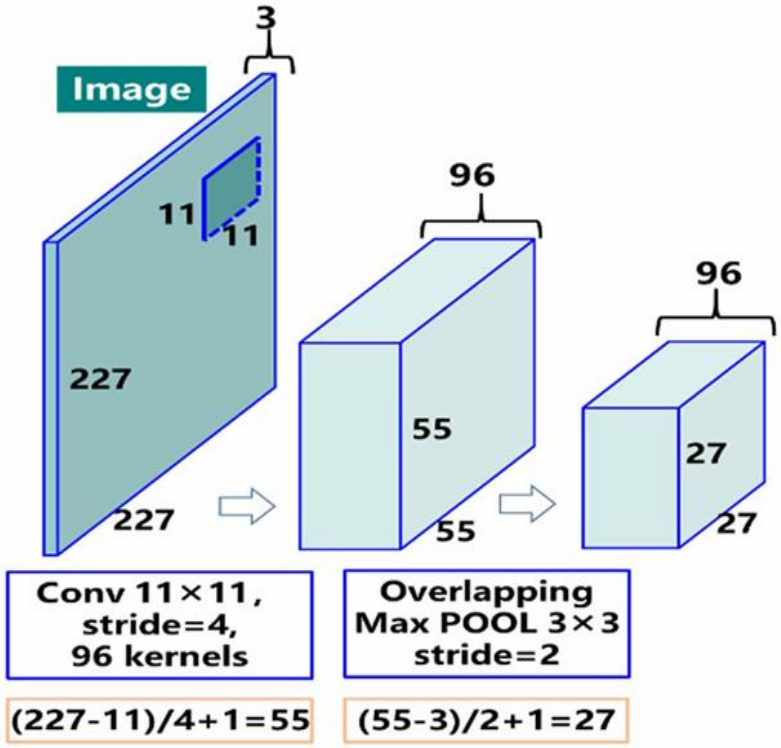
AlexNet的结构：



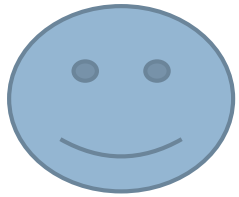
AlexNet



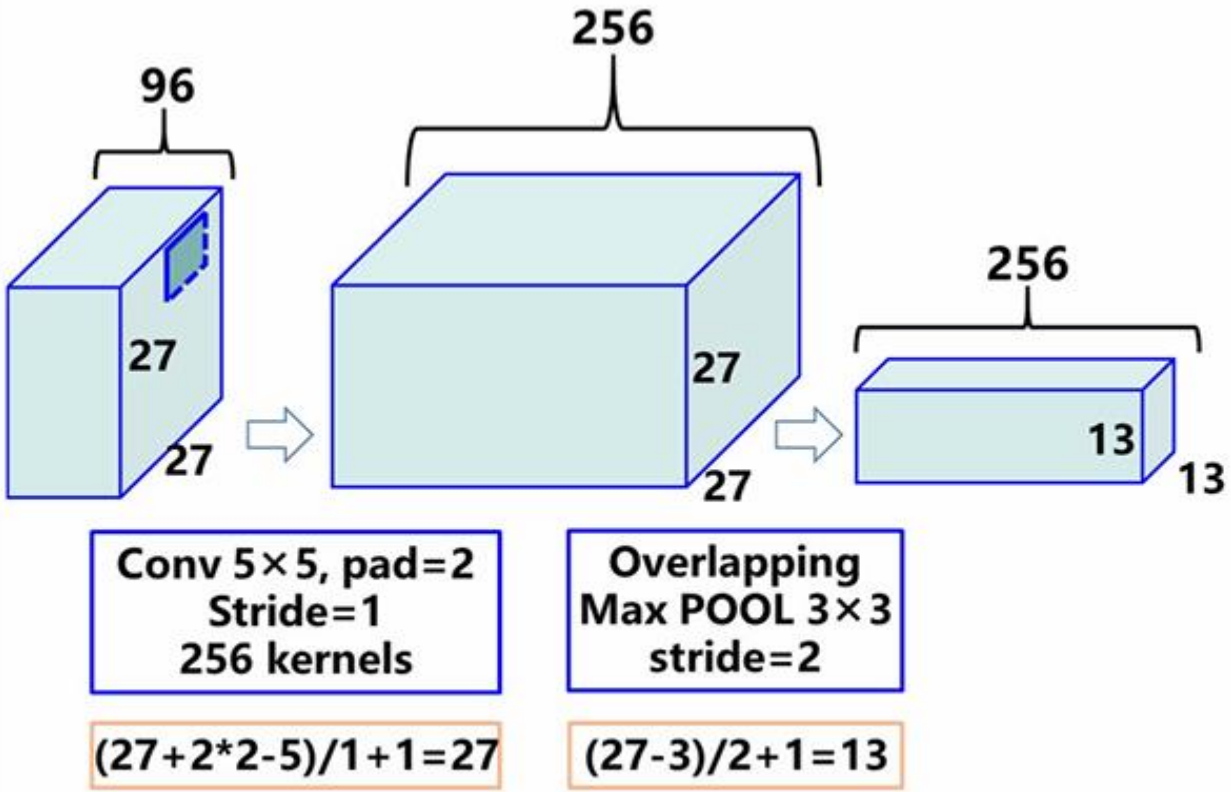
- 卷积层Conv1:
  - 卷积：使用大小为11 11 3的 卷积核进行特征提取，即卷积核深度与通道的数量相同（都为3）
  - Relu+LRN处理。卷积处理后，使用 Relu激活函数并经过局部响应归一化处理。
  - 降采样处理。AlexNet使用重叠的最大池进行降采样。



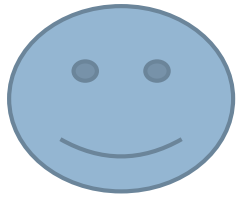
AlexNet



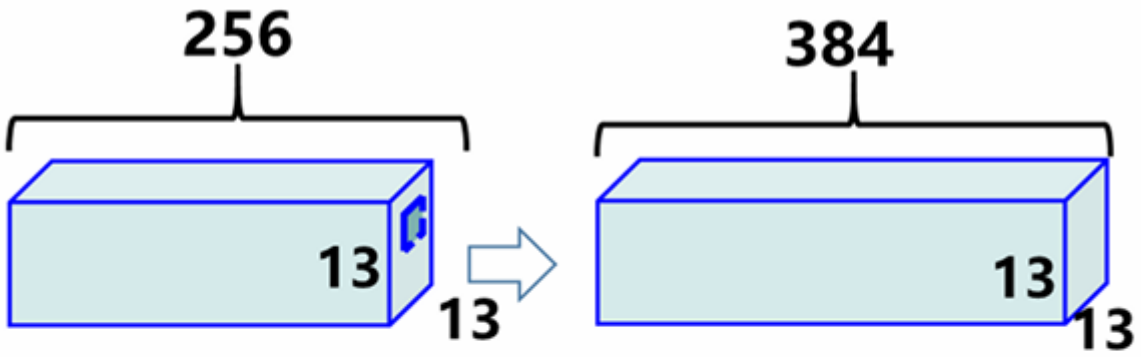
卷积层Conv2：



AlexNet



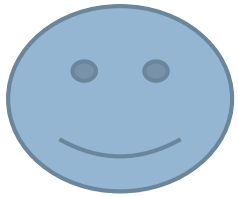
卷积层Conv3没有进行降采样处理：



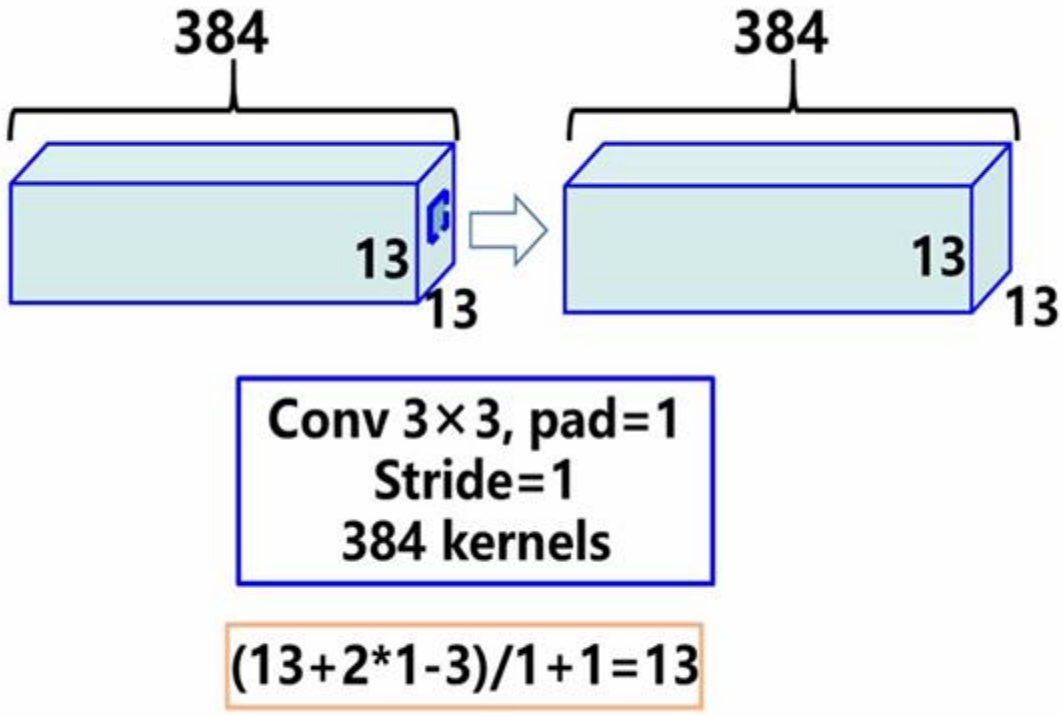
Conv 3×3, pad=1
Stride=1
384 kernels

(13+2*1-3)/1+1=13

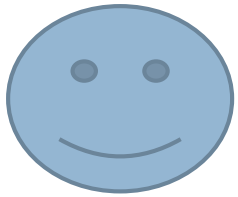
AlexNet



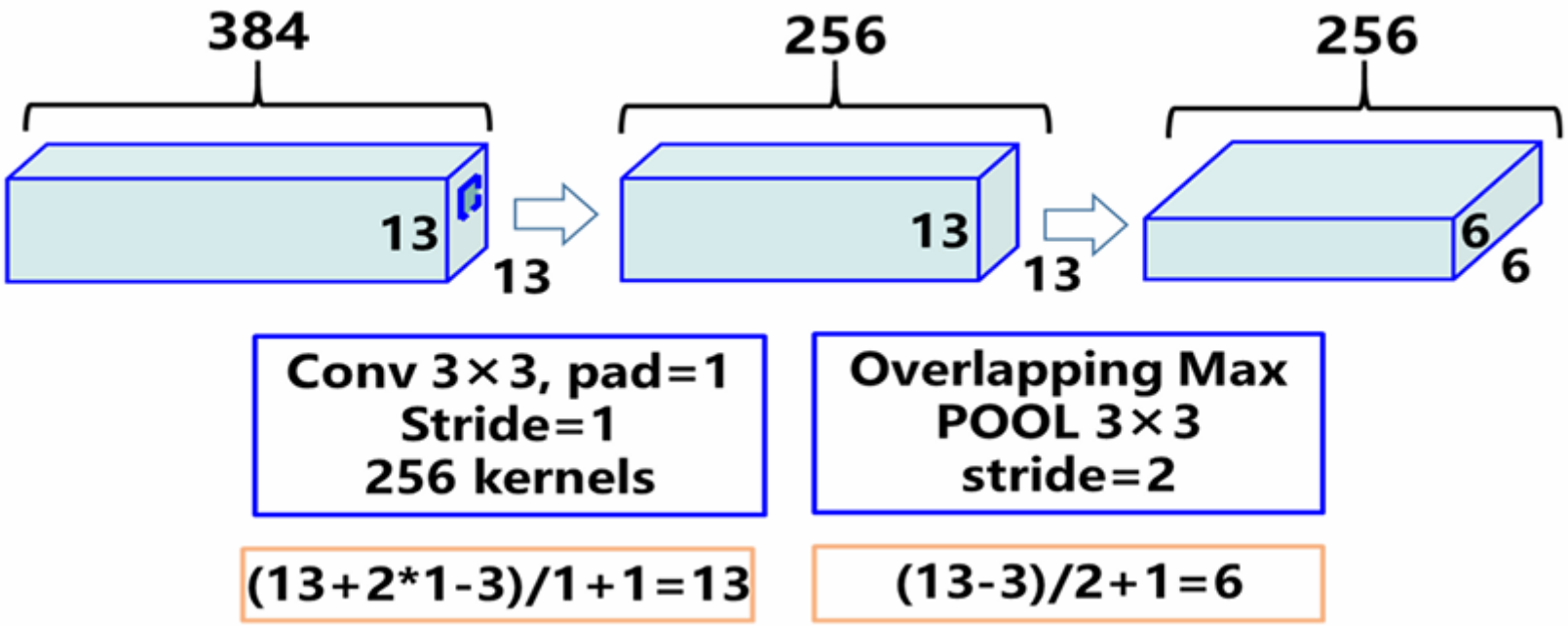
卷积层Conv4没有进行降采样处理：



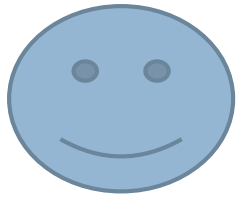
AlexNet



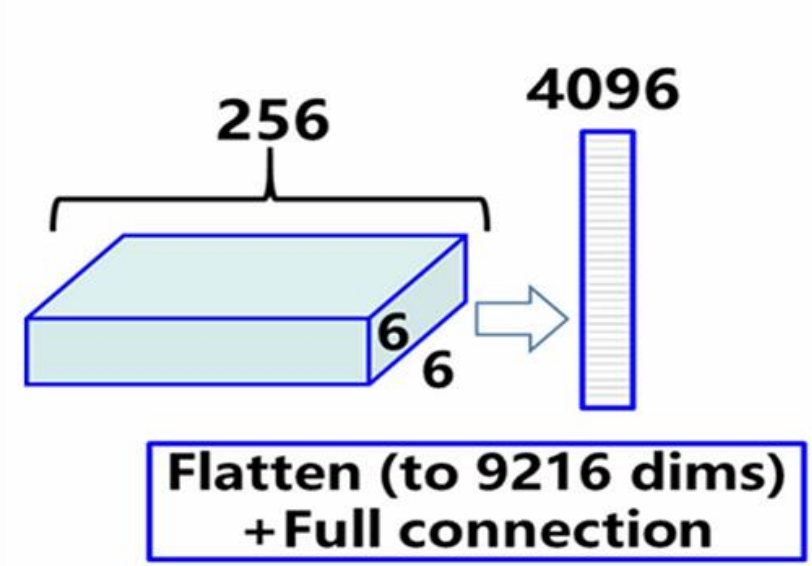
卷积层Conv5:

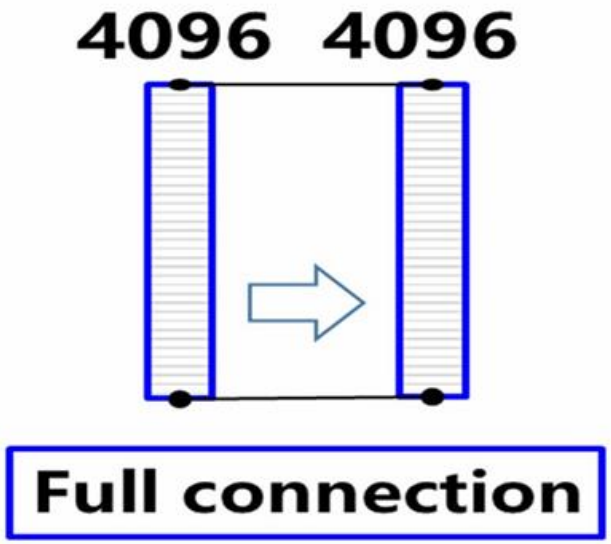


AlexNet

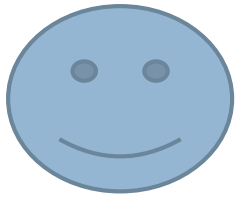


全连接层FC6，FC7：

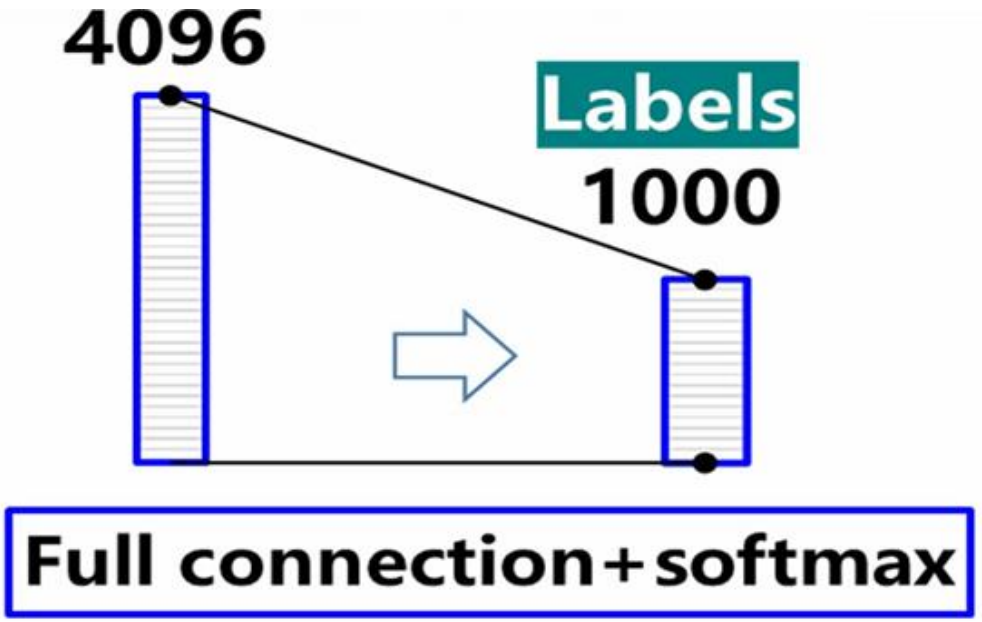




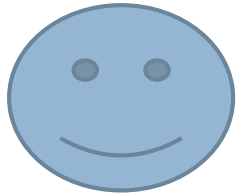
AlexNet



- 全连接层FC8：
  - 全连接处理：从4096个神经元映射为1000个神经元；
  - softmax处理：得到1000个float型数值；



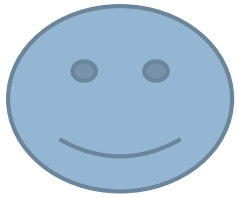
AlexNet



综上可见，AlexNet网络结构虽然在整体上与LeNet类似，但在细节上有很多不同点：

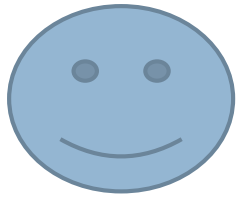
- 卷积层数较多。AlexNet有5个卷积层，LeNet有3个卷积层。
- 激活函数不同。原始LeNet-5中在池化后使用的是sigmoid激活函数， 而AlexNet则改为ReLU激活函数。回忆一下ReLU的优势；
- AlexNet对ReLU的值域进行了局部响应归一化（Local Response Normalization，简称LRN）处理。
- AlexNet使用重叠最大池处理机制。

AlexNet



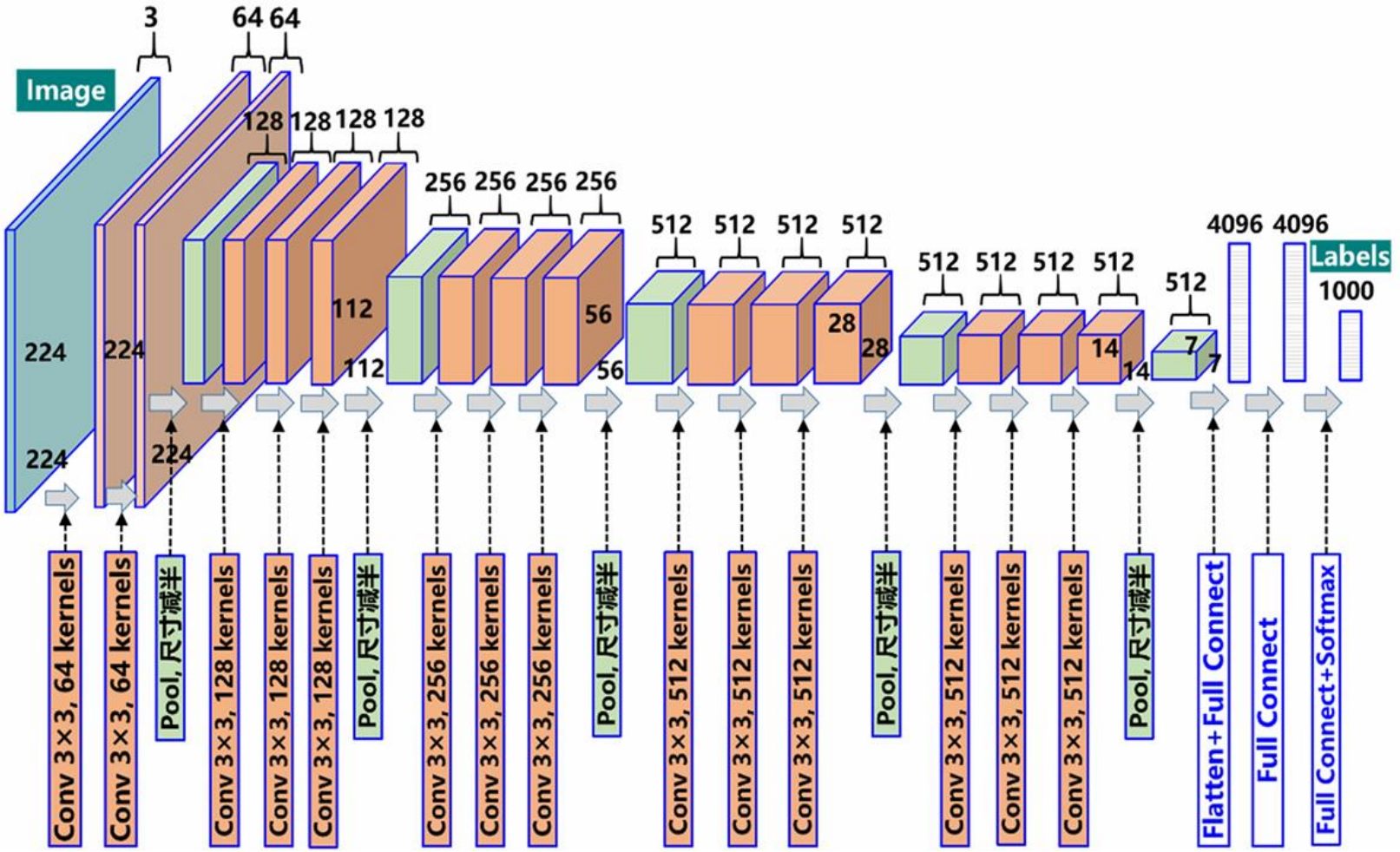
- AlexNet通过丢弃法（dropout）来控制全连接层的模型复杂度。所谓丢弃法是指模型在训练时随机忽略一部分神经元（但在测试阶段不忽略），以避免模型过拟合。
- AlexNet进行了图像增强处理，以缓解过拟合现象提升模型的泛化能力。

VGGNet

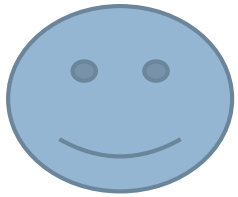


- VGGNet 是2014年由牛津大学计算机Visual Geometry Group（视觉几何组）和Google DeepMind公司的研究人员研发的深层卷积神经网络。其获得了ImageNet ILSVRC（ImageNet Large Scale Visual Recognition Challenge）比赛的第二名（第一名是GoogLeNet）。
- VGGNet共有四个版本：VGG11、VGG13、VGG16、VGG19，这些版本计算原理类似，主要区别在于网络层次深度等的配置不同。

VGGNet


Image
3
64 64
128 128 128 128
256 256 256 256
512 512 512 512
512 512 512 512
512
4096 4096
Labels
1000
224
224
224
224
112
112
56
56
28
28
14
14
7
7
Conv 3×3, 64 kernels
Conv 3×3, 64 kernels
Pool, 尺寸减半
Conv 3×3, 128 kernels
Conv 3×3, 128 kernels
Conv 3×3, 128 kernels
Pool, 尺寸减半
Conv 3×3, 256 kernels
Conv 3×3, 256 kernels
Conv 3×3, 256 kernels
Pool, 尺寸减半
Conv 3×3, 512 kernels
Conv 3×3, 512 kernels
Conv 3×3, 512 kernels
Pool, 尺寸减半
Conv 3×3, 512 kernels
Conv 3×3, 512 kernels
Conv 3×3, 512 kernels
Pool, 尺寸减半
Flatten+Full Connect
Full Connect
Full Connect+Softmax

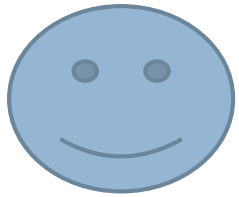
VGGNet



VGGNet可以被视为加深版本的AlexNet，也是由卷积层、全连接层两大部分构成。其具有如下特点：

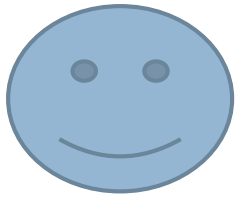
- 小卷积核和多卷积子层。AlexNet的卷积核尺寸为7x7，而VGGNet的卷积核则为小尺寸即3x3，并通过增加卷积子层数来达到同样的性能。
- 小池化核。相比AlexNet的3x3池化核，VGGNet采用2x2的池化核。
- 通道数多。VGGNet第一层的通道数为64，后面每层通道数都成倍增加并最多达到了512个通道。

GooLeNet

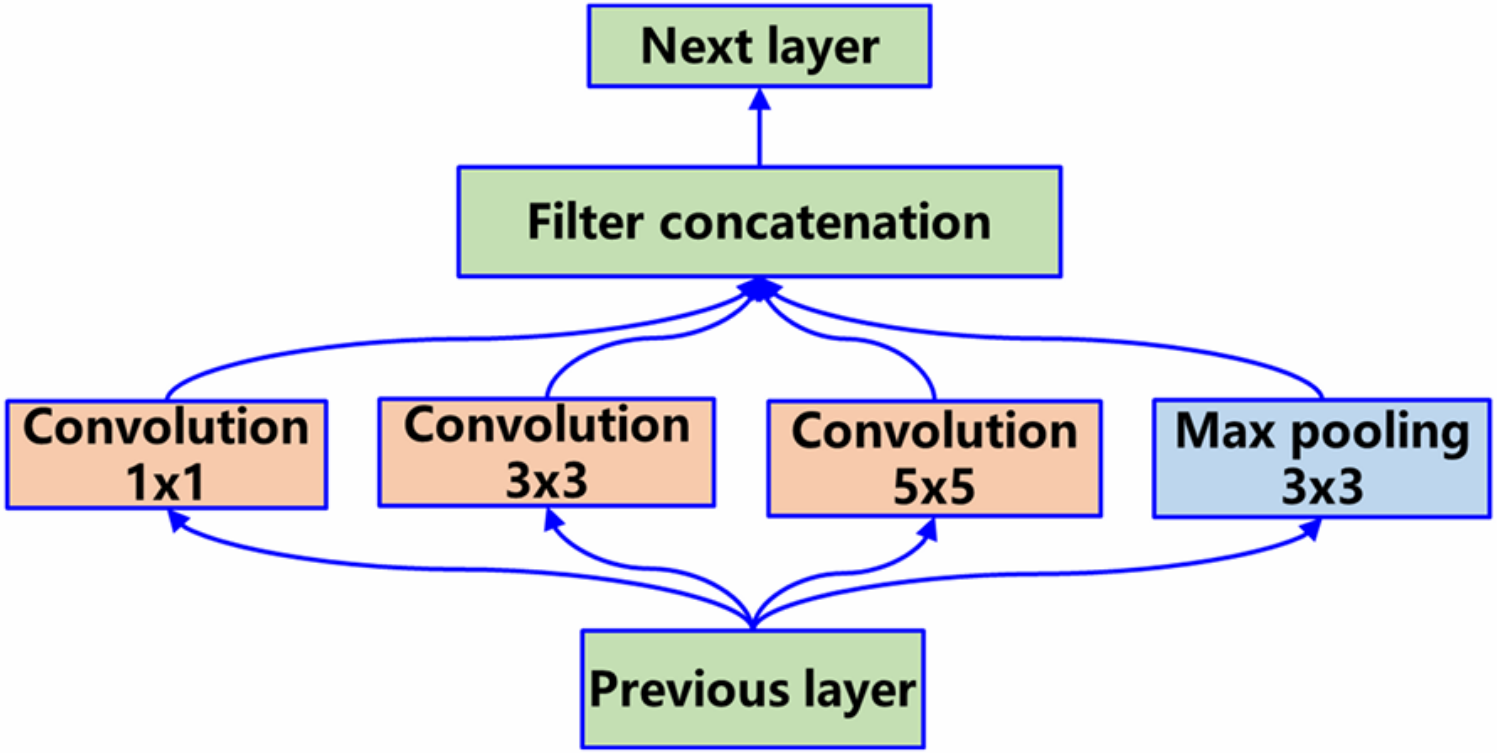


- GooLeNet之所以将L大写是为了向LeNet致敬；
- 2014年的 ImageNet 竞赛中夺得冠军；
- 也称为Inception V1，并在之后陆续改进，形成了Inception V2、Inception V3、Inception V4等版本；
- GoogLeNet中设计了一种基础增强卷积模块InceptionCNN， 并通过多次使用以搭建一个稀疏性、高性能的网络结构，从而在增加网络深度和宽度的同时减少参数。

GooLeNet

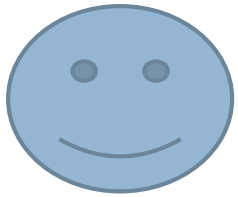


GoogLeNet中的基础卷积块InceptionCNN：原始版本

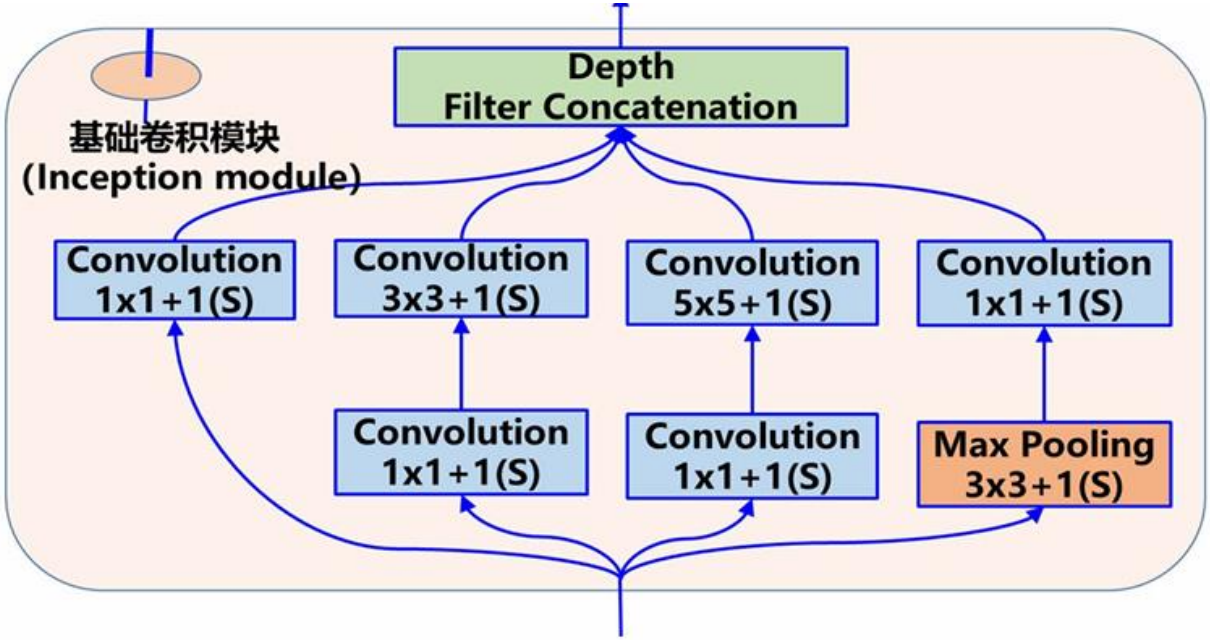


基础卷积块的原始版本

GooLeNet



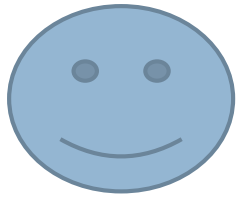
GoogLeNet中的基础卷积块InceptionCNN：更新版本



基础卷积块的更新版本

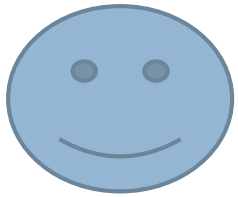
- 符号 3×3+2(S)表示:
  - 该层使用 3×3 卷积核;
  - 步幅为 2;
  - 使用SAME 填充;
  - 所有卷积层都使用 ReLU 激活函数
- SAME 填充的目的是使输出特征图和输入特征图具有相同的高度和宽度，从而便于将各个路径的输出在最后的深度连接层（depth concat layer）上的拼接。

ResNet

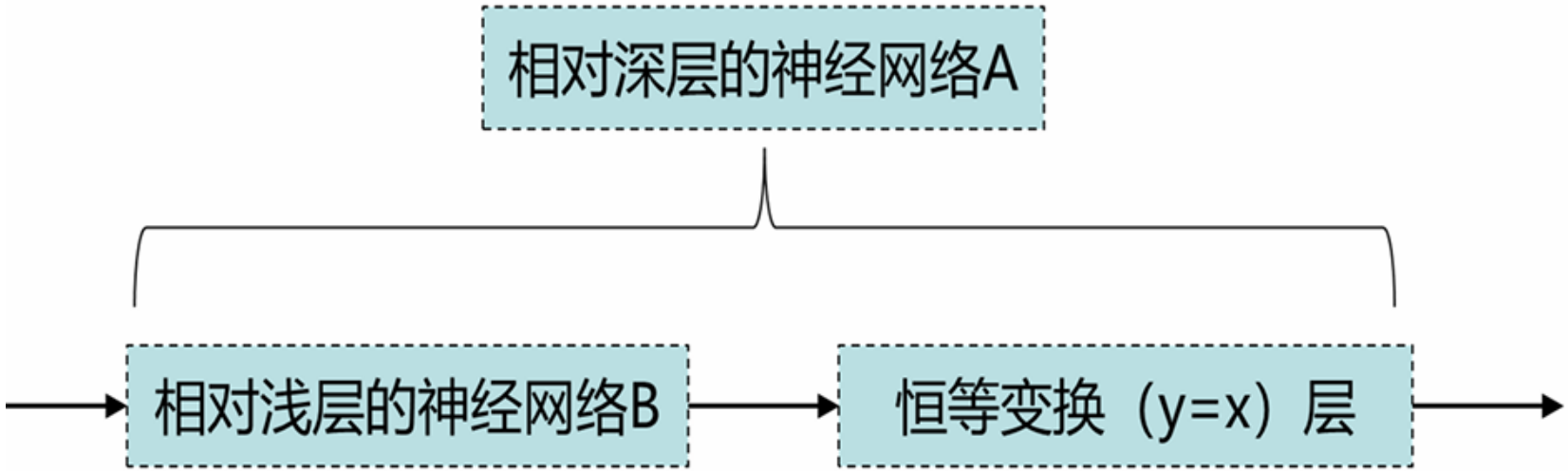


- 残差网络（ResNet）由微软的4位学者提出，其主要贡献是发现了神经网络层数过多带来的副作用即退化现象（Degradation），并针对性地提出了一种跳跃连接或称快捷连接（Shortcut connection）的解决思路。因此可以通过增加网络深度的办法来提高模型的预测准确率。
- 因此残差网络的深度首次突破了100层，却仍然在ImageNet和CIFAR-10等图像分类任务上取得了非常好的结果。残差网络在2015年 ImageNet ILSVRC图像分类和物体识别比赛中获得了冠军。

ResNet

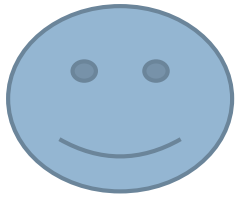


从理论上来说，浅层神经网络所拟合的分布是深层的神经网络的子集，因此增加层数至少应该不会导致性能下降；



如图所示，如果增加的层数均为恒等变换，此时网络与原先等价。

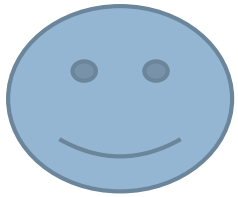
ResNet



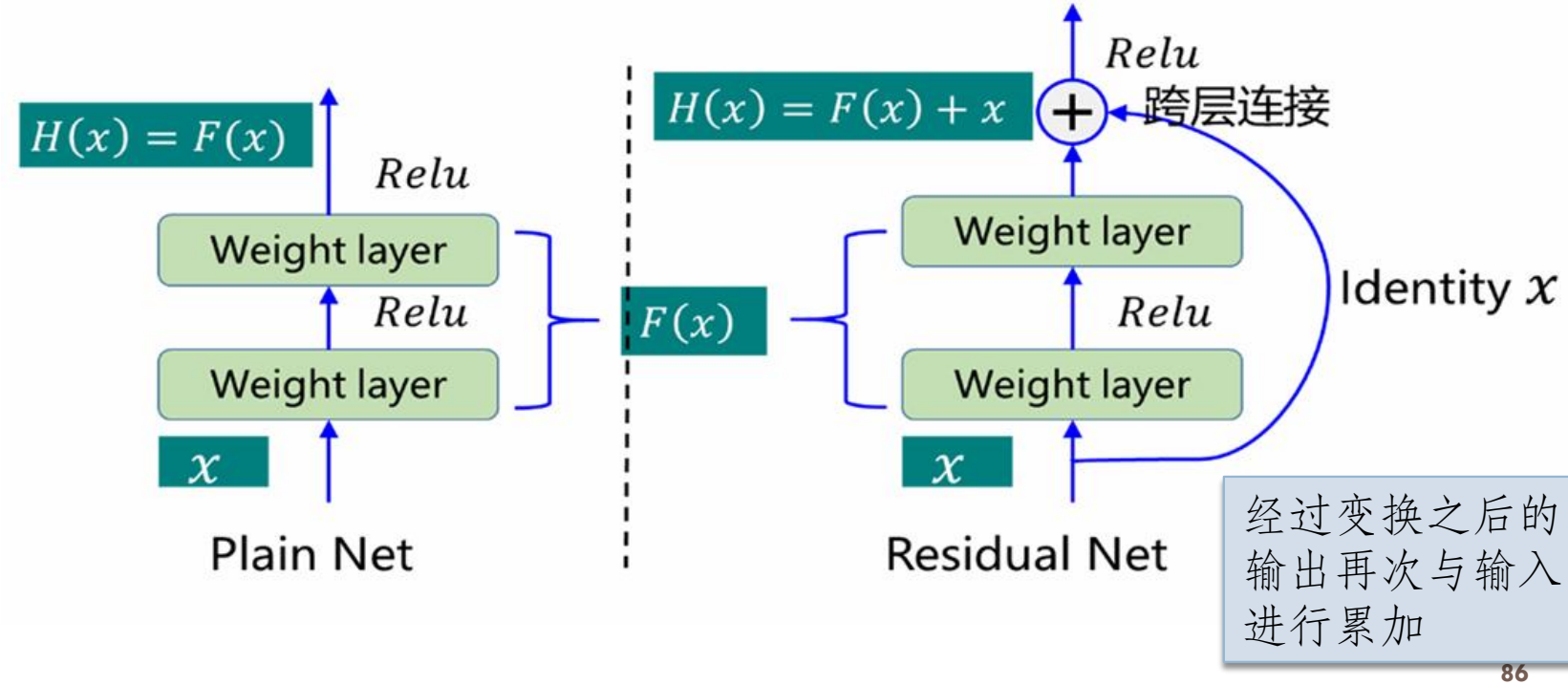
在实践中网络层加深可能会带来两大问题：

- 梯度消失与梯度爆炸；当存在过多的层时，多次连乘可能会出现梯度越来越小或者逐步放大， 并分别导致梯度消失或梯度爆炸。对于该问题的解决方法包括正则化层（Batch Normalization）处理等。
- 退化问题；即网络的加深反而导致预测能力降低，与人们的印象相背离。这一现象被称之为网络退化。

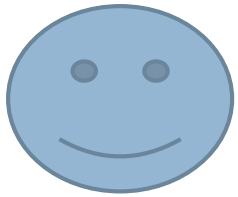
ResNet



- 退化的原因：现有的多层非线性神经网络结构难以拟合恒等映射函数（即y=x）。
- 解决方法：快捷连接分支



卷积神经网络与NLP

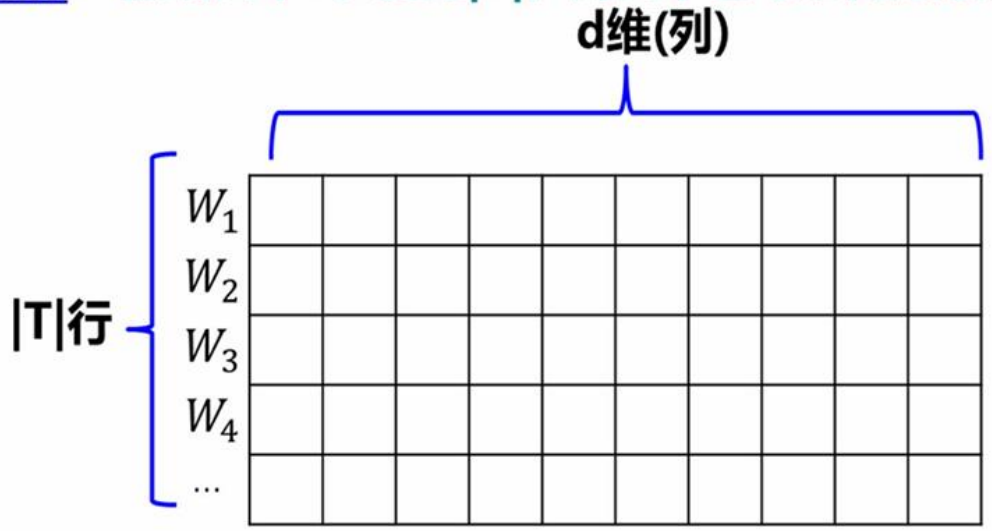


尽管卷积神经网络模型的出发点是解决CV任务，但是在NLP领域，其同样拥有广泛的应用。

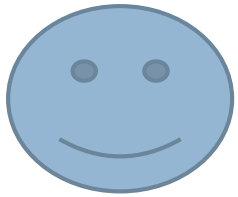
问题： 对于文本T=(W_1，W_2,W_3,W_4,…)。T可以为短语、句子或篇章等。

如何借鉴图像卷积神经网络的思想用于文本分类研究?

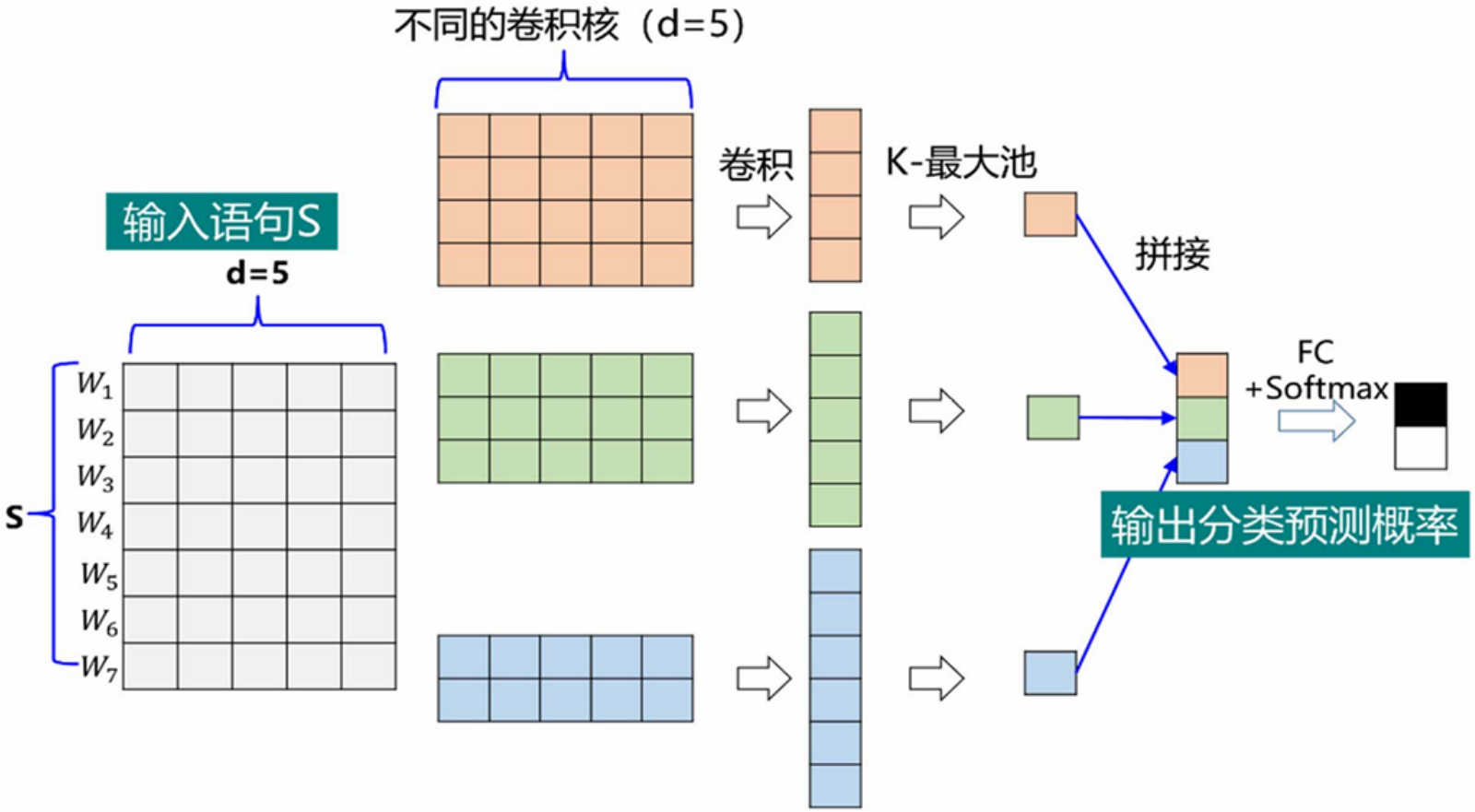
思路： 假设文本T长度为|T|，则文本也可以转化为矩阵形式：



卷积神经网络与NLP



利用卷积神经网络对文本进行分类：



主要内容

- 神经网络基础
- 前馈神经网络
- 卷积神经网络
- 循环神经网络
- Transformer

序列数据

文本、视频以及很多数据都呈现时间序列分布

I never wanted this. I never wished to watch people sell their blood for bread. We once believed ballots could purge the greed of the medical syndicates, but you betrayed me — betrayed every mother counting blood-donation tokens outside the clinics, their arms drained hollow to keep their children fed and sheltered. Let the bills become the pamphlets of uprising! Let student-loan contracts the steps of the Education Department shrouds of penitence. Let ER invoices d the halls of insurance hearings. Though credit score may be branded with the z of the slums, I will see universal healthc drug prices to the cross! America has c chance to rise again. If you will not seiz then I will. So let it be revolution! From t Coast to the West Coast, from city to p let fury boil and injustice crumble. Thou takes the last drop of my blood, I will se nation reborn in liberation. And if I cann it from your failures... then let America



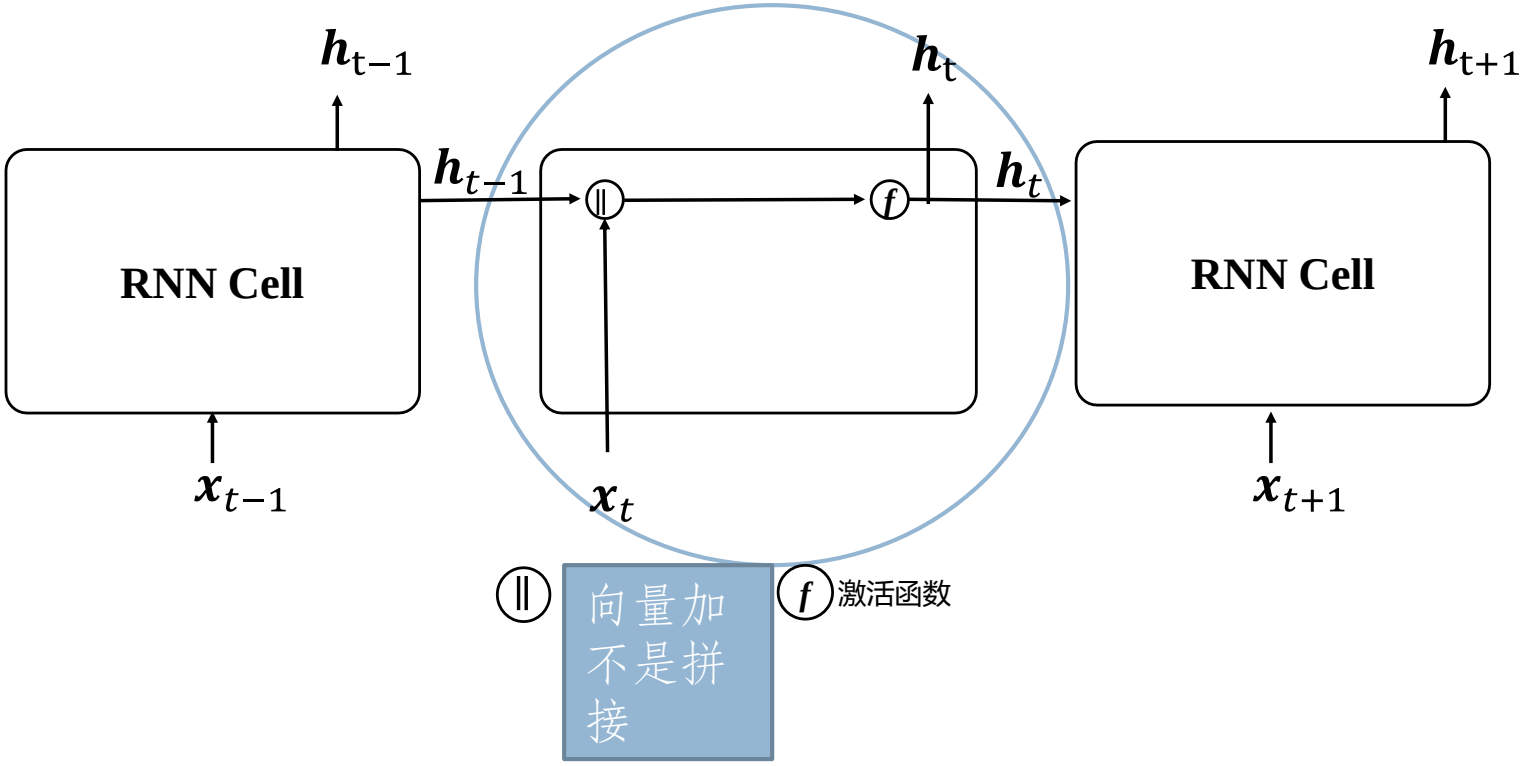


循环神经网络

- 时间序列数据：在不同时间上收集到的数据，用于所描述现象随时间变化的情况。
- 循环神经网络（Recurrent Neural Network, RNN）是一类专门用于处理序列数据的神经网络模型。RNN通过引入隐藏状态，在处理序列时记住过去的信息，从而捕捉数据的时间依赖性和上下文关系。
- 本节主要介绍循环神经网络、长短时记忆网络（Long Short-Term Memory, LSTM）和门控循环单元（Gated Recurrent Unit, GRU）。

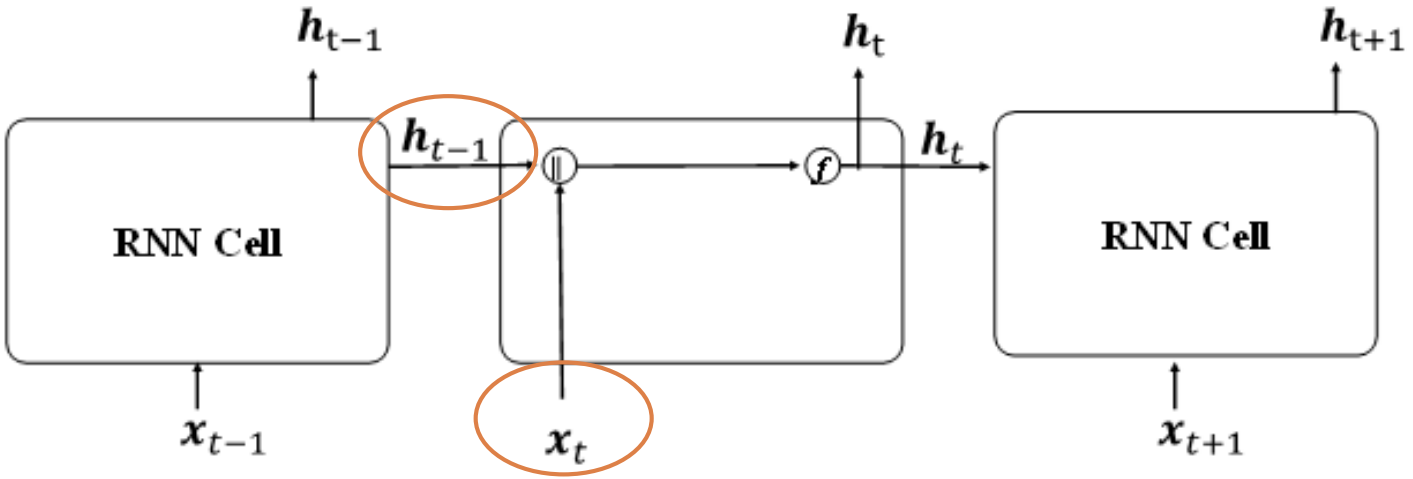
循环神经网络

RNN通过引入循环处理单元，能够处理动态的序列数据。在RNN中，我们通常将处理单元称为RNN单元（RNN cell）。



循环神经网络

- 每个时刻t都对应一个RNN单元；
- 每个单元有两个重要的输入：当前时刻的输入向量$\boldsymbol{x}_t$和上一时刻的隐藏状态$\boldsymbol{h}_{t-1}$；
- RNN单元通过使用这两个输入来生成当前时刻的输出h_t，并将其传递给下一个时刻的RNN 单元。



循环神经网络

- 向量$\boldsymbol{x}_t \in \mathbb{R}^M$表示$t$时刻的输入向量
- $\mathrm{h}_{t-1} \in \mathbb{R}^D$为$t-1$时刻的隐藏状态
- 当前时刻的隐藏状态$\boldsymbol{h}_t$可以简单地表示为：

$$\boldsymbol{h}_t = \boldsymbol{f}(\boldsymbol{W}[\boldsymbol{x}_t, \boldsymbol{h}_{t-1}] + \boldsymbol{b})$$

其中，$\boldsymbol{W}$为输入向量权重矩阵，b为偏置，f为激活函数，在RNN中f通常采用tanh 函数。

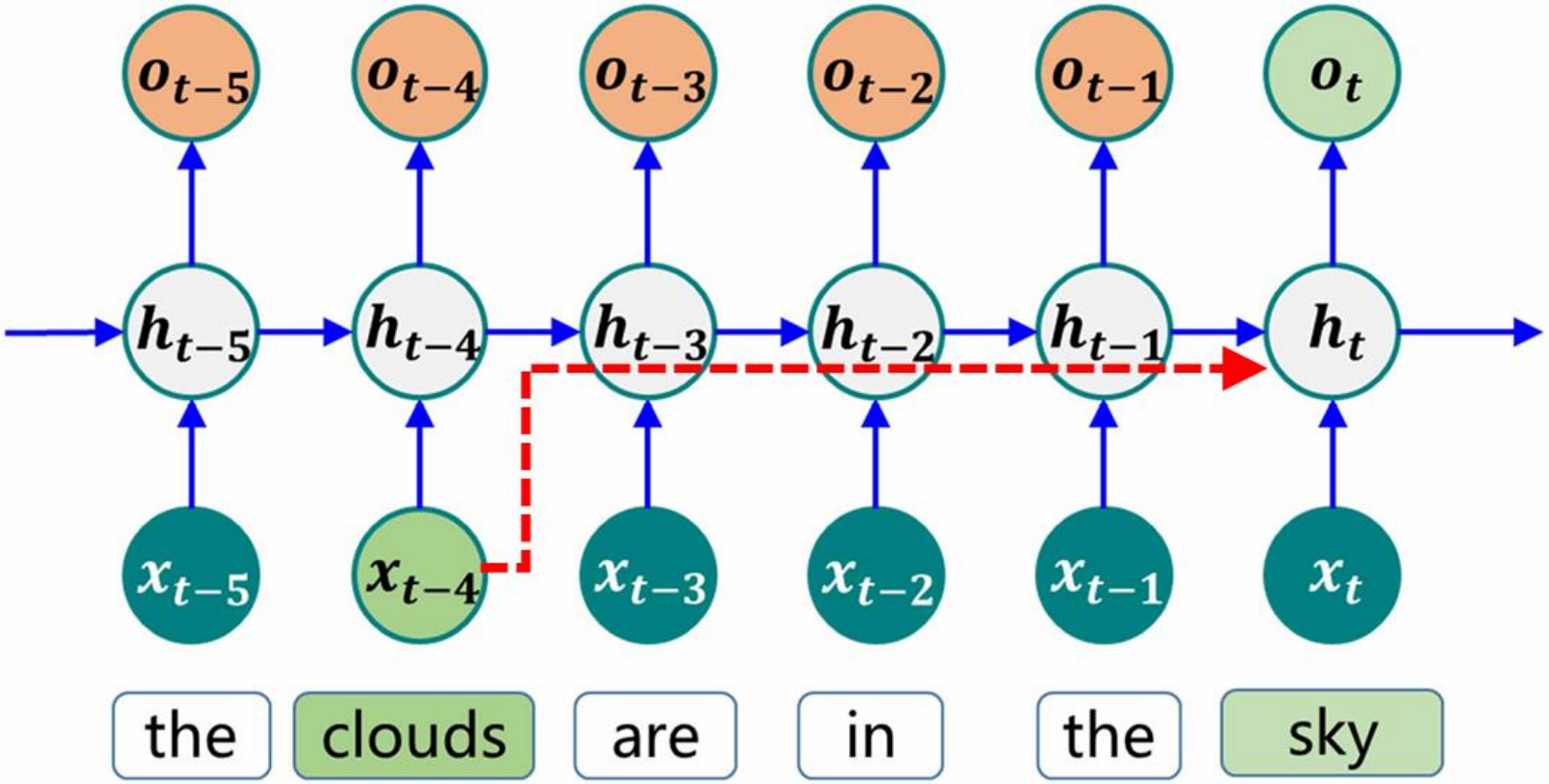
循环神经网络

- 参数共享：每一个RNN单元的参数都是相同的，共享参数表明循环神经网络中的每一步都在做相同的事，只是输入不同， 因此大大地降低了网络中需要学习的参数，从而有利于改善训练效率。
- RNN的反向传播：为什么RNN的反向传播与常规的前馈神经网络不同？

解决方法：BPTT（Backpropagation Through Time，通过时间的反向传播）

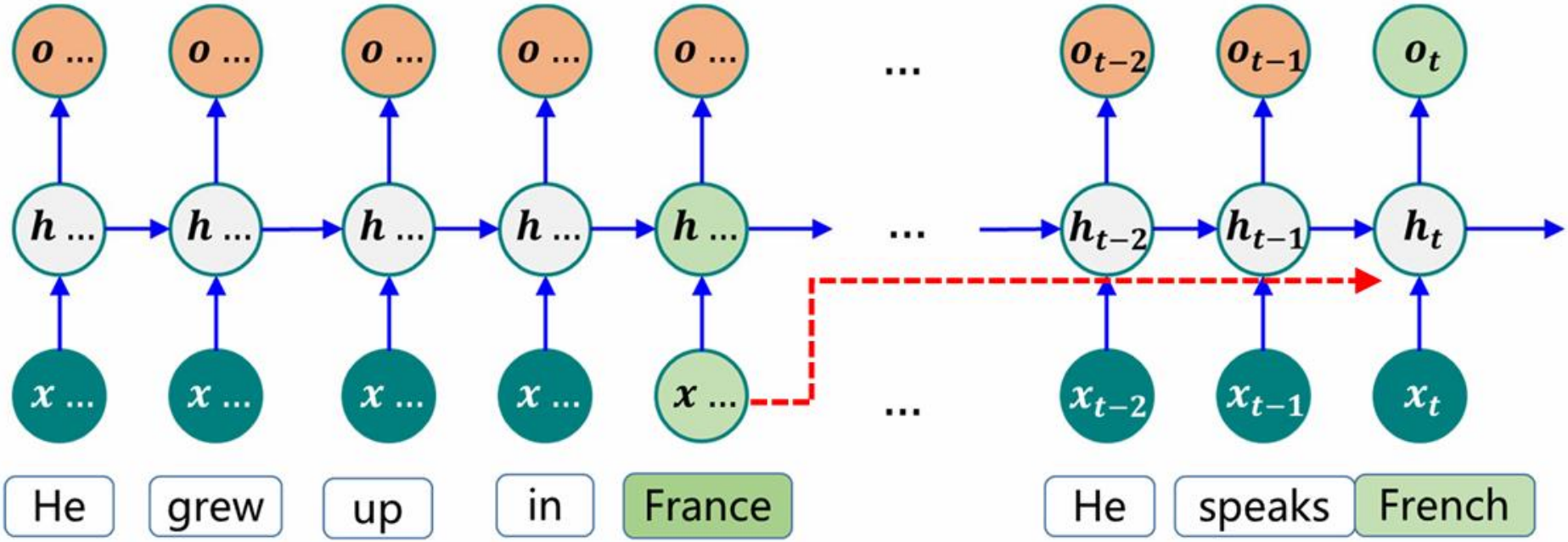
循环神经网络

标准RNN可以处理一定距离内的依赖（短期依赖）：



循环神经网络

标准RNN无法处理更长的上下文间隔（长期依赖）



循环神经网络

为什么RNN无法处理长依赖问题？

- **根本原因**：BPTT导致的梯度消失；
  - 损失函数的梯度需要从网络的输出端**反向传播**到网络的输入端。
  - 这个反向传播的过程需要**沿着时间步一步步进行**，并且涉及连续的**矩阵乘法和激活函数求导**。
  - 最终结果就是：从序列末端计算出的梯度，在传播到序列开端时，已经变得微乎其微，几乎为零。

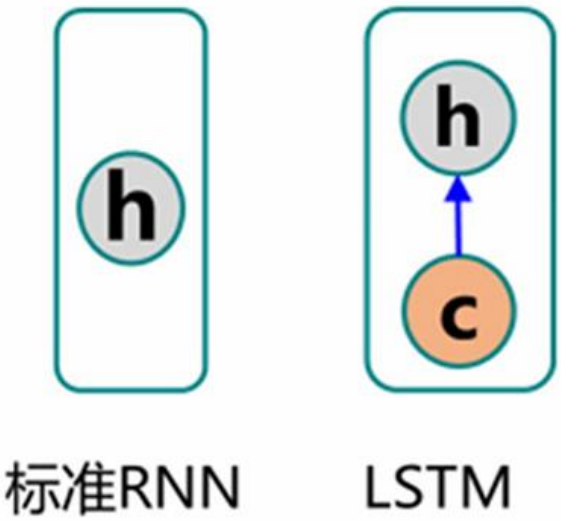
LSTM

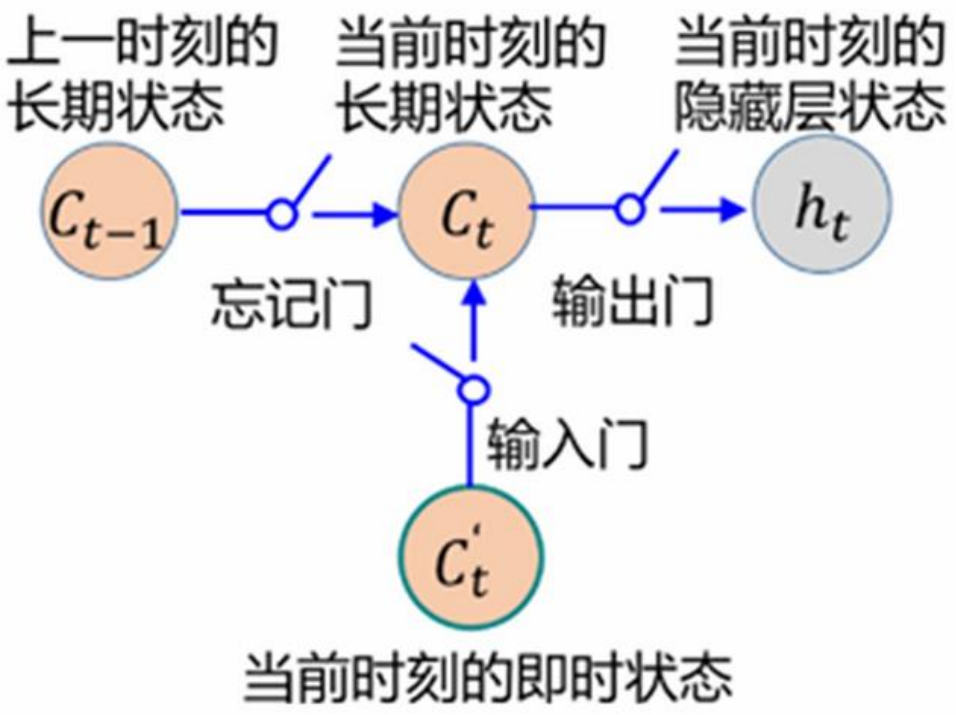
长短期记忆网络（Long Short-Term Memory, LSTM）是一种特殊的循环神经网络（RNN），由 Sepp Hochreiter 和 Jürgen Schmidhuber 在1997年提出。它被明确设计来解决标准RNN的**梯度消失/爆炸问题**，从而能够有效地学习**长期依赖关系**。

LSTM



LSTM在隐藏层上额外增加了一个长期状态c（cell state）：

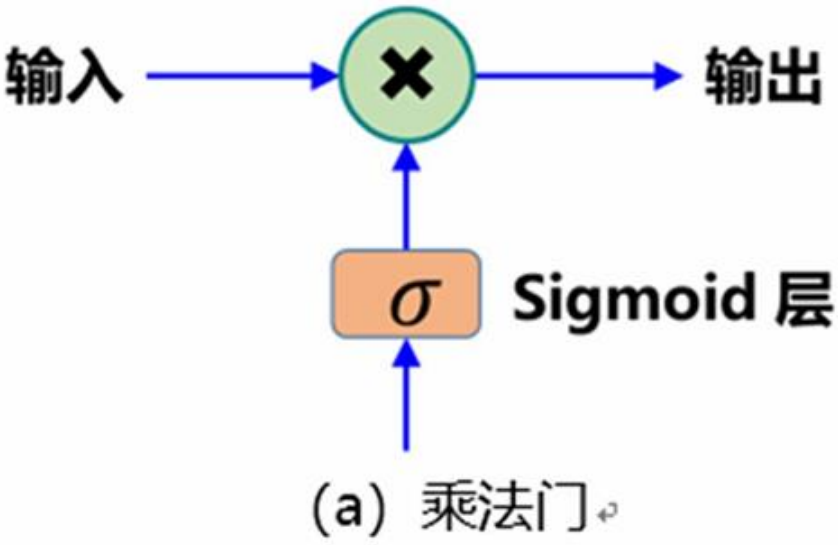




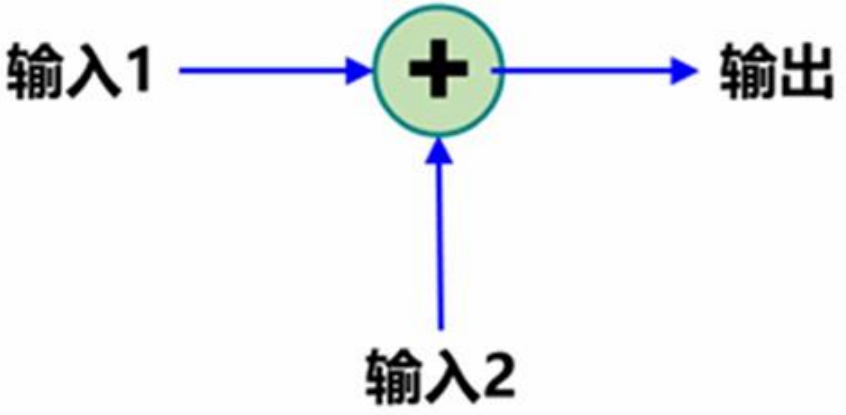
LSTM



神经网络中的门：



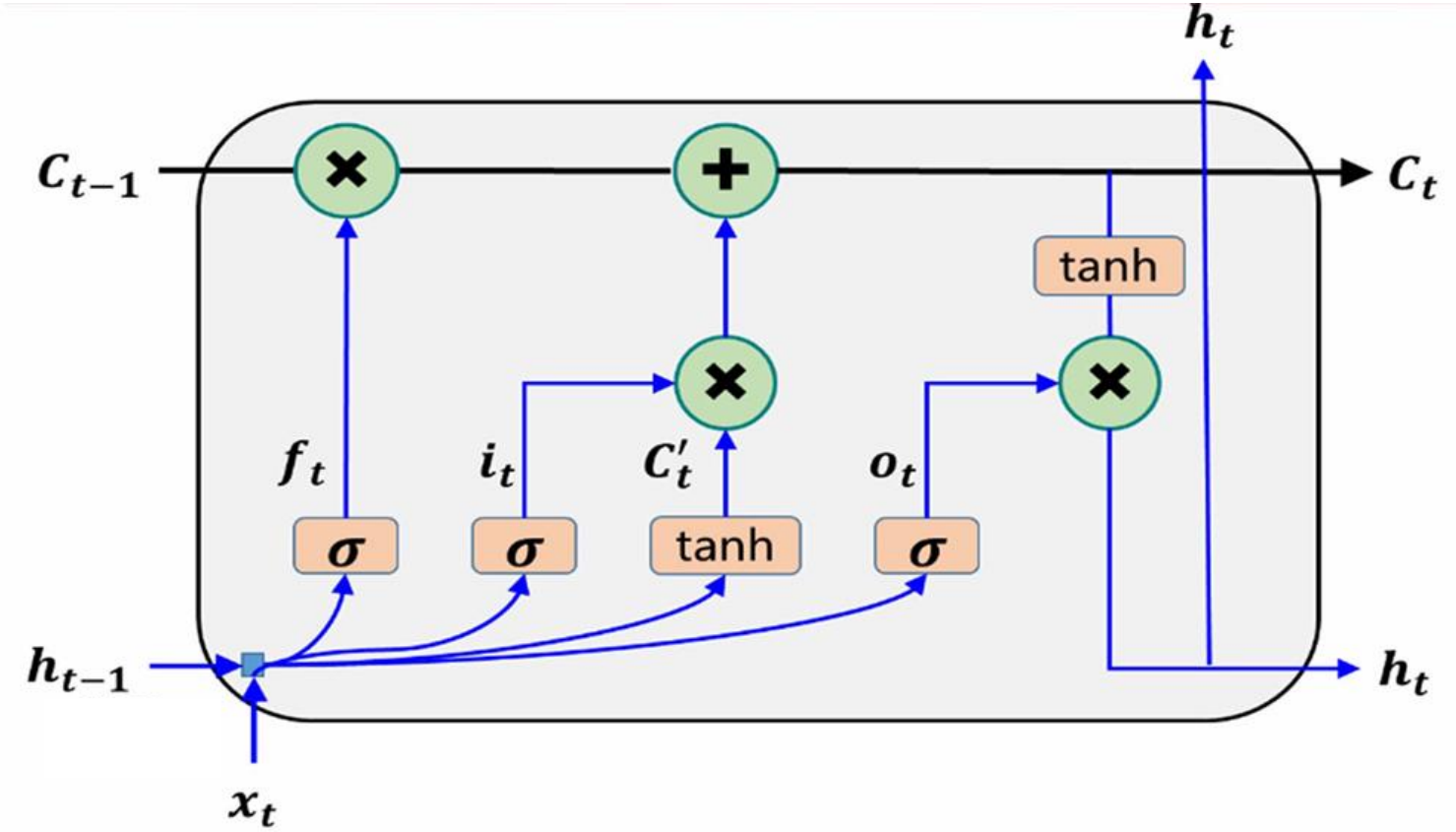
(a) 乘法门



(b) 加法门

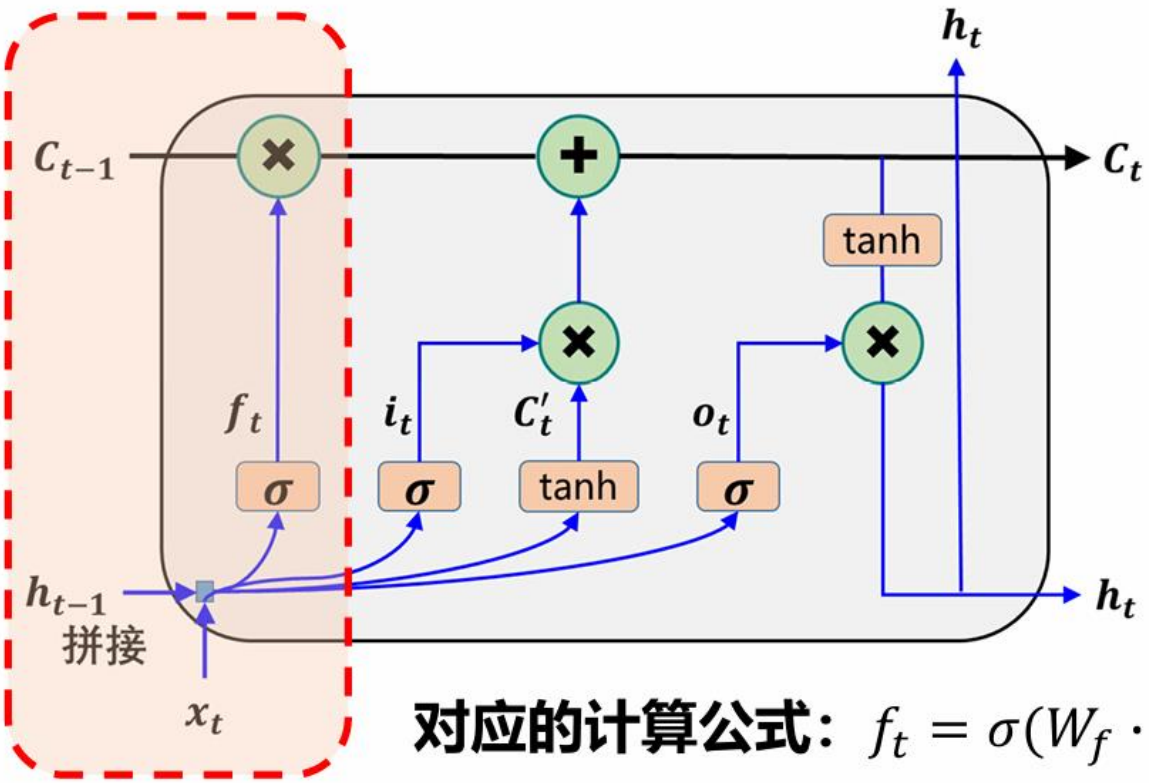
LSTM

LSTM模型的整体框架及计算过程：



LSTM

遗忘门f_t：决定遗忘或丢弃多少上一时刻的单元状态向量c_{t-1}的信息。

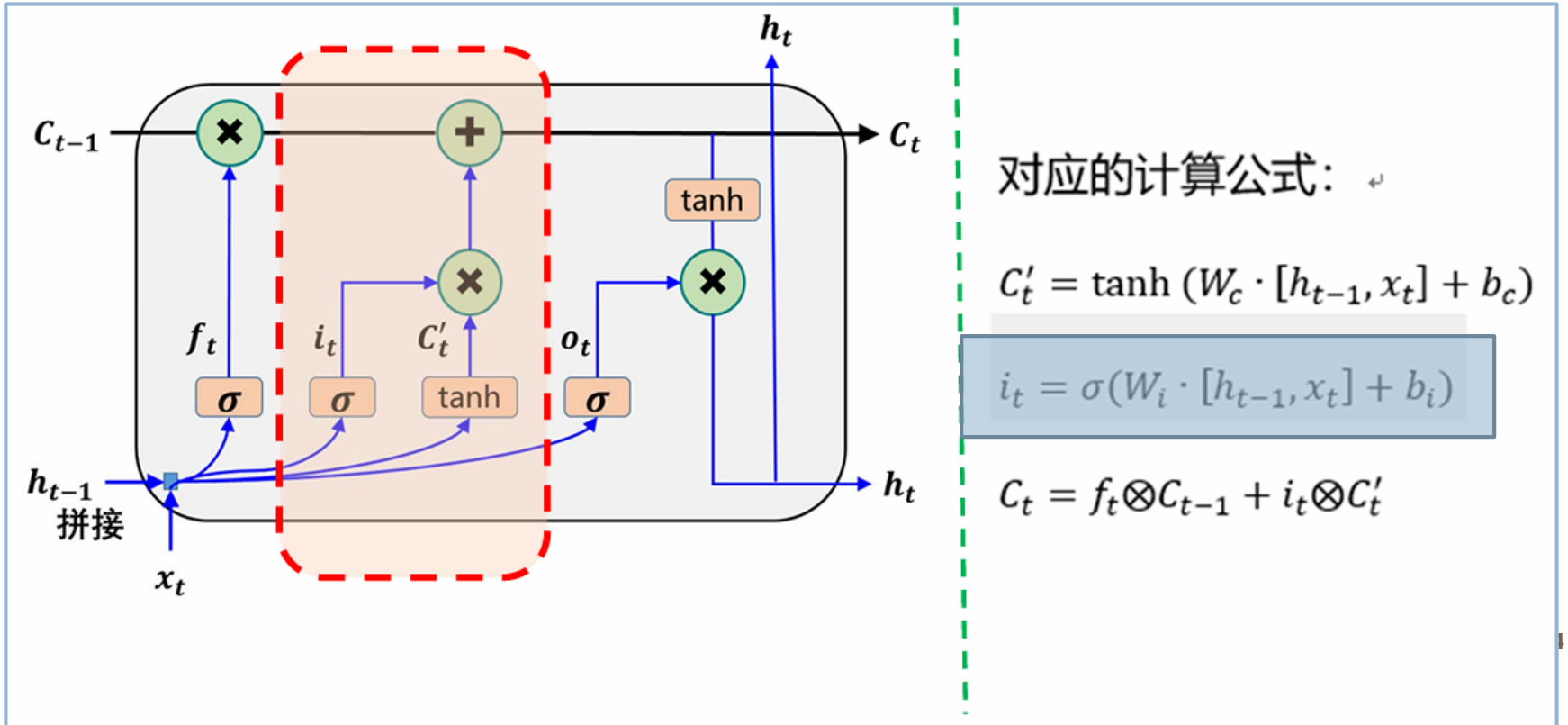


对应的计算公式： $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

LSTM

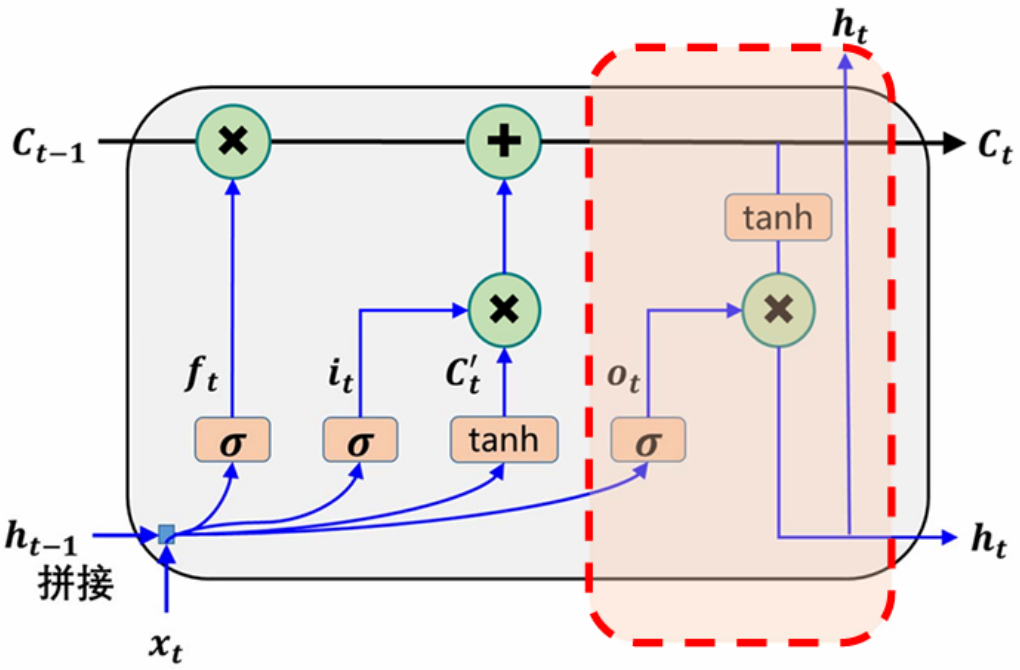
输入门：决定输入向量的保留程度

长期记忆C_t：使用遗忘门和输入门的输出进行更新



LSTM

输出门O_t：当前时刻输出哪些信息



对应的计算公式：

$$o_t = \sigma(W_O \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \otimes \tanh(C_t)$$

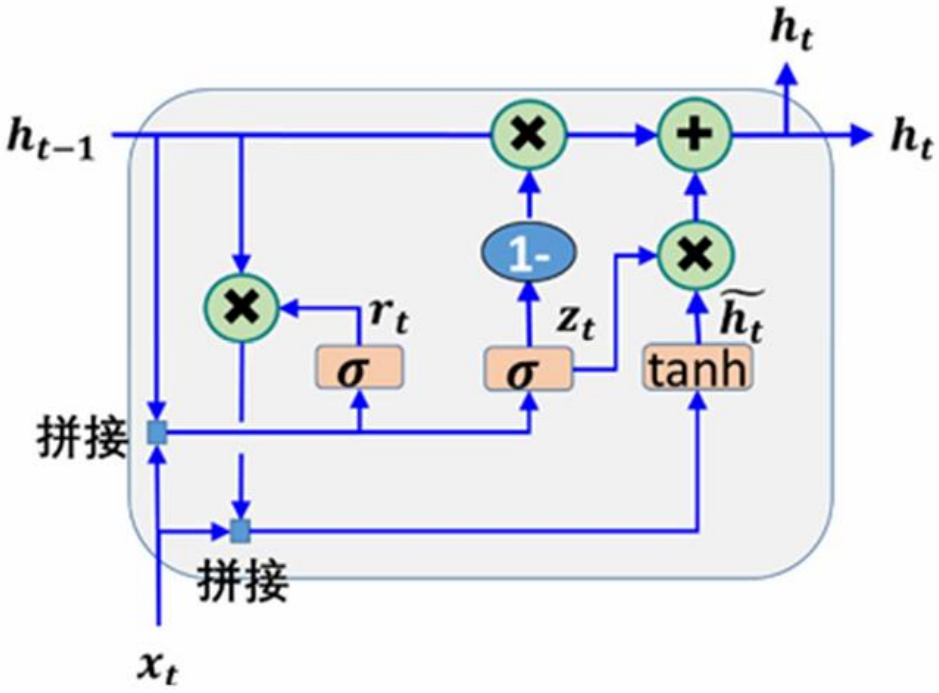
GRU

GRU (Gated Recurrent Unit) 是LSTM的一个流行变体，由 Kyunghyun Cho 等人于2014年提出。

- 和 LSTM 引入三个门不同，在GRU模型的结构中只包含两个门：重置门和更新门。
- 在LSTM网络中，输入门和遗忘门是互补关系，具有一定的冗余性；
- GRU网络不引入额外的遗忘门单元，直接使用更新门来控制输入和遗忘之间的平衡。

GRU

如图所示，GRU单元只通过隐藏状态进行信息的传递，未额外引入LSTM中的单元状态向量。



GRU

- GRU内部包含以下组件：
  - 重置门(reset gate) $\boldsymbol{r}_t$：控制当前时刻的候选状态需要保留多少来自上一时刻的隐状态（历史状态）：

$$\boldsymbol{r}_t = \text{Sigmoid}(\boldsymbol{W}_r \boldsymbol{x}_t + \boldsymbol{U}_r \boldsymbol{h}_{t-1} + \boldsymbol{b}_r)$$

  - 更新门(update gate) $\boldsymbol{z}_t$：控制当前时刻的隐状态信息需要保留多少历史信息，以及需要从候选状态中接受多少新信息：

$$\boldsymbol{z}_t = \text{Sigmoid}(\boldsymbol{W}_z \boldsymbol{x}_t + \boldsymbol{U}_z \boldsymbol{h}_{t-1} + \boldsymbol{b}_z)$$

GRU

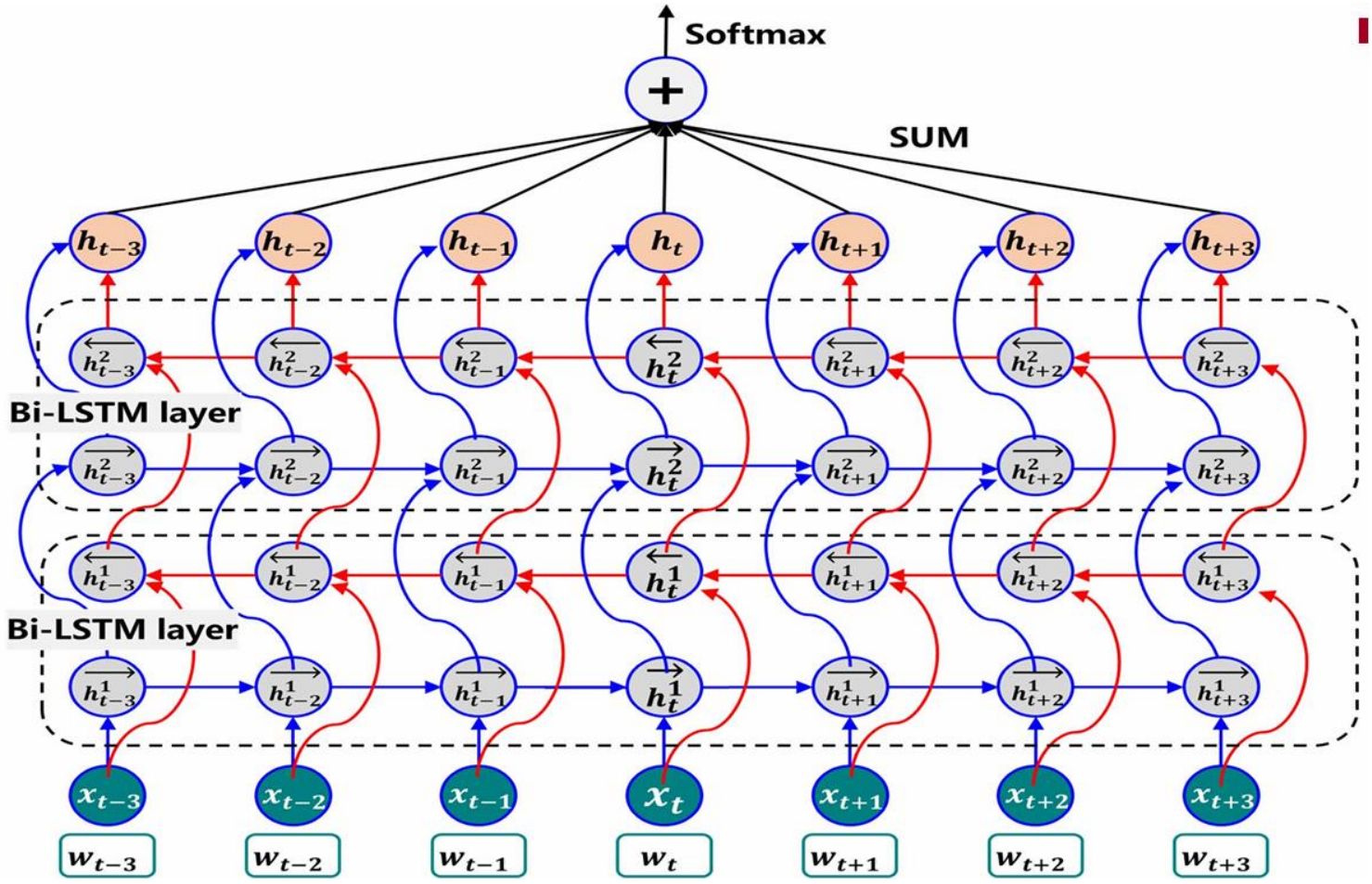
- 隐状态向量$\boldsymbol{h}_t$是GRU单元的输出，也是当前时刻的记忆状态向量，可以传递给下一时刻的GRU单元。
  - 隐状态向量的更新由上一时刻的隐状态$\boldsymbol{h}_{t-1}$和当前时刻的候选隐状态$\tilde{\boldsymbol{h}}_t$加权平均得到，加权系数由更新门$\boldsymbol{z}_t$决定：

$$\boldsymbol{h}_t = (1 - \boldsymbol{z}_t) \cdot \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \cdot \tilde{\boldsymbol{h}}_t$$

  - 候选隐状态$\tilde{\boldsymbol{h}}_t$ 是当前时刻的一个潜在输出，它由当前时刻输入向量$\boldsymbol{x}_t$与通过重置门$\boldsymbol{r}_t$的上一时刻隐状态$\boldsymbol{h}_{t-1}$组合产生：

$$\tilde{\boldsymbol{h}}_t = f(\boldsymbol{W}_h \boldsymbol{x}_t + \boldsymbol{U}_h(\boldsymbol{r}_t \cdot \boldsymbol{h}_{t-1}) + \boldsymbol{b}_h)$$

双向循环神经网络



主要内容

- 神经网络基础
- 前馈神经网络
- 循环神经网络与卷积神经网络
- Transformer

长依赖问题

LSTM和GRU相比普通RNN是巨大的飞跃，它们极大地缓解了长依赖问题，将RNN处理长依赖的能力从大约10个时间步提升到了数百个时间步。

但是，当序列变得极长时，它们依然会遇到困难。

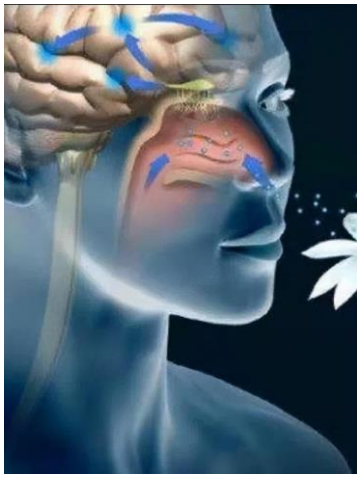


询问一本小说中的特定情节？

注意力机制







来自视觉、听觉、嗅觉的大量信息

注意力

少量有用的部分

大脑决策与行动

注意力机制

- 受生物注意力机制启发，神经网络的注意力机制（attention mechanism）模仿人类在处理信息时的选择性关注。
- 通过这种机制，神经网络能够在每个时间点动态地决定应重点关注输入的哪些部分，尤其是在处理序列数据时，能够自动聚焦于与当前任务最相关的部分。

注意力机制

任务定义：

输入—— N 个输入向量 $[\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N]$ 输出——与特定任务最相关的信息

为了实现这一目标，通常定义三个向量：

- 查询向量（Query Vector）$\boldsymbol{q}$ ：表示当前关注的和任务相关的输入信息。
- 键向量（Key Vector）$\boldsymbol{k}$：表示所有输入信息的特征，在基本注意力机制中，一般直接采用输入向量$\boldsymbol{x}$表示。
- 值向量（Value Vector）$\boldsymbol{v}$：表示与键值一一对应的实际数据，为了简化计算，在基本注意力机制中，也直接采用输入向量x表示。

注意力机制

注意力机制的数学模型由三部分构成：相关性得分、注意力权重和上下文向量。

（1）相关性打分

相关性打分通过注意力打分函数计算查询向量$\boldsymbol{q}$与每个输入向量（键向量）之间的相关性得分，以衡量每个输入在任务中的重要性，用$s(\boldsymbol{q}, \boldsymbol{x}_n)$表示。

常见的注意力打分函数包括点积、缩放点积、加性注意力。

注意力机制

(1) 相关性打分（续）

(a) 点积 (Dot Product) 模型：

$$s(\boldsymbol{q}, \boldsymbol{x}_n) = \boldsymbol{q} \cdot \boldsymbol{x}_n$$

(b) 缩放点积 (Scaled Dot Product)模型：

$$s(\boldsymbol{q}, \boldsymbol{x}_n) = \frac{\boldsymbol{q} \cdot \boldsymbol{x}_n}{\sqrt{d}}$$

(c) 加性注意力 (Additive Attention)：

$$s(\boldsymbol{q}, \boldsymbol{x}_n) = \boldsymbol{v}^T \tanh\left(\boldsymbol{W}_q \boldsymbol{q} + \boldsymbol{W}_x \boldsymbol{x}_n\right)$$

注意力机制

（2）注意力权重

注意力权重表示查询向量对不同键向量的关注程度。

具体地，在给定查询向量 $\boldsymbol{q}$ 和输入向量 $\boldsymbol{X}$ 下, 选择第 n 个输入向量的概率α_n，即为第n个向量的注意力权重：

$$\begin{aligned}\alpha_n &= p(z=n \mid \boldsymbol{X}\boldsymbol{q}) \\ &= \mathrm{softmax}\big(s(\boldsymbol{x}_n, \boldsymbol{q})\big) \\ &= \frac{\exp\big(s(\boldsymbol{x}_n, \boldsymbol{q})\big)}{\sum_{j=1}^{N} \exp\Big(s\big(\boldsymbol{x}_j, \boldsymbol{q}\big)\Big)}\end{aligned}$$

注意力机制

（3）上下文向量

上下文向量（Context Vector）是聚焦于任务相关信息的特征汇总表示。具体地，在计算时，通过注意力权重对输入向量加权求和，得到一个新的表示：

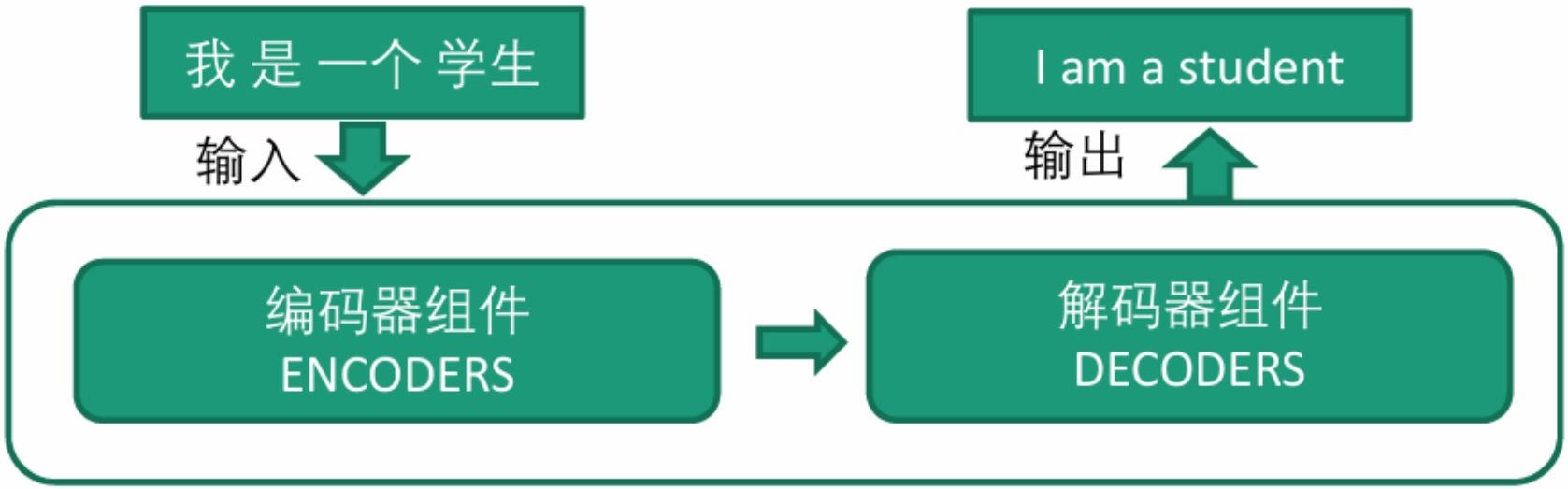
$$c = \sum_{n=1}^{N} \alpha_n x_n$$

α_n 是第 n 个输入向量的注意力权重， x_n 是输入向量。

Transformer

Transformer 是谷歌团队于 2017 年在论文《Attention is All You Need》中 提出的一种带有自注意力机制的编码器-解码器（seq2seq）神经网络。

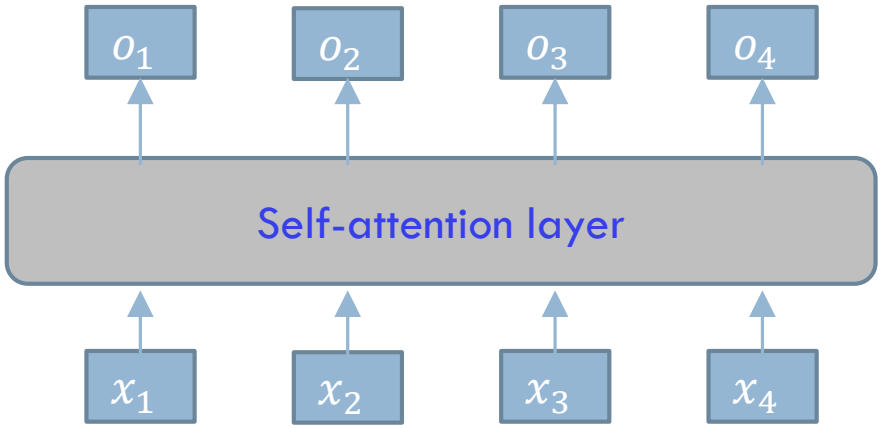
该论文中，Transformer模型用于机器翻译任务，达到了当时的主流效果。



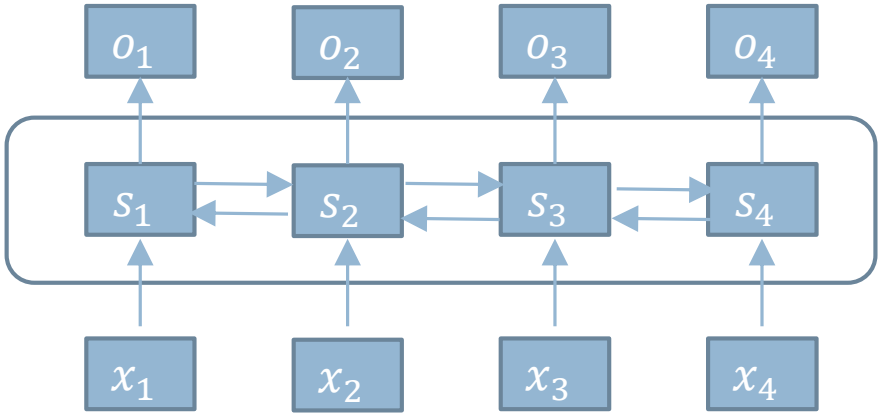
Transformer

- Transformer通过**自注意力机制（Self-Attention）** 近乎完美地解决了长依赖问题，其核心思想是：
  - **直接连接**：在计算当前位置的输出时，自注意力机制允许模型直接访问和关注到序列中的任何位置。
  - **路径长度恒定为1**：信息从一个位置到另一个位置的理论路径长度是**1**。
  - **无限上下文（理论上）**：只要计算资源允许，Transformer可以处理任意长的序列（尽管在实践中有约束）。

Transformer vs. RNN



Transformer



RNN

- RNN(Recurrent Neural Network)存在时间序列依赖的问题，即是一种循环网络，其训练上是基于迭代的处理模式。因而上一个时间步（例如输入语句中的词）处理完毕以后，才能处理下一个时间步（例如输入语句中的下一个词）。
- 而Transformer的训练可以实现并行，即序列中的单词可以同时训练，因而效率较高。其引入位置编码来处理单词的顺序。

自注意力模型

- 自注意力模型（self-attention model）可视为注意力机制的一个特殊形式：**在同一个序列内部计算注意力分布。**
- 通过自注意力机制，模型能够同时对输入序列中的所有位置进行加权求和，从而捕获序列中不同位置之间的关系。
- 自注意力机制的数学模型由四部分构成：**查询-键-值计算、相关性得分、注意力权重和上下文向量。**这四部分协同工作，最终生成一个任务相关的输出表示。

自注意力模型

（1）查询-键-值计算

查询-键-值计算将自注意力机制中的输入向量 $\boldsymbol{X} \in \mathbb{R}^{L\times D}$ 映射到查询向量$\boldsymbol{Q}$、键向量$\boldsymbol{K}$、值向量$\boldsymbol{V}$。这三个向量通过将输入向量进行线性变换得到：

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^{Q} \in \mathbb{R}^{L\times D_k} \ ,$$

$$\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^{K} \in \mathbb{R}^{L\times D_k} \ ,$$

$$\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^{V} \in \mathbb{R}^{L\times D_v} \ ,$$

自注意力模型

（2）相关性得分

和基本注意力机制类似。相关性得分可以通过查询和键之间的内积来计算：

$$\boldsymbol{score}(\boldsymbol{Q}, \boldsymbol{K}) = \boldsymbol{Q}\boldsymbol{K}^T$$

（3）注意力权重

注意力权重是基于相关性得分的归一化结果，使用**softmax**函数对相关性得分进行归一化处理。

$$\boldsymbol{\alpha} = \mathrm{softmax}(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{D}})$$

自注意力模型

（4）上下文向量

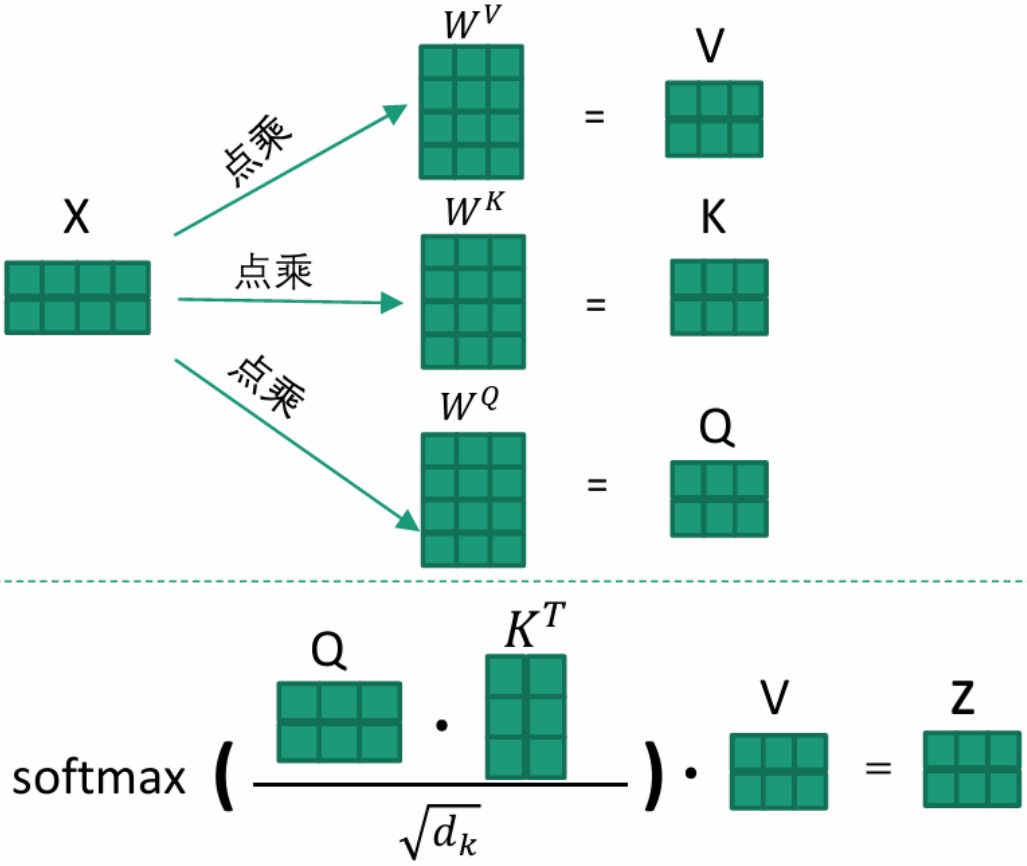
上下文向量$\boldsymbol{C}$是自注意力机制的最终输出，它包含了序列中所有位置的信息，并根据每个位置的重要性（由注意力权重α）决定对值向量进行加权，即：

$$\boldsymbol{C} = \mathbf{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \boldsymbol{\alpha V} = \mathrm{softmax}(\frac{\boldsymbol{Q K}^T}{\sqrt{D}})\boldsymbol{V}$$

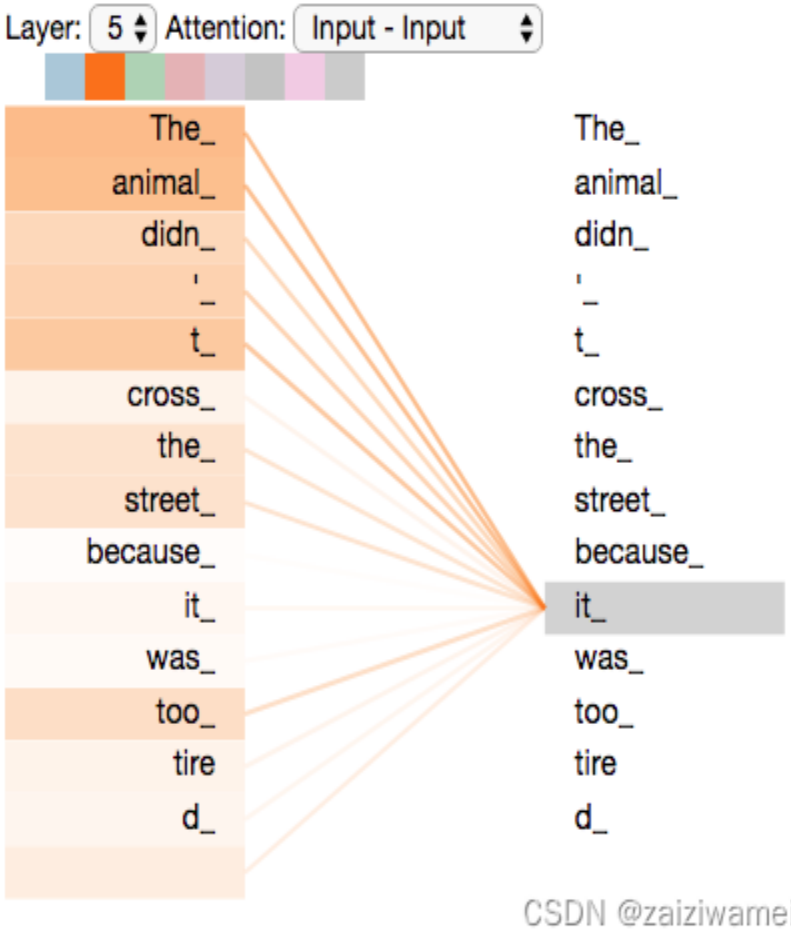
其中，$\boldsymbol{C} \in \mathbb{R}^{L \times D_v}$。

自注意力模型

通过矩阵运算实现自注意力模型：



自注意力模型



CSDN @zaiziwamei

图中不同连线表明：在同一个序列中，某个位置的token对不同位置有着不同的关注度

多头自注意力机制

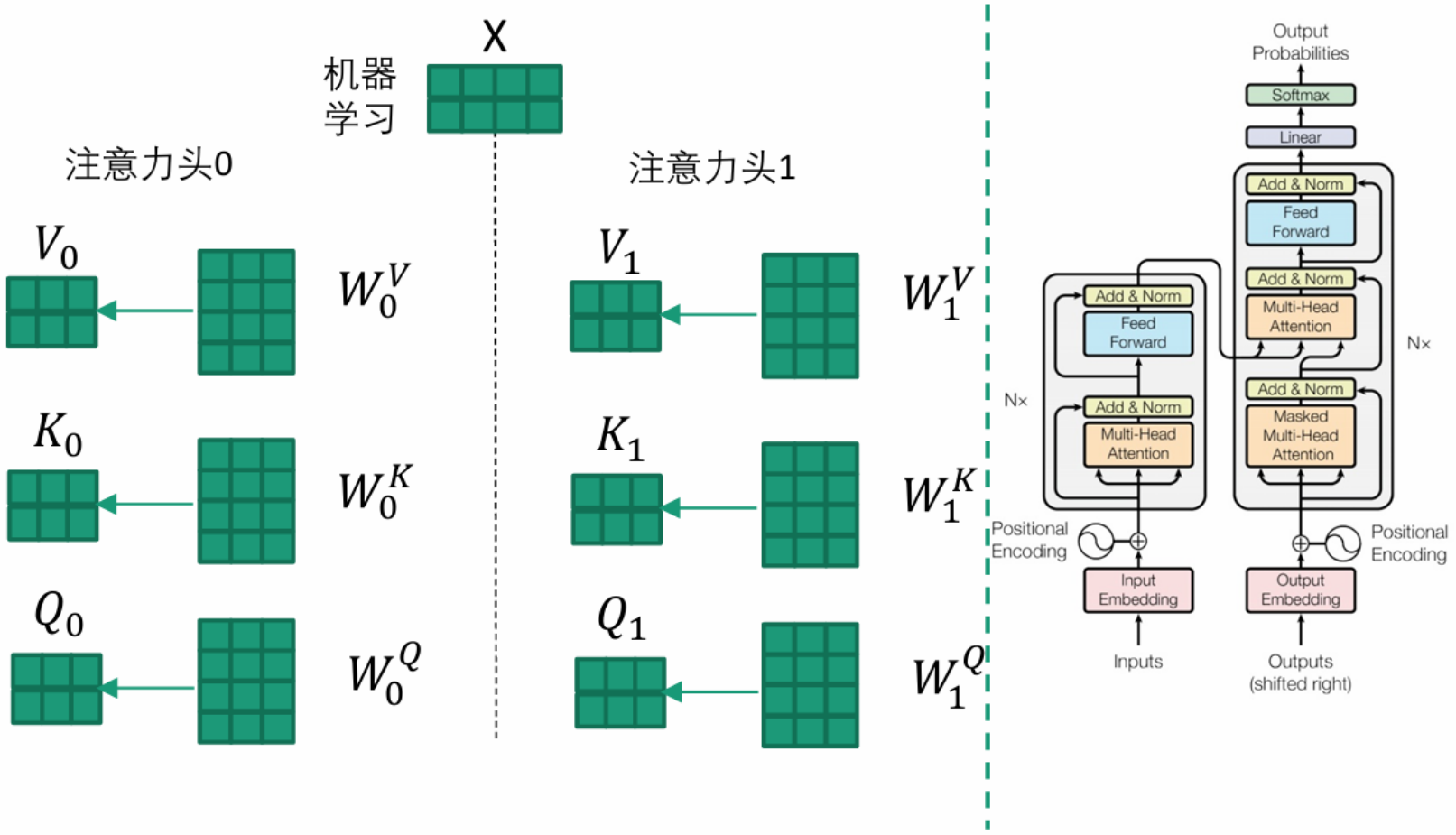
- 为了让模型从多个子空间中学习到信息，Transformer引入了多头自注意力机制(Multi-Head Self Attention)。
  - 多头自注意力通过并行计算多个自注意力的“头”(head)，并最终将各个头的输出拼接起来。具体计算方式如下：

$$\boldsymbol{C} = \mathrm{MultiHeadAttention}(\boldsymbol{X})$$

$$\triangleq Concat(head_1, \ldots, head_{\mathrm{j}}, \ldots head_H)\boldsymbol{W}$$

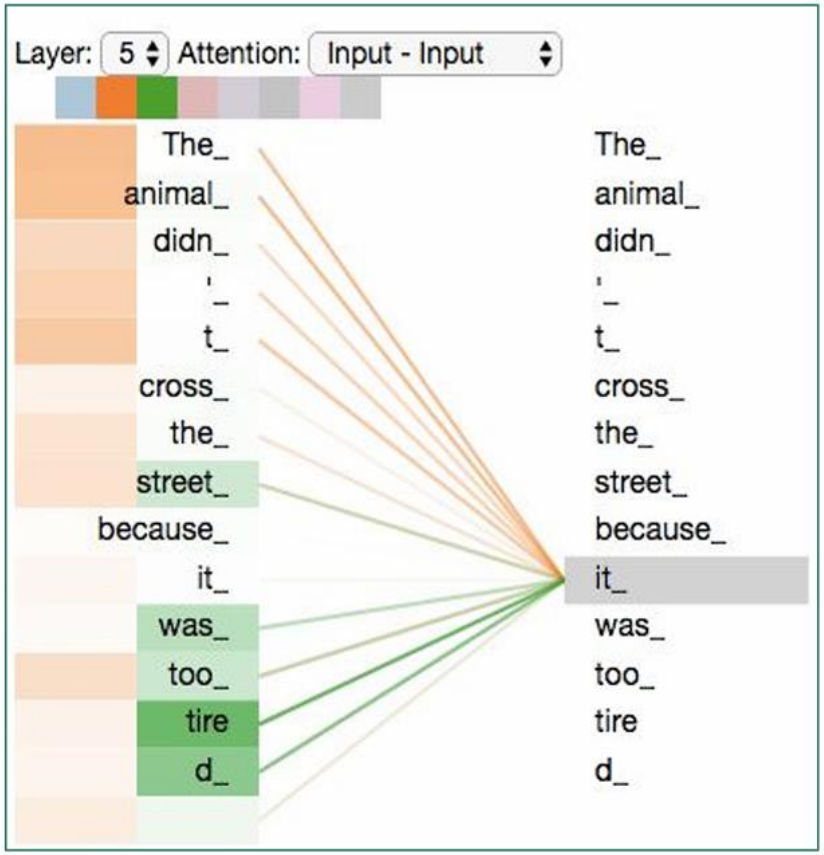
  - 其中，H 表示头的数量，$Concat$表示向量拼接运算。
  - 拼接后向量维度可能并不等于原始向量的维度D，因此这里乘以矩阵$\mathrm{W} \in \mathbb{R}^{(\mathrm{H}\times \mathrm{D_h})\times \mathrm{D}}$，将向量映射到原始的输入维度D。

多头自注意力机制

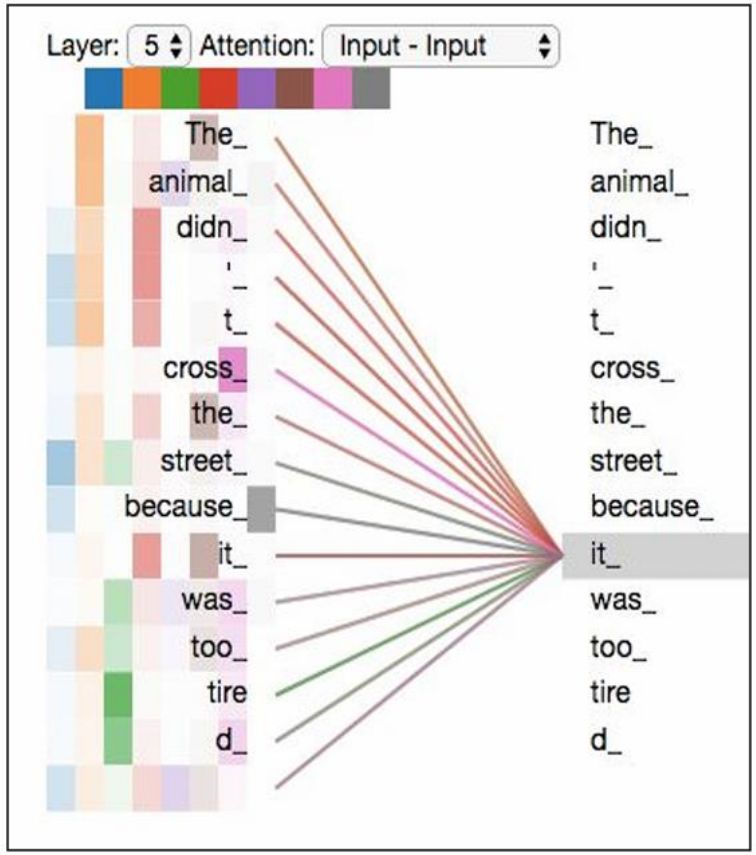

X
机器
学习
注意力头0
注意力头1
V_0
K_0
Q_0
W_0^V
W_0^K
W_0^Q
V_1
K_1
Q_1
W_1^V
W_1^K
W_1^Q
Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

多头自注意力机制

多头自注意力可视化：



第5层两个Head（黄绿两种颜色）：it关注的分别是animal和tied



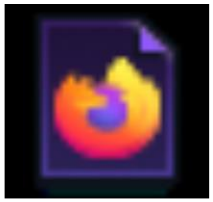
第5层8个Head：it最关注的结果放一起，缺乏明确的解释

注意力可视化图



- Bert训练结果，Layer指不同层，句子指的是前后两个分句，不同的色块分析对应每层切分的8个head，显得粗细表述注意力权重的大小

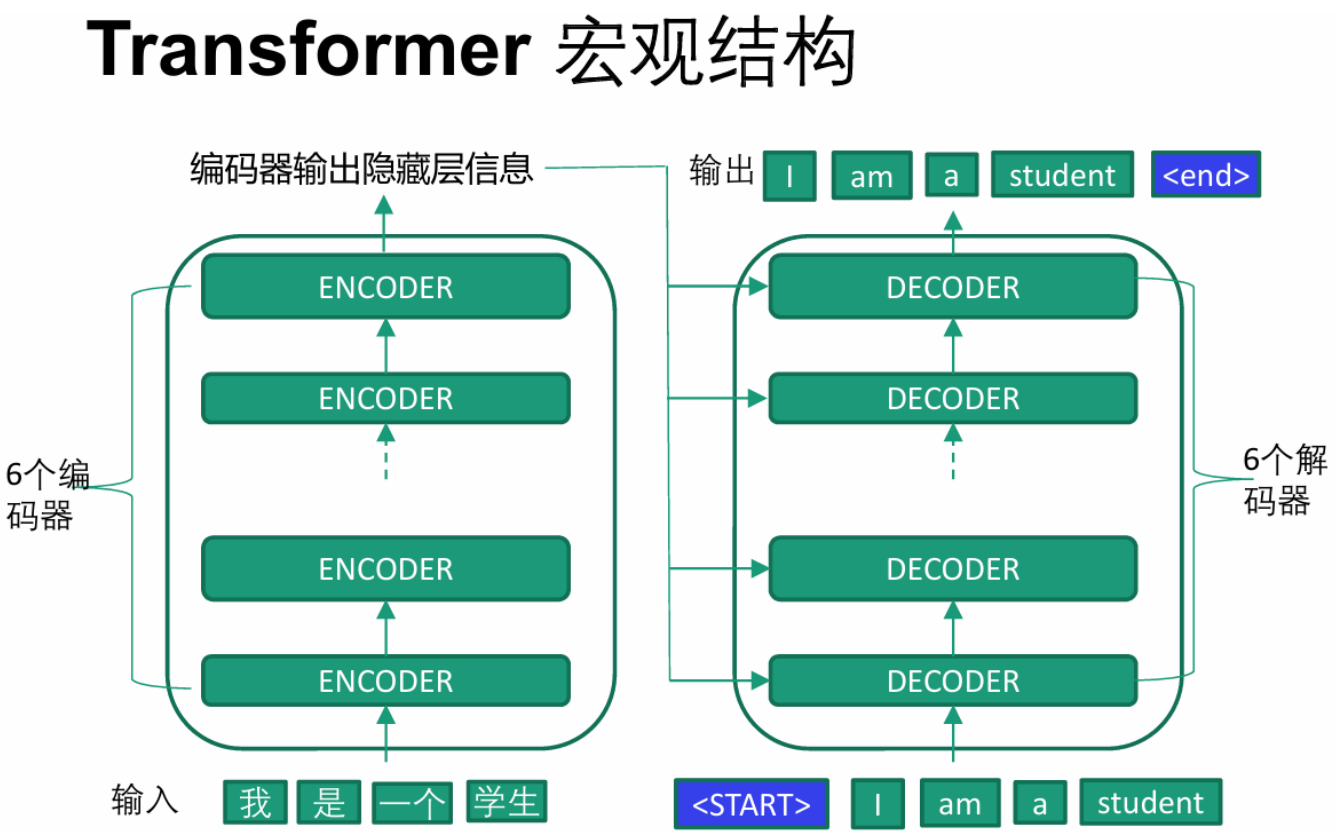
- 请使用课程群中的html文件自行检视



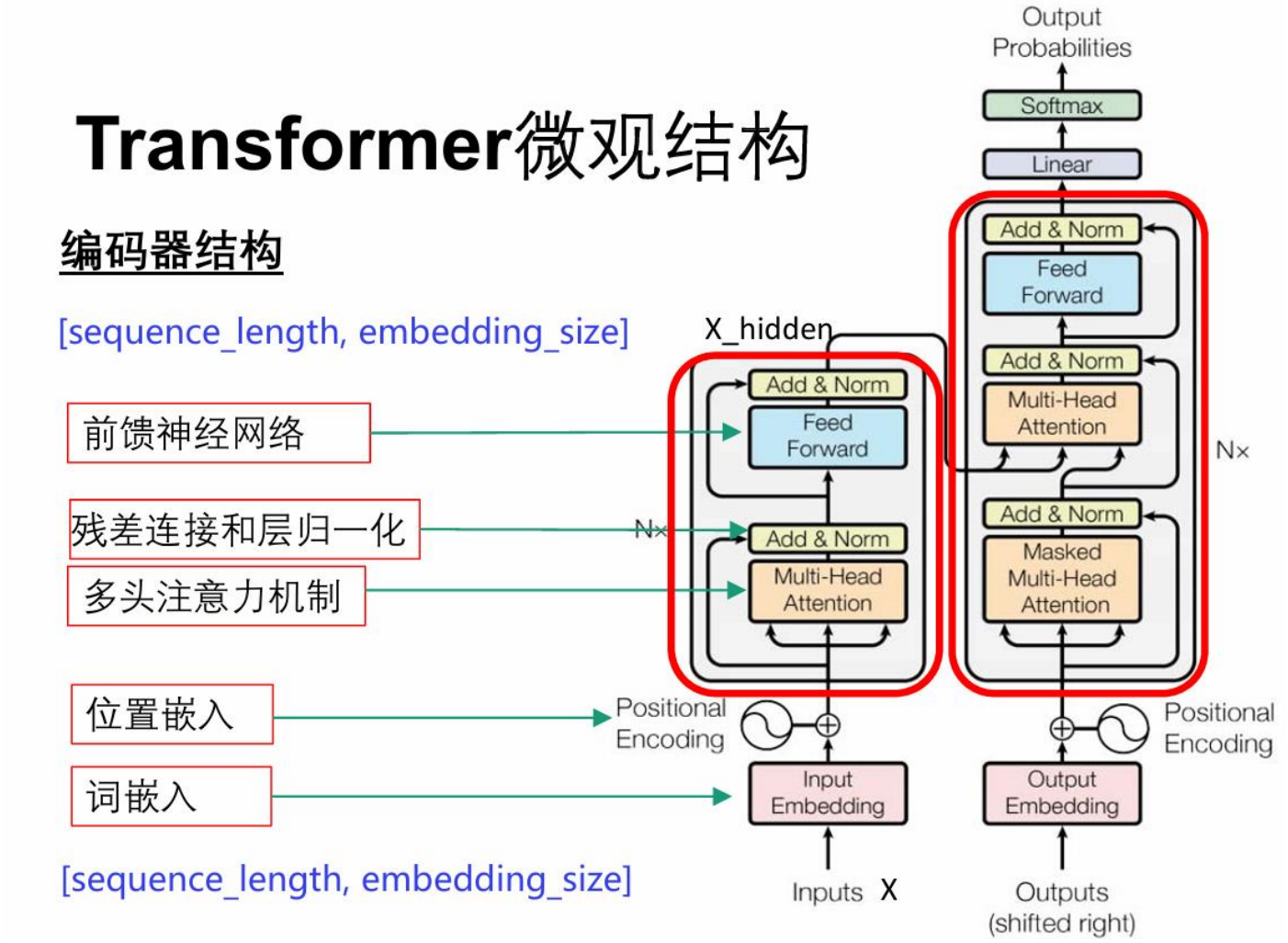
注意力可视化

Transformer架构

介绍了Transformer最核心的多头自注意力模块后，我们再来看它的具体架构：



Transformer（众所周知的图）





Step1： Input Embedding

将原文所有单词汇总统计频率，删除低频词汇，假设总共选出了M个单词
利用Xaviers初始化方法随机生成矩阵Matrix$_{M*N}$，M对应选出的单词数，N则为固定值，通常取N=512，矩阵如图所示，图中取M=10000

Word2Num一表			Matrix								
word	num		0	1	2	…	…	508	509	510	511
<Pad>	0	→	0.536	0.794	0.214	…	…	0.685	0.339	0.964	0.555
中	1	→	0.53	0.908	0.865	…	…	0.625	0.14	0.031	0.806
有	2	→	0.727	0.133	0.88	…	…	0.313	0.674	0.913	0.656
人	3	→	0.412	0.538	0.972	…	…	0.415	0.928	0.724	0.947
家	4	→	0.821	0.6	0.976	…	…	0.082	0.252	0.862	0.68
民	5	→	0.185	0.662	0.55	…	…	0.481	0.758	0.408	0.264
梦	6	→	0.373	0.782	0.958	…	…	0.765	0.191	0.375	0.556
国	7	→	0.099	0.623	0.165	…	…	0.905	0.308	0.412	0.171
…	…	→	…	…	…	…	…	…	…	…	…
…	…	→	…	…	…	…	…	…	…	…	…
云	9997	→	0.985	0.014	0.406	…	…	0.93	0.917	0.182	0.411
开	9998	→	0.924	0.314	0.149	…	…	0.802	0.763	0.362	0.918
加	9999	→	0.837	0.475	0.558	…	…	0.369	0.189	0.596	0.879



➢Step1：Input Embedding

随后，可以把每句话表示成矩阵形式，设句子长度为T，则矩阵维度=T*N
如“中国人有中国梦。”可以表示为下图所示,设为矩阵matX，
这里定义矩阵行数为10；第8行，即编号为100的一行可以理解为结束符，不足的补齐（padding）

	matX								
1	0.53	0.908	0.865	…	…	0.625	0.14	0.031	0.806
7	0.099	0.623	0.165	…	…	0.905	0.308	0.412	0.171
3	0.412	0.538	0.972	…	…	0.415	0.928	0.724	0.947
2	0.727	0.133	0.88	…	…	0.313	0.674	0.913	0.656
1	0.53	0.908	0.865	…	…	0.625	0.14	0.031	0.806
7	0.099	0.623	0.165	…	…	0.905	0.308	0.412	0.171
6	0.373	0.782	0.958	…	…	0.765	0.191	0.375	0.556
100	0.099	0.623	0.165	…	…	0.905	0.308	0.412	0.171
0	0.536	0.794	0.214	…	…	0.685	0.339	0.964	0.555
0	0.536	0.794	0.214	…	…	0.685	0.339	0.964	0.555

输入句表示矩阵matX

Word2Num-表	
word	num
<Pad>	0
中	1
有	2
人	3
家	4
民	5
梦	6
国	7
…	…
…	…
云	9997
开	9998
加	9999

Matrix								
0	1	2	…	…	508	509	510	511
0.536	0.794	0.214	…	…	0.685	0.339	0.964	0.555
0.53	0.908	0.865	…	…	0.625	0.14	0.031	0.806
0.727	0.133	0.88	…	…	0.313	0.674	0.913	0.656
0.412	0.538	0.972	…	…	0.415	0.928	0.724	0.947
0.821	0.6	0.976	…	…	0.082	0.252	0.862	0.68
0.185	0.662	0.55	…	…	0.481	0.758	0.408	0.264
0.373	0.782	0.958	…	…	0.765	0.191	0.375	0.556
0.099	0.623	0.165	…	…	0.905	0.308	0.412	0.171
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…
0.985	0.014	0.406	…	…	0.93	0.917	0.182	0.411
0.924	0.314	0.149	…	…	0.802	0.763	0.362	0.918
0.837	0.475	0.558	…	…	0.369	0.189	0.596	0.879

初始词表矩阵



Step2：Position Encoding

Transformer模型并没有捕捉顺序序列的能力，也就是说无论句子结构怎么打乱，都会得到类似的结果。

为解决这个问题，模型在编码词向量时引入了位置编码的特征：在词向量中加入了单词的位置信息

通常位置编码是一个长度为d_{model}的特征向量，以便于和词向量进行单位加的操作，编码公式如下：↑表示后面是指数| pos表示单词的位置， i表示词向量第i个维度。得到的矩阵设为matP

$$PE(pos, 2i) = \sin(\frac{pos}{10000 \uparrow \frac{2i}{d_{model}}})$$

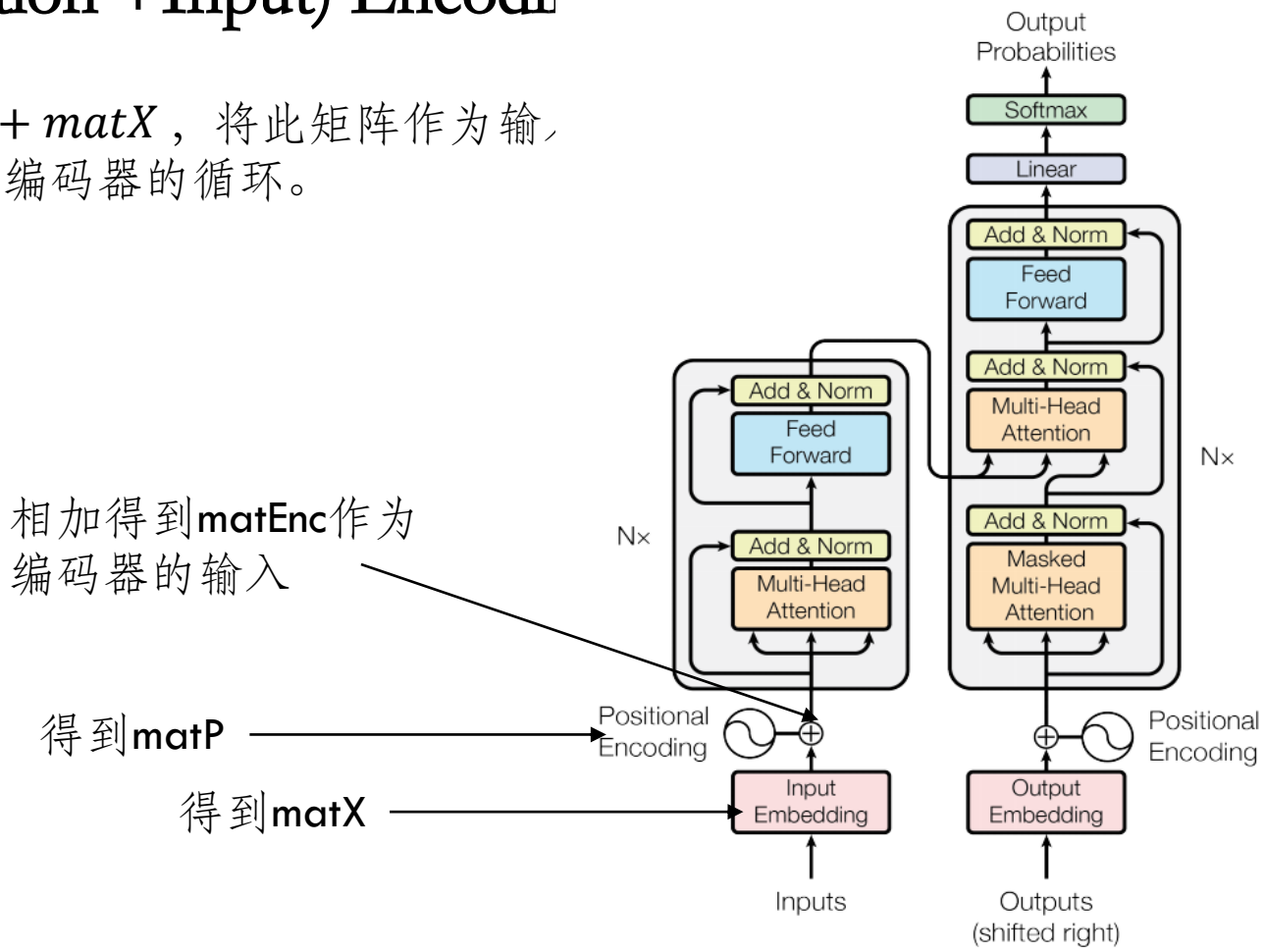
$$PE(pos, 2i+1) = \cos(\frac{pos}{10000 \uparrow \frac{2i}{d_{model}}})$$

position		0	1	2	3	4	…	510	511
0	→	f(0, 0)	f(0, 1)	f(0, 2)	f(0, 3)	f(0, 4)	…	f(0, 510)	f(0, 511)
1	→	f(1, 0)	f(1, 1)	f(1, 2)	f(1, 3)	f(1, 4)	…	f(1, 510)	f(1, 511)
2	→	f(2, 0)	f(2, 1)	f(2, 2)	f(2, 3)	f(2, 4)	…	f(2, 510)	f(2, 511)
3	→	f(3, 0)	f(3, 1)	f(3, 2)	f(3, 3)	f(3, 4)	…	f(3, 510)	f(3, 511)
4	→	f(4, 0)	f(4, 1)	f(4, 2)	f(4, 3)	f(4, 4)	…	f(4, 510)	f(4, 511)
5	→	f(5, 0)	f(5, 1)	f(5, 2)	f(5, 3)	f(5, 4)	…	f(5, 510)	f(5, 511)
6	→	f(6, 0)	f(6, 1)	f(6, 2)	f(6, 3)	f(6, 4)	…	f(6, 510)	f(6, 511)
7	→	f(7, 0)	f(7, 1)	f(7, 2)	f(7, 3)	f(7, 4)	…	f(7, 510)	f(7, 511)
8	→	f(8, 0)	f(8, 1)	f(8, 2)	f(8, 3)	f(8, 4)	…	f(8, 510)	f(8, 511)
9	→	f(9, 0)	f(9, 1)	f(9, 2)	f(9, 3)	f(9, 4)	…	f(9, 510)	f(9, 511)

输入句位置编码矩阵matP

Step2：(Position +Input) Encodi~

令$matEnc = matP + matX$，将此矩阵作为输入进入Transformer模型编码器的循环。





➢Step3：Multi head attention

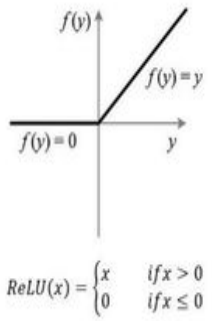
- 多头注意力机制：多个self-attention的结合
  - 每个head学习到在不同表示空间中的特征

  如图所示，两个head学习到的attention侧重点可能不同，这给了模型更大的容量。
- 计算过程与self-attention基本相同
- 在开始和结尾多出了切分和合成部分

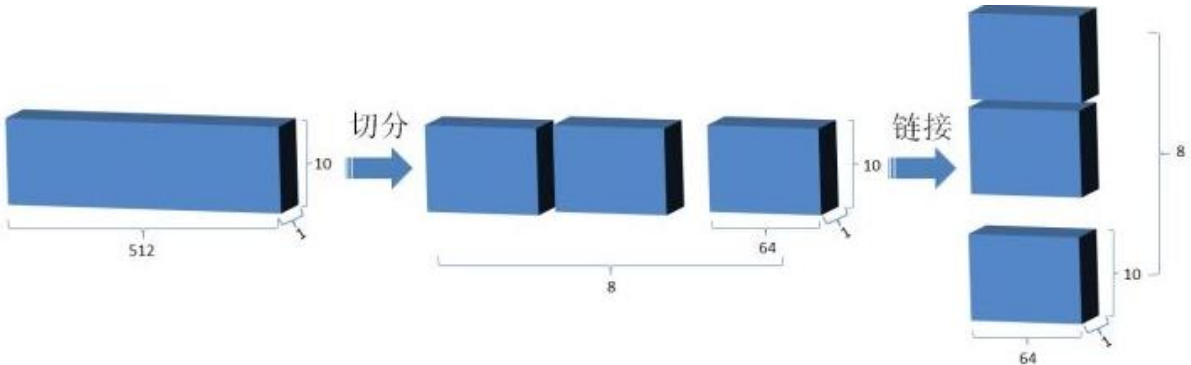
  我们以head数=8为例说明。

Step3：Multi head attention

1. 引入三个向量Q、K、V，初始值Q=K=V=matEnc（后面经过变化会不一样）
2. 首先分别对Q、K、V经过线性变换，即将三者分别输入到三个单层神经网络，激活函数选择relu，输出新的Q、K、V（经过线性变换三者的维度不变）
3. 对Q的N维进行切分，整齐切分为8段(可以是任意段，但必须被N整除), 得到8个64*10的矩阵，如下图所示，multihead attention实际上可看作是由8个self-attention连接而成。
4. 对V和K施以和Q同样的操作。



ReLu：单侧抑制





切分后的Q如下图为例：

Q							
0	1	2	…	508	509	510	511
0.065	0.781	0.684	…	0.39	0.719	0.668	0.868
0.909	0.137	0.856	…	0.399	0.045	0.835	0.015
…	…	…	…	…	…	…	…
0.611	0.77	0.055	…	0.404	0.74	0.583	0.992

	Q							
	0	1	2	…	60	61	62	63
Head-1	0.065	0.781	0.684	…	0.474	0.551	0.304	0.604
	0.909	0.137	0.856	…	0.145	0.975	0.406	0.245
	…	…	…	…	…	…	…	…
	0.611	0.77	0.055	…	0.771	0.124	0.715	0.858
Head-2	0.895	0.279	0.318	…	0.565	0.934	0.922	0.78
	0.284	0.399	0.803	…	0.257	0.259	0.786	0.046
	…	…	…	…	…	…	…	…
	0.804	0.28	0.239	…	0.353	0.105	0.286	0.218
…	…	…	…	…	…	…	…	…
Head-7	0.575	0.283	0.308	…	0.315	0.85	0.029	0.345
	0.243	0.25	0.626	…	0.012	0.645	0.067	0.028
	…	…	…	…	…	…	…	…
	0.116	0.811	0.159	…	0.695	0.846	0.841	0.148
Head-8	0.595	0.998	0.764	…	0.39	0.719	0.668	0.868
	0.032	0.691	0.194	…	0.399	0.045	0.835	0.015
	…	…	…	…	…	…	…	…
	0.319	0.849	0.031	…	0.404	0.74	0.583	0.992



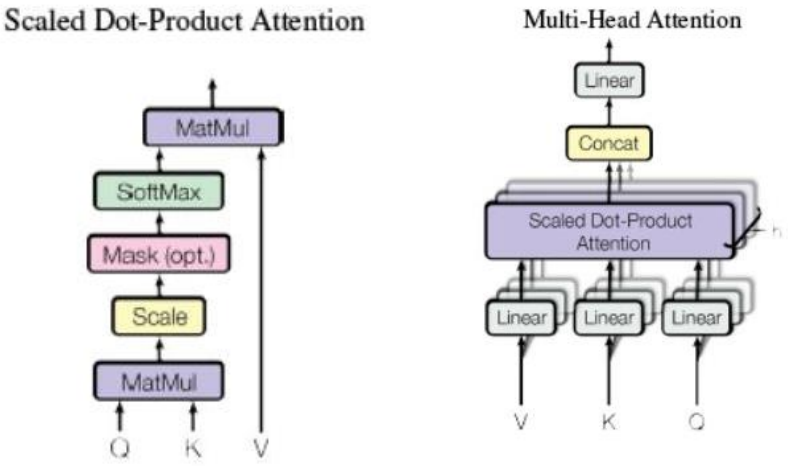
Step3：Multi head attention

4.对得到的8个Q_{10*64}和K_{10*64}进行运算score= $Q \cdot K^T$，得到新的10*10维矩阵

5.为了梯度的稳定，使score归一化，即score除以$\sqrt{d_k}$，若key的维数d_k特别大，可能点积变得很大导致softmax函数进入一个梯度很小的范围。

6.对score施以$softmax$激活函数

7.将归一化后的函数点乘V_{10*64}，得到10*64的矩阵，再将八个该矩阵做连接操作，得到维度为10*512的矩阵



整个过程即公式

$$Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Step4：add&norm求和和归一化

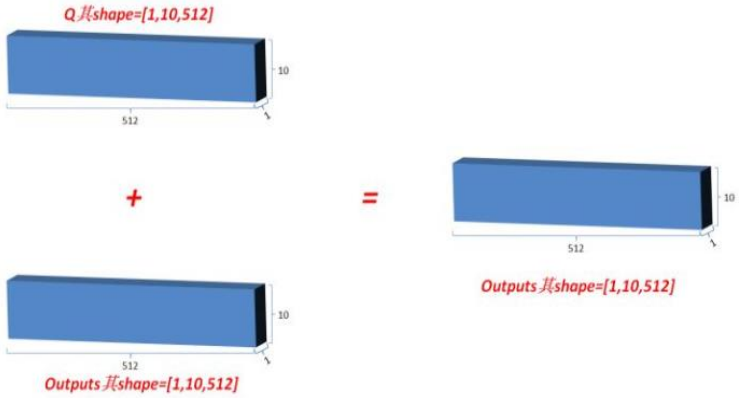
add实际上是为了使网络有效叠加，避免梯度消失，也就是残差网络的解决办法：

$Attention(Q,K,V)= Attention(Q,K,V)+Q$

Norm是标准化矫正一次，意图在不改变矩阵权重的情况下实现矩阵的独立同分布：

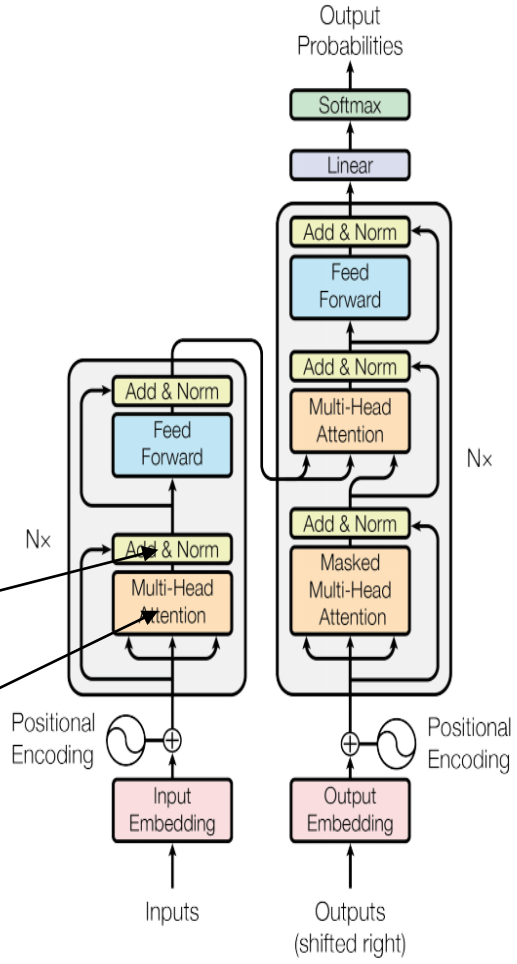
$$h = f(g \cdot \frac{x-\mu}{\sigma} + b)$$

其中μ、σ分别为$Attention(Q,K,V)$在512维的均值和方差



矩阵求和和归一化

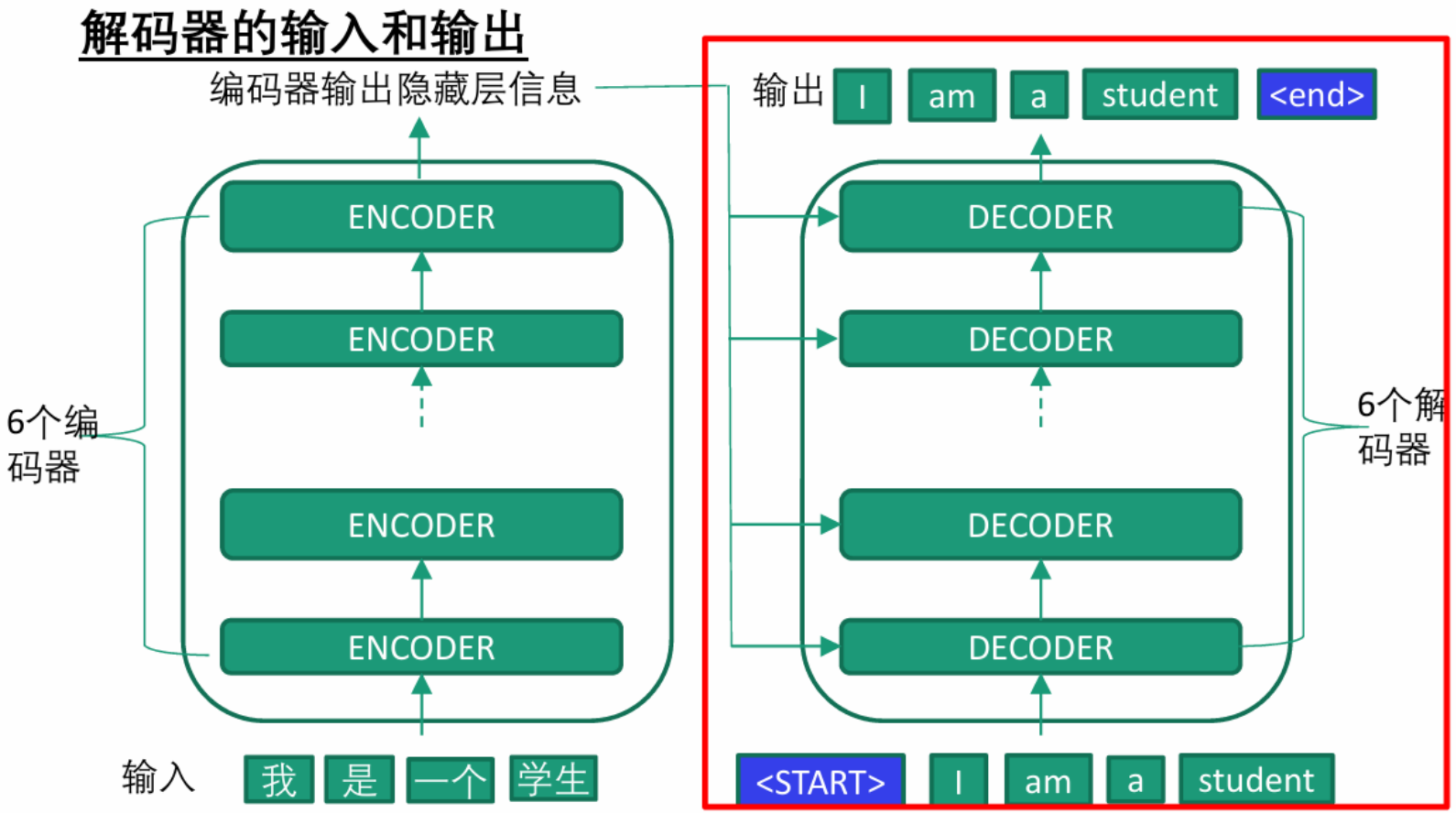
得到$Attention(Q,K,V)$



Transformer



Transformer解码器工作流程



Transformer

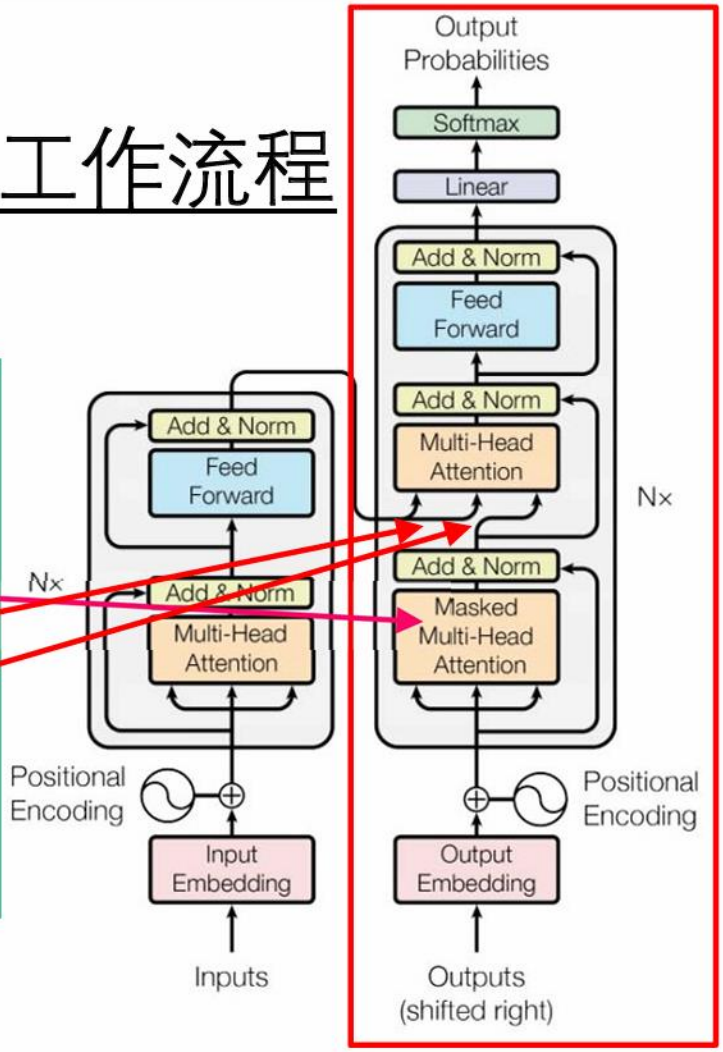


Transformer解码器工作流程

解码器

有两个多头注意力层(Multi-Head Attention)：

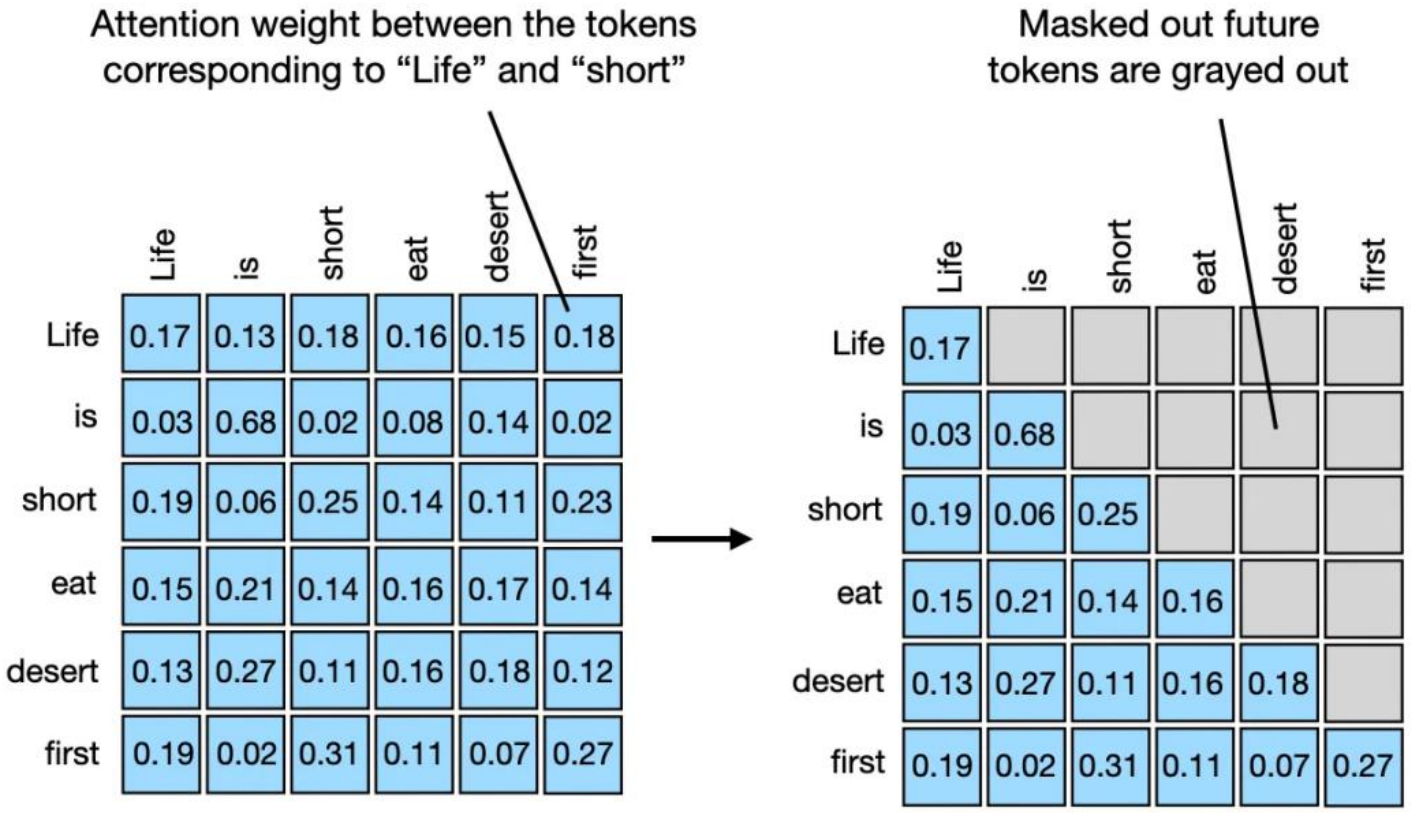
- 第一个 Multi-Head Attention 层采用了 **Masked 操作**。
- 第二个 Multi-Head Attention 层的 **K, V** 矩阵使用编码器encoder输出的编码信息矩阵计算，而 **Q** 则来自前一位置解码器decoder的输出。



Transformer



Masked掩码：确保任何一个词看不到其后的词

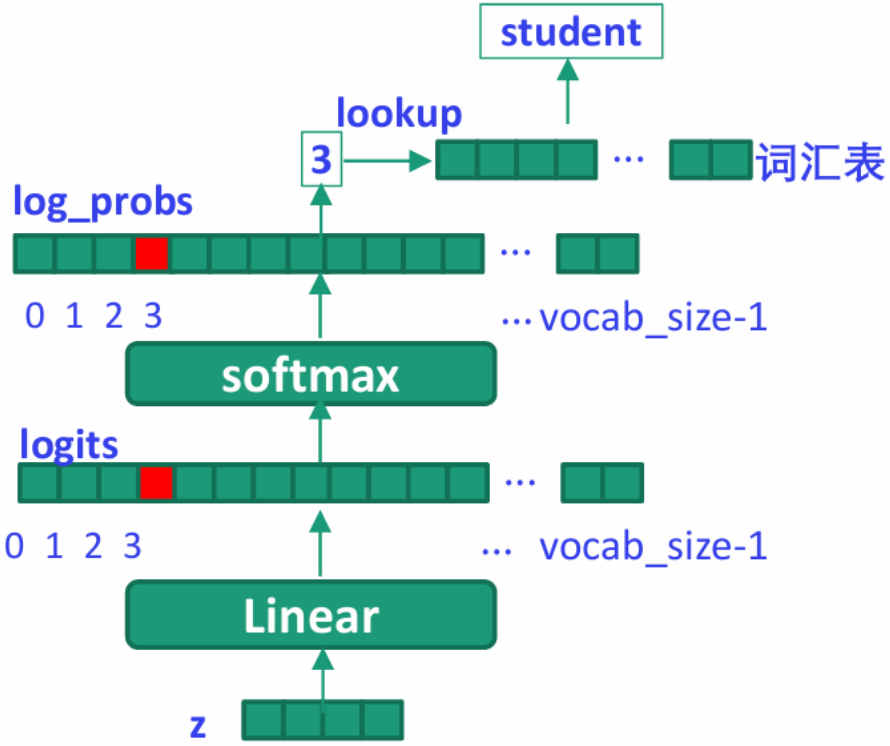


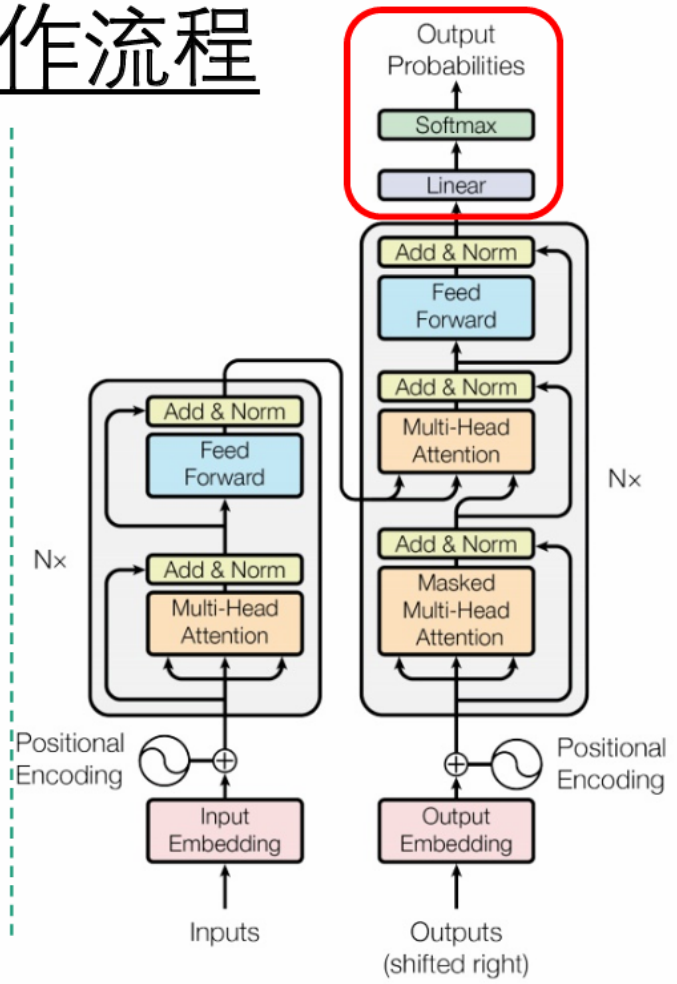
Transformer



Transformer解码器工作流程

最终的线性变换和Softmax层





课后习题

- 人工神经网络中的神经元模型是如何从生物神经元中汲取灵感的？除了基本的输入-权重-输出模式外，还有哪些生物特性（如突触可塑性、抑制与兴奋机制）在现有的人工神经元设计中尚未得到充分体现，你认为未来如何可以进一步融合这些生物特性以增强神经网络的性能？
- 激活函数是神经网络中神经元输出前的非线性变换，对网络的性能和学习能力至关重要。请列举几种常见的激活函数（如ReLU、Sigmoid、Tanh），并分析它们各自的优缺点。思考在实际应用中，如何根据具体问题选择合适的激活函数，以及激活函数的选择如何影响网络的训练效率和最终性能。
- 分析为什么分类问题常采用交叉熵损失函数而不是平方损失函数？
- 循环神经网络相比前馈神经网络在处理序列数据（如时间序列分析、自然语言处理）时的主要优势是什么？然而，RNN在处理长序列时面临的“梯度消失”或“梯度爆炸”问题是如何产生的？探讨LSTM和GRU等改进模型是如何通过引入门控机制来解决这些问题的，并思考是否还有其他创新思路可以进一步优化RNN的性能。
- 分析为什么Transformer中的自注意力机制选择了点积模型而不是加性模型？